

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Danh - Lý Trường Nam - Phạm
Tân Nghĩa - Hà Gia Phúc - Nguyễn Trường
Tân - Trần Nhật Tân

HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC
TẬP CHO SINH VIÊN SỬ DỤNG AI
AGENT

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 04/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Danh - 22120048

Lý Trường Nam - 22120218

Phạm Tấn Nghĩa - 22120230

Hà Gia Phúc - 22120272

Nguyễn Trường Tân - 22120326

Trần Nhật Tân - 22120328

**HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC
TẬP CHO SINH VIÊN SỬ DỤNG AI
AGENT**

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN

TS. Nguyễn Trần Minh Thư

Tp. Hồ Chí Minh, tháng 04/2026

Lời cam đoan

Nhóm xin cam đoan Dự án “Hệ thống hỗ trợ thông tin học tập cho sinh viên sử dụng AI AGENT” là công trình nghiên cứu độc lập của nhóm, với sự hướng dẫn tận tình của cô Nguyễn Trần Minh Thư.

Công trình được nhóm nghiên cứu và hoàn thành tại Trường Đại học Khoa học Tự nhiên từ tháng 01 năm 2026 đến tháng 6 năm 2026. Các tài liệu tham khảo, các số liệu thống kê phục vụ mục đích nghiên cứu công trình này được sử dụng đúng quy định, không vi phạm quy chế bảo mật của Nhà nước.

Những số liệu, bảng biểu phục vụ cho việc phân tích và dẫn dắt đề tài này được thu thập từ các nguồn tài liệu khác nhau được trích dẫn một cách đầy đủ, đồng thời ghi rõ ràng về nguồn gốc theo quy định của nhà trường.

Ngoài ra, đối với các tài liệu diễn giải để làm rõ thêm các luận điểm đã phân tích và trích dẫn trong phần phụ lục cũng được chú thích nguồn gốc dữ liệu. Nếu những lời cam đoan trên của nhóm không chính xác, nhóm xin chịu hoàn toàn trách nhiệm và chịu mọi hình thức kỷ luật từ khoa và nhà trường

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Dự án thực tập tốt nghiệp chuyên ngành Hệ Thống Thông Tin trên đánh dấu cột mốc hoàn thành.

Chúng em xin gửi lời cảm ơn chân thành đến Quý Ban Giám hiệu và toàn thể cán bộ giảng viên của Trường Đại học Khoa học Tự nhiên - Đại học Quốc gia TP.HCM, vì đã cung cấp cho chúng em môi trường học tập và nghiên cứu tuyệt vời trong suốt khoảng thời gian học tập và làm việc tại nhà trường.

Và chúng em xin chân thành cảm ơn sâu sắc dành cho cô Nguyễn Trần Minh Thư vì sự tận tâm, kiến thức sâu rộng và sự hỗ trợ quý báu trong suốt quá trình thực hiện đồ án tốt nghiệp. Cô đã dành thời gian và nỗ lực để chỉ dẫn, định hướng và đánh giá công việc của chúng em. Sự tận tâm và kiến thức chuyên môn của cô đã giúp chúng em vượt qua những khó khăn và phát triển kỹ năng nghiên cứu và thực hiện đồ án của mình.

Mặc dù chúng em đã cố gắng hoàn thành dự án thực tập tốt nghiệp với kết quả tốt nhất có thể trong khả năng, nhưng vẫn có những khiếm khuyết, sai sót nhất định do thiếu kinh nghiệm thực tế. Chúng em mong nhận được sự góp ý chân thành và dạy bảo của quý Thầy Cô để có thể hoàn thiện bản thân hơn nữa.

Lời cảm ơn cuối cùng dành cho gia đình, bạn bè và những người thân yêu đã luôn động viên và đồng hành cùng chúng em trong suốt quá trình nghiên cứu và thực hiện đồ án tốt nghiệp. Sự ủng hộ và động viên của họ là nguồn động lực vô cùng quan trọng giúp chúng em vượt qua khó khăn và hoàn thành công việc một cách tốt nhất.

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó (*phải có chữ ký của giảng viên hướng dẫn*).

Mục lục

Lời cam đoan	i
Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vi
Mục lục	vii
Danh sách hình	xi
Danh sách bảng	xii
Tóm tắt	xii
1 Giới thiệu	1
2 Các công trình liên quan	3
2.1 Tổng quan về hệ thống hỗ trợ học tập thông minh	3
2.1.1 Các hướng tiếp cận truyền thống	3
2.1.2 Ứng dụng AI trong giáo dục hiện đại	4

2.1.3	Ứng dụng AI trong giáo dục hiện đại	6
2.2	Mô hình ngôn ngữ lớn (LLMs) trong giáo dục	6
2.2.1	Ứng dụng LLM trong hệ thống hỏi đáp học tập	6
2.2.2	Hạn chế của LLM (Hallucination, thiếu context, v.v.)	7
2.2.3	Hạn chế của LLM (Hallucination, thiếu context, v.v.)	8
2.3	Retrieval-Augmented Generation (RAG)	9
2.3.1	Kiến trúc RAG truyền thống	9
2.3.2	Các thành phần chính: Retriever – Generator	10
2.3.3	Ưu điểm và hạn chế của Naive RAG	11
2.3.4	Các thành phần chính: Retriever – Generator	12
2.3.5	Ưu điểm và hạn chế của Naive RAG	12
2.4	GraphRAG và Knowledge Graph	13
2.4.1	Tổng quan về Knowledge Graph	13
2.5	Các hệ thống RAG dựa trên Đồ thị tri thức	14
2.6	Lý do lựa chọn hệ thống LightRAG	15
2.7	Hiện trạng của LightRAG	16
2.8	Hiện trạng của LightRAG (Phiên bản 1.4.11)	17
2.9	Hệ thống Đa tác nhân (Multi-Agent Systems) và Luồng suy luận (Agentic Workflow)	19
2.9.1	Khái niệm AI Agent	19
2.9.2	Tích hợp LightRAG và Multi-Agent	20
2.10	Đánh giá hệ thống RAG và Multi-Agent	20
2.10.1	Các chỉ số đánh giá RAG (RAGAS)	20
2.10.2	Đánh giá hệ thống đa agent (Multi-Agent Evaluation)	21
2.10.3	Các phương pháp benchmark hiện nay	22
2.10.4	Đánh giá hệ thống đa agent (Multi-Agent Evaluation)	23
2.10.5	Các phương pháp benchmark hiện nay	23
3	Phương pháp đề xuất	24
3.1	Quy trình xử lý dữ liệu	25
3.1.1	Tiền xử lý dữ liệu tri thức	26

3.1.2	Luồng logic xử lý và cơ chế kiểm tra trùng lặp	28
3.1.3	Chiến lược phân đoạn và chuẩn hóa văn bản	32
3.1.4	Hệ thống thu thập tri thức từ trang web	38
3.1.5	Quy trình điều phối và phân đoạn dữ liệu	40
3.1.6	Cơ chế trích xuất thực thể và xây dựng quan hệ	42
3.1.7	Luồng xử lý Prompt linh hoạt cho đa dạng nguồn dữ liệu	45
3.1.8	Kiểm chứng tri thức bằng chuẩn SHACL và cơ chế Staging	46
3.1.9	Hợp nhất đồ thị và chuẩn hóa kho tri thức	47
3.1.10	Quản lý và duy trì bền vững hệ thống	49
3.2	Mô hình hóa trạng thái hệ thống (Agent State)	51
3.2.1	Kiến trúc các tác vụ (Agents)	52
3.2.2	Quy trình điều phối và Luồng dữ liệu	52
3.2.3	Cơ chế truy xuất tri thức (LightRAG Integration)	53
3.2.4	Thuật toán điều phối (Pseudocode)	53
4	Thiết kế và xây dựng ứng dụng	55
4.1	Ý tưởng ứng dụng	55
4.1.1	Các tình huống sử dụng tiêu biểu	55
4.1.2	Đối tượng người dùng mục tiêu	56
4.2	Phân tích và đặc tả yêu cầu	56
4.2.1	Yêu cầu chức năng	56
4.2.2	Yêu cầu phi chức năng	57
4.2.3	Danh sách Actor	58
4.3	Mô hình Use Case	58
4.3.1	Danh sách Use Case	59
4.3.2	Đặc tả Use Case	59
4.4	Kiến trúc hệ thống	63
4.4.1	Tầng Giao diện (Client Layer)	64
4.4.2	Tầng Nghiệp vụ (Application Layer)	65

4.4.3	Tầng Điều phối AI (AI Processing Layer)	65
4.4.4	Tầng Dữ liệu và Hạ tầng (Data & Infrastructure Layer)	66
4.5	Thiết kế chi tiết các thành phần	66
4.5.1	Thiết kế cơ sở dữ liệu	67
4.5.2	Kiến trúc triển khai lưu trữ cho LightRAG	74
4.6	Thiết kế giao diện	76
4.6.1	Màn hình AI Chat	76
5	Kết quả thí nghiệm	78
5.1	Đánh giá chất lượng MinerU trên tập tài liệu đầu vào . . .	78
5.2	Đánh giá chất lượng đồ thị tri thức	81
5.2.1	Tổng quan và Lý do lựa chọn bộ tiêu chí đánh giá .	82
5.2.2	Kết quả đánh giá thực nghiệm	85
5.3	Đánh giá chất lượng truy xuất và sinh văn bản (RAG Metrics)	88
5.4	Đánh giá hiệu năng điều phối Đa tác nhân (Agentic Metrics)	89
5.5	Đánh giá hệ thống từ phía người dùng cuối	89
6	Kết luận	90
6.1	Kết quả đạt được	90
6.2	Hạn chế của hệ thống	91
6.3	Hướng phát triển trong tương lai	91
	Danh mục công trình của tác giả	93
	Tài liệu tham khảo	94
	A Ngữ pháp tiếng Việt	99
	B Ngữ pháp tiếng Nôm	100

Danh sách hình

3.1	Quy trình tổng quát từ văn bản thô đến đồ thị tri thức . . .	25
3.2	Quy trình xử lý Hybrid-Auto-Engine trong MinerU mineru_2024	27
3.3	Luồng logic kiểm tra trùng lặp bằng cơ chế băm nội dung .	29
3.4	Chi tiết kiến trúc hệ thống VMiner hỗ trợ đa Backend . . .	30
3.5	Sơ đồ hoạt động của hệ thống MarkItDown	31
3.6	Quy trình thu thập tri thức từ trang web với Smart Scraper	38
3.7	Luồng logic xử lý từ dữ liệu thô đến giai đoạn phân đoạn .	41
3.8	Luồng logic quy trình trích xuất tri thức và cơ chế Gleaning	44
3.9	Quy trình truy vấn đa chế độ và cơ chế tối ưu phản hồi trong LightRAG	50
4.1	Sơ đồ Use Case hệ thống	58
4.2	Mô hình kiến trúc hệ thống tổng thể	64
4.3	Mô hình sơ đồ cơ sở dữ liệu quan hệ người dùng	67
4.4	Mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ	70
4.5	Ví dụ subgraph trích xuất từ đề cương môn học trong Knowl- edge Graph	73
4.6	Tổng quan kiến trúc lưu trữ và triển khai LightRAG . . .	74
4.7	Giao diện Màn hình AI Chat	76

Danh sách bảng

3.1	So sánh hai chiến lược phân đoạn trong hệ thống	36
4.1	Danh sách yêu cầu chức năng	57
4.2	Yêu cầu phi chức năng	57
4.3	Danh sách Actor	58
4.4	Danh sách Use Case	59
4.8	Danh sách các bảng trong cơ sở dữ liệu quan hệ	68
4.9	Mô tả khóa ngoại của các bảng trong CSDL quan hệ	68
4.10	Danh sách các Collection trong cơ sở dữ liệu hcmus_langgraph	69
4.11	Thuộc tính của node thực thể trong Knowledge Graph (Neo4j)	72
4.12	Thuộc tính của relationship trong Knowledge Graph (Neo4j)	73
4.13	Các thành phần lưu trữ trong kiến trúc LightRAG	75
5.1	Kết quả chi tiết đánh giá MinerU trên các tập tài liệu Chương trình đào tạo (2022-2025)	80
5.2	Tổng hợp kết quả đánh giá đồ thị tri thức trên các cấp độ lấy mẫu khác nhau	87

Tóm tắt

Nhiều sinh viên của trường Đại học Khoa học Tự nhiên – ĐHQG TPHCM đang học tập và làm việc trong tình trạng không có định hướng cho bản thân. Hệ thống hỗ trợ cung cấp thông tin học tập ra đời sẽ giúp sinh viên có góc nhìn tổng quan hơn trong con đường học vấn. Hệ thống sẽ đưa ra được các thông tin học tập theo chuyên ngành hiện tại, mong muốn sau này của sinh viên, gợi ý lộ trình học tập rõ ràng, chi tiết để đạt được hiệu quả học tập tốt nhất trong quá trình học. Nghiên cứu này áp dụng Knowledge Graph (KG) để lưu các thông tin liên quan tới học tập, kết hợp AI Agent hỗ trợ trong mapping các tri thức thành đồ thị và các AI Agent chuyên biệt cho quá trình tìm kiếm thông tin học tập, phân tích, đánh giá và đưa ra kết quả tối ưu nhất cho người dùng. Phương pháp, đánh giá và kết quả (Viết thêm sau khi hoàn thành)

Chương 1

Giới thiệu

Trong bối cảnh hiện nay sinh viên ngành Công nghệ thông tin tại Trường Đại học Khoa học Tự nhiên đang phải đối mặt với một lượng kiến thức cực kỳ lớn và rời rạc. Tuy tài liệu có sẵn rất nhiều trên mạng nhưng chúng thường tồn tại ở dạng các mẫu tin riêng biệt khiến người học khó có thể tự mình kết nối chúng lại thành một thông tin tổng hợp học tập hoàn chỉnh. Các công cụ tìm kiếm thông thường hiện nay chỉ giúp tìm ra thông tin của một môn học cụ thể chứ không thể chỉ ra mối quan hệ giữa các môn hay giải thích tại sao kiến thức này lại cần thiết cho kiến thức kia. Thêm vào đó các mô hình trí tuệ nhân tạo phổ biến đôi khi vẫn cung cấp thông tin không chính xác hoặc không hiểu rõ các quy định và chương trình học đặc thù của nhà trường.

Vì những lý do đó đề tài này được ra đời nhằm xây dựng một trợ lý học tập thông minh có khả năng hiểu sâu về lộ trình và tài liệu của sinh viên. Mục đích cuối cùng là tạo ra một trợ lý ảo có thể trả lời chính xác các thắc mắc về học vụ đồng thời hiểu được sự liên kết logic giữa các khái niệm khác nhau. Điểm thú vị của bài toán này nằm ở việc biến những tài liệu thô cứng thành một mạng lưới tri thức giúp sinh viên không còn cảm thấy lạc lõng giữa biển thông tin. Chúng ta cần giải quyết vấn đề này vì việc định hướng đúng đắn ngay từ đầu sẽ giúp sinh viên tiết kiệm rất nhiều thời gian và công sức trong suốt quá trình theo học tại trường.

Tuy nhiên việc xây dựng một hệ thống như vậy không hề đơn giản vì nó

đòi hỏi trí tuệ nhân tạo phải có khả năng tư duy và kết nối thông tin như con người. Đề tài lựa chọn hướng giải quyết thông qua việc kết hợp giữa một nhóm AI Agent phối hợp với nhau cùng một phương pháp tổ chức dữ liệu theo dạng đồ thị tri thức. Những hướng đi trước đây thường chỉ tập trung vào việc tìm kiếm từ khóa nên thường bỏ sót các mối quan hệ ngầm giữa các môn học. Bằng cách xây dựng một bản đồ kiến thức chặt chẽ hệ thống mới sẽ khắc phục được tình trạng trả lời thiếu chiều sâu hoặc thông tin không liên quan.

Nghiên cứu này tập trung chủ yếu vào sinh viên Công nghệ thông tin tại HCMUS với dữ liệu là các môn học cơ sở, môn chuyên ngành và chương trình đào tạo. Thông qua việc trả lời các câu hỏi về cách học và lộ trình phát triển đề tài mang lại một công cụ hỗ trợ thiết thực giúp cá nhân hóa trải nghiệm học tập cho từng người. Đóng góp lớn nhất của nghiên cứu là tạo ra một nền tảng tri thức tin cậy nơi mọi thông tin học tập được tổng hợp và kết nối một cách logic nhất phục vụ hiệu quả cho việc nghiên cứu và rèn luyện của sinh viên trong kỷ nguyên số.

Chương 2

Các công trình liên quan

2.1 Tổng quan về hệ thống hỗ trợ học tập thông minh

Trong môi trường giáo dục đại học, khối lượng kiến thức và cấu trúc chương trình đào tạo ngày càng trở nên phức tạp. Việc xây dựng một lộ trình học tập tối ưu không chỉ dừng lại ở việc hoàn thành các tín chỉ, mà còn phải đáp ứng đúng định hướng nghề nghiệp và năng lực cốt lõi của từng cá nhân. Các hệ thống hỗ trợ học tập thông minh (Intelligent Tutoring/Learning Support Systems) ra đời nhằm giải quyết bài toán này, đóng vai trò như một cầu nối thông tin giữa nhà trường, chương trình đào tạo và sinh viên [40].

2.1.1 Các hướng tiếp cận truyền thống

«»»< HEAD Trước khi có sự bùng nổ của Trí tuệ Nhân tạo và các Mô hình Ngôn ngữ Lớn (LLMs), các hệ thống hỗ trợ học vụ tại các trường đại học chủ yếu hoạt động dựa trên các quy tắc tĩnh (rule-based) và kiến trúc dữ liệu phẳng (flat data representations). Một số đặc điểm và hạn chế của các hệ thống này bao gồm:

- Hệ thống cổng thông tin tĩnh (Static Portals): Thông tin về chương

trình đào tạo, đề cương chi tiết (syllabus), môn học tiên quyết và song hành thường được cung cấp dưới dạng các tệp văn bản hoặc PDF rời rạc. Sinh viên phải tự tra cứu thủ công, dẫn đến tình trạng phân mảnh thông tin và khó hình dung được bức tranh toàn cảnh của mạng lưới môn học.

- Chatbot dựa trên quy tắc (Rule-based Chatbots): Các thể hệ chatbot cũ thường hoạt động dựa trên bộ từ khóa (keywords) và các kịch bản hội thoại được lập trình sẵn. Hệ thống này gặp hạn chế lớn trong việc xử lý ngôn ngữ tự nhiên, không hiểu được ngữ cảnh tổng thể hoặc các truy vấn phức tạp của sinh viên.
- Hạn chế trong việc liên kết dữ liệu: Các phương pháp truyền thống thiếu khả năng mô hình hóa các mối quan hệ đa chiều. Việc kết nối giữa điểm số, sở thích cá nhân và môn học hầu như không được tự động hóa. Điều này làm mất đi tính cá nhân hóa, khiến sinh viên dễ bị mất phương hướng khi tự lập kế hoạch học tập.

2.1.2 Ứng dụng AI trong giáo dục hiện đại

Sự phát triển của Trí tuệ Nhân tạo (AI), đặc biệt là Xử lý Ngôn ngữ Tự nhiên (NLP) và Học máy (Machine Learning), đã tạo ra những bước nhảy vọt trong việc phát triển các hệ thống giáo dục thông minh. Thay vì chỉ cung cấp thông tin thụ động, các hệ thống hiện đại đã chuyển dịch sang hướng tương tác và cá nhân hóa sâu sắc:

- Hệ thống gợi ý và cá nhân hóa (Recommender Systems in Education): AI cho phép phân tích dữ liệu lịch sử học tập (điểm số, tiến độ) kết hợp với dữ liệu phi cấu trúc (nguyện vọng, mục tiêu nghề nghiệp) để đưa ra các gợi ý môn học phù hợp nhất. Các thuật toán này giúp sinh viên vạch ra lộ trình học tập linh hoạt theo từng học kỳ, bám sát với định hướng chuyên môn.

- Chuyển đổi từ Chatbot sang AI Agents (Tác nhân AI): Khác với chatbot thông thường chỉ phản hồi truy vấn đơn lẻ, AI Agents (đặc biệt là kiến trúc Multi-Agent) có khả năng tự động hóa chuỗi suy luận.
- Đồ thị hóa tri thức giáo dục (Educational Knowledge Graphs): Để khắc phục tình trạng AI tạo ra thông tin sai lệch (“ảo giác” - hallucination) và thiếu ngữ cảnh, các nghiên cứu hiện đại đang kết hợp AI với Đồ thị tri thức [12]. Việc số hóa và biểu diễn cấu trúc chương trình đào tạo dưới dạng các thực thể (Môn học, Kỹ năng, Sơ thích) và mối quan hệ (Tiên quyết, Song hành, Kỹ năng đầu ra) giúp hệ thống có khả năng truy xuất thông tin đa chiều. Điều này cho phép AI giải thích được lý do tại sao một môn học lại cần thiết.

===== Trước khi có sự bùng nổ của Trí tuệ Nhân tạo và các Mô hình Ngôn ngữ Lớn (LLMs), các hệ thống hỗ trợ học vụ tại các trường đại học chủ yếu hoạt động dựa trên các quy tắc tĩnh (rule-based) và kiến trúc dữ liệu phẳng (flat data representations). Một số đặc điểm và hạn chế của các hệ thống này bao gồm các hệ thống cổng thông tin tĩnh, nơi thông tin về chương trình đào tạo, đề cương chi tiết, môn học tiên quyết và song hành thường được cung cấp dưới dạng các tệp văn bản hoặc PDF rời rạc. Điều này khiến sinh viên phải tự tra cứu thủ công, dẫn đến tình trạng phân mảnh thông tin và khó hình dung được bức tranh toàn cảnh của mạng lưới môn học. Bên cạnh đó, các chatbot dựa trên quy tắc thường hoạt động dựa trên bộ từ khóa và các kịch bản hội thoại được lập trình sẵn, gặp hạn chế lớn trong việc xử lý ngôn ngữ tự nhiên và không hiểu được ngữ cảnh tổng thể. Cuối cùng, sự hạn chế trong việc liên kết dữ liệu khiến các phương pháp truyền thống thiếu khả năng mô hình hóa các mối quan hệ đa chiều, làm mất đi tính cá nhân hóa và khiến sinh viên dễ bị mất phương hướng khi tự lập kế hoạch học tập.

2.1.3 Ứng dụng AI trong giáo dục hiện đại

Sự phát triển của Trí tuệ Nhân tạo, đặc biệt là Xử lý Ngôn ngữ Tự nhiên và Học máy, đã tạo ra những bước nhảy vọt trong việc phát triển các hệ thống giáo dục thông minh. Thay vì chỉ cung cấp thông tin thụ động, các hệ thống hiện đại đã chuyển dịch sang hướng tương tác và cá nhân hóa sâu sắc thông qua các hệ thống gợi ý cho phép phân tích dữ liệu lịch sử học tập kết hợp với nguyện vọng cá nhân để đưa ra lộ trình linh hoạt. Đồng thời, sự chuyển dịch từ Chatbot sang các tác nhân AI (AI Agents) với kiến trúc Multi-Agent đã mang lại khả năng tự động hóa chuỗi suy luận phức tạp. Đặc biệt, việc đồ thị hóa tri thức giáo dục giúp khắc phục tình trạng "ảo giác" và thiếu ngữ cảnh bằng cách biểu diễn cấu trúc chương trình đào tạo dưới dạng các thực thể và mối quan hệ chặt chẽ, cho phép hệ thống truy xuất thông tin đa chiều và giải thích logic một cách minh bạch. »»»> 1994009 (sửa lỗi viết phần 2 và cập nhập mineru chương 3 và chương 4)

2.2 Mô hình ngôn ngữ lớn (LLMs) trong giáo dục

2.2.1 Ứng dụng LLM trong hệ thống hỏi đáp học tập

«««< HEAD Với khả năng hiểu và sinh ngôn ngữ tự nhiên, LLMs đã nhanh chóng được tích hợp vào các hệ thống giáo dục, đặc biệt là trong các hệ thống hỏi đáp (Q&A):

- Giao tiếp bằng ngôn ngữ tự nhiên (Natural Language Interface): Thay vì phải sử dụng các từ khóa cứng nhắc hay câu lệnh phức tạp, sinh viên có thể đặt câu hỏi theo cách giao tiếp thông thường (ví dụ: “Nếu em trượt môn Cơ sở dữ liệu thì kỳ sau em có được học Hệ quản trị cơ sở dữ liệu không?”). Hệ thống có thể hiểu chính xác ý định và đưa ra câu trả lời trực tiếp.

- Hỗ trợ giải thích và tóm tắt kiến thức: LLMs có khả năng phân tích các đề cương môn học phức tạp và tóm tắt lại những kỹ năng cốt lõi mà sinh viên sẽ đạt được. Ngoài ra, nó có thể đóng vai trò như một người hướng dẫn, giải thích các khái niệm học thuật bằng nhiều cách diễn đạt khác nhau.
- Hỗ trợ sinh tự động (Generative Assistance): LLMs có thể hỗ trợ sinh viên trong việc phác thảo kế hoạch học tập tạm thời, gợi ý cách viết email trao đổi học thuật với giảng viên, hoặc sinh ra các câu hỏi trắc nghiệm để sinh viên tự ôn tập dựa trên tài liệu được cung cấp.

2.2.2 Hạn chế của LLM (Hallucination, thiếu context, v.v.)

Mặc dù sở hữu những khả năng vượt trội, việc triển khai trực tiếp các LLMs thuần túy (vanilla LLMs) vào hệ thống tư vấn học vụ gặp phải những rủi ro và rào cản, đặc biệt là trong môi trường đại học yêu cầu tính chính xác tuyệt đối:

- Hiện tượng ảo giác AI (Hallucination): Đây là điểm yếu lớn nhất của LLMs. Khi không có đủ thông tin hoặc câu hỏi vượt quá dữ liệu huấn luyện, LLM có xu hướng bịa ra các câu trả lời nghe có vẻ rất hợp lý và logic nhưng thực chất là sai sự thật [15]. Trong ngữ cảnh tư vấn học tập, việc AI tự sáng tác ra một môn học tiên quyết không tồn tại hoặc cung cấp sai quy chế học vụ của Khoa Công nghệ Thông tin có thể dẫn đến hậu quả nghiêm trọng, làm sai lệch toàn bộ lộ trình học tập của sinh viên.
- Thiếu ngữ cảnh chuyên biệt và tính cập nhật (Lack of Domain-specific Context & Up-to-date Knowledge): LLMs được huấn luyện trên dữ liệu đại trà tính đến một thời điểm cắt (cut-off date) nhất định. Chúng không thể tự động biết được những thay đổi mới nhất trong

chương trình đào tạo, quy định chuẩn đầu ra, hay danh sách giảng viên phụ trách của Trường Đại học Khoa học Tự nhiên.

- Hạn chế trong suy luận đa bước phức tạp (Complex Multi-hop Reasoning): Để xây dựng một lộ trình học tập, hệ thống cần phải duyệt qua một mạng lưới phức tạp các điều kiện ràng buộc (môn A là tiên quyết của môn B, môn B song hành với môn C). LLMs thuần túy gặp rất nhiều khó khăn trong việc suy luận các mối quan hệ đa tầng (multi-hop) này nếu thông tin chỉ được biểu diễn dưới dạng văn bản phẳng.
- Khó xác minh nguồn gốc (Lack of Traceability): LLMs trả về văn bản sinh tự động mà không cung cấp được nguồn trích dẫn rõ ràng (ví dụ: thông tin này được lấy từ văn bản quy chế nào, trang bao nhiêu). Điều này làm giảm đi độ tin cậy của hệ thống trong mắt sinh viên và giảng viên.

===== Với khả năng hiểu và sinh ngôn ngữ tự nhiên, LLMs đã nhanh chóng được tích hợp vào các hệ thống giáo dục, đặc biệt là trong các hệ thống hỏi đáp thông qua việc cung cấp giao diện giao tiếp tự nhiên. Sinh viên có thể đặt câu hỏi theo cách thông thường và nhận được câu trả lời chính xác về ý định thay vì sử dụng các từ khóa cứng nhắc. Bên cạnh đó, LLMs còn hỗ trợ giải thích và tóm tắt kiến thức từ các đề cương môn học phức tạp, đóng vai trò như một người hướng dẫn đa năng. Cuối cùng, khả năng hỗ trợ sinh tự động giúp sinh viên phác thảo kế hoạch học tập, soạn thảo văn bản học thuật hoặc tạo ra các bộ câu hỏi ôn tập dựa trên tài liệu có sẵn một cách hiệu quả.

2.2.3 Hạn chế của LLM (Hallucination, thiếu context, v.v.)

Mặc dù sở hữu những khả năng vượt trội, việc triển khai trực tiếp các LLMs thuần túy vào hệ thống tư vấn học vụ gặp phải những rủi ro và rào

cẩn lớn, đặc biệt là hiện tượng ảo giác AI khiến mô hình có xu hướng bịa ra các câu trả lời sai lệch nhưng nghe có vẻ logic. Trong ngữ cảnh học vụ, điều này có thể dẫn đến việc cung cấp sai quy chế hoặc lộ trình học tập, gây hậu quả nghiêm trọng. Hơn nữa, LLMs thường thiếu ngữ cảnh chuyên biệt và tính cập nhật do dữ liệu huấn luyện có thời điểm cắt nhất định, không thể tự động nắm bắt các thay đổi mới nhất trong chương trình đào tạo của nhà trường. Sự hạn chế trong suy luận đa bước phức tạp cũng là một thách thức khi hệ thống phải duyệt qua mạng lưới các điều kiện ràng buộc giữa các môn học. Cuối cùng, việc khó xác minh nguồn gốc thông tin làm giảm đi độ tin cậy của hệ thống đối với người dùng. »»»> 1994009 (sửa lỗi viết phần 2 và cập nhập mineru chương 3 và chương 4)

2.3 Retrieval-Augmented Generation (RAG)

Để giải quyết các hạn chế chí mạng của Mô hình ngôn ngữ lớn (LLMs) như hiện tượng “ảo giác” (hallucination) và thiếu thông tin cập nhật (đã phân tích ở mục 2.2), kiến trúc sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation - RAG) [22] đã được đề xuất và nhanh chóng trở thành tiêu chuẩn trong việc xây dựng các hệ thống AI chuyên biệt. Trong bối cảnh đề tài, RAG đóng vai trò là cơ chế cốt lõi giúp AI Agent truy cập vào cơ sở dữ liệu học vụ thực tế của Trường Đại học Khoa học Tự nhiên trước khi đưa ra câu trả lời cho sinh viên.

2.3.1 Kiến trúc RAG truyền thống

Kiến trúc RAG truyền thống (còn gọi là Naive RAG) là một phương pháp kết hợp giữa khả năng truy xuất thông tin và khả năng sinh ngôn ngữ (Text Generation). Thay vì ép buộc LLM phải tự nhớ mọi kiến thức trong các trọng số tham số (parametric memory) của nó, RAG cung cấp cho LLM một “bộ nhớ ngoài” (non-parametric memory) chứa dữ liệu thực tế.

«««< HEAD Quy trình hoạt động cơ bản của Naive RAG diễn ra qua các bước:

- Lập chỉ mục (Indexing): Dữ liệu thô (như file PDF đề cương môn học, quy chế học vụ) được chia nhỏ thành các đoạn văn bản ngắn (chunks), sau đó được chuyển đổi thành các vector số học (embeddings) và lưu trữ vào Cơ sở dữ liệu Vector (Vector Database).
- Truy xuất (Retrieval): Khi sinh viên đặt câu hỏi, hệ thống sẽ chuyển đổi câu hỏi đó thành vector và tìm kiếm các đoạn văn bản (chunks) có độ tương đồng ngữ nghĩa cao nhất trong Vector Database.
- Sinh văn bản (Generation): Các đoạn văn bản tìm được sẽ được ghép nối vào câu lệnh (prompt) như một “ngữ cảnh” (context). LLM sẽ đọc ngữ cảnh này và sinh ra câu trả lời cuối cùng dựa trên các thông tin thực tế vừa được cung cấp.

2.3.2 Các thành phần chính: Retriever – Generator

Hệ thống RAG truyền thống được cấu thành từ hai module độc lập nhưng phối hợp chặt chẽ với nhau:

- Retriever (Bộ truy xuất): Làm nhiệm vụ tìm kiếm và trích xuất thông tin. Chất lượng của Retriever phụ thuộc hoàn toàn vào các mô hình nhúng (Embedding Models) để biểu diễn ý nghĩa của văn bản dưới dạng các điểm trong không gian vector đa chiều. Các thuật toán như Cosine Similarity (độ tương đồng Cosine) thường được sử dụng để đo lường khoảng cách giữa câu hỏi của người dùng và dữ liệu trong cơ sở dữ liệu. Trong hệ thống hỗ trợ học tập, Retriever làm nhiệm vụ tìm ra đúng đoạn văn bản mô tả về môn học hoặc quy định học vụ mà sinh viên đang thắc mắc.
- Generator (Bộ sinh văn bản): Thường là một Mô hình ngôn ngữ lớn (LLM) đã được tinh chỉnh cho tác vụ làm theo chỉ dẫn (Instruction-

tuned LLM). Nhiệm vụ của Generator không phải là tự nghĩ ra câu trả lời từ số không, mà là đọc hiểu các đoạn dữ liệu do Retriever cung cấp, tổng hợp, tóm tắt và diễn đạt lại thành một câu trả lời mạch lạc, tự nhiên và dễ hiểu nhất cho sinh viên.

2.3.3 Ưu điểm và hạn chế của Naive RAG

Ưu điểm:

- Giảm thiểu ảo giác (Hallucination): Bằng cách buộc LLM phải dựa vào dữ liệu học vụ nội bộ của Khoa, RAG gần như loại bỏ được tình trạng AI tự bịa ra thông tin sai lệch.
- Chi phí hiệu quả và dễ cập nhật: Không cần phải huấn luyện lại (fine-tuning) LLM đắt đỏ mỗi khi chương trình đào tạo thay đổi. Nhà trường chỉ cần cập nhật các file tài liệu mới vào Vector Database.
- Tính minh bạch (Traceability): RAG cho phép hệ thống trích dẫn nguồn gốc của thông tin (ví dụ: “Theo mục 2 của Quy chế Đào tạo tín chỉ...”), giúp tăng độ tin cậy cho sinh viên.

Hạn chế chí mạng đối với dữ liệu giáo dục đại học: Dù Naive RAG hoạt động rất tốt với các tác vụ hỏi đáp thông thường (Q&A), nhưng có nhiều điểm yếu khi xử lý với dữ liệu có tính liên kết chặt chẽ như chương trình đào tạo đại học:

- Hạn chế của biểu diễn dữ liệu phẳng (Flat Data Representation): Naive RAG chia cắt tài liệu thành các khối (chunks) rời rạc. Khi phân tách, hệ thống vô tình cắt đứt các mối liên hệ logic phức tạp. Ví dụ: Môn A là tiên quyết của môn B, và môn B kết hợp với môn C. Vector Database chỉ lưu các đoạn văn bản rời, không “hiểu” được bản đồ mạng lưới của các môn học này.

- Thất bại trong suy luận đa bước (Multi-hop Reasoning Failure): Naive RAG sẽ bối rối khi câu hỏi đòi hỏi phải duyệt qua và tổng hợp từ hàng chục đề cương môn học khác nhau.
- Bỏ sót thông tin: Nếu từ khóa truy vấn của sinh viên không khớp hoàn toàn về mặt ngữ nghĩa cục bộ với các chunk trong database, Retriever sẽ thất bại trong việc mang về ngữ cảnh đúng, dẫn đến việc Generator không có dữ liệu để trả lời.

===== Quy trình hoạt động cơ bản của Naive RAG diễn ra qua ba giai đoạn chính. Đầu tiên là giai đoạn lập chỉ mục, trong đó dữ liệu thô được chia nhỏ thành các đoạn văn bản, chuyển đổi thành vector và lưu trữ vào cơ sở dữ liệu chuyên biệt. Tiếp theo, trong giai đoạn truy xuất, khi có câu hỏi từ người dùng, hệ thống sẽ tìm kiếm các đoạn văn bản có độ tương đồng ngữ nghĩa cao nhất. Cuối cùng, ở giai đoạn sinh văn bản, các đoạn dữ liệu này được ghép nối vào câu lệnh làm ngữ cảnh để mô hình ngôn ngữ sinh ra câu trả lời dựa trên các thông tin thực tế vừa thu thập được.

2.3.4 Các thành phần chính: Retriever – Generator

Hệ thống RAG truyền thống được cấu thành từ hai module phối hợp chặt chẽ. Module Retriever làm nhiệm vụ tìm kiếm và trích xuất thông tin bằng cách sử dụng các mô hình nhúng để biểu diễn ý nghĩa văn bản trong không gian vector, từ đó tìm ra đúng đoạn văn bản mô tả về môn học hoặc quy định học vụ liên quan. Song song đó, module Generator, thường là các mô hình ngôn ngữ lớn đã được tinh chỉnh, chịu trách nhiệm đọc hiểu dữ liệu từ bộ truy xuất để tổng hợp và diễn đạt lại thành câu trả lời mạch lạc, tự nhiên cho sinh viên thay vì tự tạo ra thông tin từ đầu.

2.3.5 Ưu điểm và hạn chế của Naive RAG

Kiến trúc Naive RAG mang lại nhiều ưu điểm quan trọng như khả năng giảm thiểu ảo giác bằng cách buộc mô hình dựa trên dữ liệu nội bộ,

đồng thời đảm bảo chi phí hiệu quả và dễ cập nhật thông tin mà không cần huấn luyện lại mô hình. Tính minh bạch cũng được nâng cao khi hệ thống có thể trích dẫn chính xác nguồn gốc tài liệu, giúp tăng độ tin cậy cho người dùng.

Tuy nhiên, Naive RAG vẫn tồn tại những hạn chế chí mạng khi xử lý dữ liệu giáo dục có tính liên kết cao. Việc biểu diễn dữ liệu phẳng khiến hệ thống vô tình cắt đứt các mối liên hệ logic giữa các môn học khi chia cắt tài liệu thành các khối rời rạc. Điều này dẫn đến sự thất bại trong các suy luận đa bước phức tạp đòi hỏi tổng hợp từ nhiều nguồn khác nhau. Hơn nữa, nếu từ khóa truy vấn không khớp hoàn toàn về ngữ nghĩa với cơ sở dữ liệu, bộ truy xuất có thể bỏ sót thông tin quan trọng, khiến hệ thống không thể cung cấp câu trả lời chính xác. »»»> 1994009 (sửa lỗi viết phần 2 và cập nhập mineru chương 3 và chương 4)

2.4 GraphRAG và Knowledge Graph

Để khắc phục những yếu điểm của hệ thống RAG truyền thống (Vector RAG) trong việc xử lý các truy vấn phức tạp và thiếu nhận thức về ngữ cảnh tổng thể, hướng tiếp cận kết hợp Đồ thị tri thức (Knowledge Graph) vào quá trình truy xuất đã ra đời, được gọi chung là GraphRAG. Trong bối cảnh giáo dục đại học, nơi thông tin có tính liên kết cao.

2.4.1 Tổng quan về Knowledge Graph

Đồ thị tri thức (Knowledge Graph - KG) là một cấu trúc dữ liệu mô hình hóa thông tin dưới dạng một mạng lưới [12]. Thay vì lưu trữ dữ liệu dưới dạng các bảng độc lập (như cơ sở dữ liệu quan hệ) hoặc các văn bản phẳng rời rạc, KG lưu trữ thông tin dưới dạng các Thực thể (Entities - Node) và các Mối quan hệ (Relationships - Edge) giữa chúng.

«««< HEAD Trong hệ thống hỗ trợ học tập của Khoa Công nghệ Thông tin, Đồ thị tri thức có thể được thiết kế như sau:

- Thực thể (Nodes): Sinh viên, Môn học (vd: Nhập môn Lập trình, Cơ Sở Trí tuệ Nhân tạo), Kỹ năng (vd: Lập trình Python, Tư duy logic).
- Mối quan hệ (Edges/Relations): Là tiên quyết của (IS PREREQUISITE OF), Cần thiết cho (REQUIRED FOR), Thuộc khối kiến thức (BELONGS TO),...

===== Trong hệ thống hỗ trợ học tập, Đồ thị tri thức được thiết kế để mô hình hóa các thực thể như sinh viên, môn học và kỹ năng, cùng với các mối quan hệ chặt chẽ như điều kiện tiên quyết, song hành hoặc thuộc về một khối kiến thức cụ thể. Cách tiếp cận này giúp hệ thống nắm bắt được cấu trúc mạng lưới tri thức thay vì chỉ các thông tin văn bản rời rạc.

»»» > 1994009 (sửa lỗi viết phần 2 và cập nhật mineru chương 3 và chương 4)

2.5 Các hệ thống RAG dựa trên Đồ thị tri thức

Sự phát triển của RAG kết hợp với Đồ thị tri thức đã dẫn đến sự ra đời của nhiều hệ thống khác nhau nhằm giải quyết các bài toán về truy xuất và lập luận phức tạp. Các hệ thống này khác biệt chủ yếu ở cách mô hình hóa đồ thị, cơ chế truy vấn và khả năng cập nhật dữ liệu. Điển hình là hệ thống GraphRAG do Microsoft phát triển vào năm 2024 [6]. Kiến trúc này sử dụng đồ thị phân cấp dựa trên các cụm cộng đồng và thực hiện tóm tắt ở cấp độ cộng đồng để hỗ trợ tìm kiếm cục bộ lẫn toàn cục. Mặc dù thể hiện khả năng hiểu các mối quan hệ sâu và tóm tắt toàn cục rất tốt, GraphRAG đòi hỏi chi phí xây dựng chỉ mục cao và bị hạn chế trong việc cập nhật dữ liệu liên tục do phải tái cấu trúc lại các cụm mỗi khi có dữ liệu mới.

Một giải pháp khác là HippoRAG2 được giới thiệu tại NeurIPS 2025 [9],

sử dụng kiến trúc đồ thị phức hợp bao gồm các node thực thể, node đoạn văn bản và cạnh đồng nghĩa. Hệ thống ứng dụng thuật toán PageRank cá nhân hóa để trích xuất văn bản và hỗ trợ lập luận đa bước. Điểm mạnh của HippoRAG2 là khả năng học liên tục không tham số và cập nhật tăng dần trực tuyến xuất sắc mà không cần lập chỉ mục lại toàn bộ, dù khả năng truy vấn toàn cục chỉ ở mức trung bình. Tiếp đến là PathRAG [37], một nghiên cứu tiếp cận bằng cách sử dụng đồ thị đường đi quan hệ kết hợp với kỹ thuật tia nhánh dựa trên luồng. Phương pháp này giảm bớt ngữ cảnh không cần thiết trong khi vẫn duy trì độ chính xác cao cho các truy vấn đa bước, tuy nhiên lại gặp khó khăn và tốn kém khi đồ thị thay đổi vì cần phải thực hiện lại quá trình tia nhánh.

Các hệ thống chuyên biệt hơn bao gồm OG-RAG [33] và HyperGraphRAG [21]. OG-RAG hoạt động dựa trên đồ thị có ràng buộc lược đồ, giúp giảm đáng kể hiện tượng ảo giác và phù hợp cho các miền dữ liệu đặc thù, nhưng lại yêu cầu người dùng phải cập nhật lược đồ thủ công. Ngược lại, HyperGraphRAG ứng dụng siêu đồ thị để nắm bắt các mối quan hệ đa thực thể phức tạp vượt khỏi giới hạn quan hệ nhị phân. Hệ thống này rất thích hợp cho các nguồn dữ liệu khoa học, nhưng hiện đòi hỏi chi phí tính toán cao và chưa hoàn toàn tối ưu hóa cho môi trường vận hành thực tế ¹.

2.6 Lý do lựa chọn hệ thống LightRAG

Qua quá trình phân tích chuyên sâu các kiến trúc hiện đại, LightRAG (được giới thiệu năm 2024 [8]) được lựa chọn làm giải pháp cốt lõi cho hệ thống tư vấn học tập. Quyết định này xuất phát từ việc LightRAG mang lại sự cân bằng hoàn hảo giữa hiệu năng truy xuất, khả năng thích ứng với dữ liệu động và tối ưu hóa chi phí vận hành, khắc phục được nhiều khuyết điểm chí mạng của các hệ thống tiền nhiệm. Những lợi ích to lớn về khả năng lập luận đa bước chính xác, hiệu suất truy vấn toàn cục nhanh chóng

¹Tham khảo dữ liệu so sánh từ báo cáo đánh giá các hệ thống RAG sử dụng Knowledge Graph năm 2025.

và chi phí tạo lập chỉ mục siêu thấp của LightRAG là yếu tố then chốt. Việc phân chia cấp độ truy vấn còn giúp đảm bảo hệ thống không bị ảo giác khi đối mặt với các câu hỏi phức hợp, đồng thời không gây lãng phí tài nguyên cho các câu hỏi đơn điệu. Tổng hợp tất cả các khía cạnh trên, LightRAG khẳng định được vị thế là lựa chọn khả thi, tiết kiệm và tối ưu nhất để xây dựng một môi trường tư vấn học thuật cá nhân hóa và đáng tin cậy.

«««< HEAD

2.7 Hiện trạng của LightRAG

Nhằm hiểu rõ hơn về tiềm năng và khả năng ứng dụng thực tế của LightRAG trong ngữ cảnh hiện tại, việc đánh giá chi tiết cả những ưu điểm nổi trội lẫn các hạn chế còn tồn đọng là vô cùng cần thiết.

Xét về ưu điểm, khác với cấu trúc phân cụm phức tạp gây tốn kém tài nguyên của GraphRAG hay mô hình lược đồ cứng nhắc của OG-RAG, LightRAG sử dụng cấu trúc đồ thị thực thể và mối quan hệ dạng phẳng kết hợp với cơ chế truy xuất đa cấp độ. Hệ thống thực hiện song song việc khớp từ khóa, tìm kiếm vector và truy vấn đồ thị cục bộ lẫn toàn cục. Nhờ chiến lược truy xuất theo hai luồng riêng biệt này, LightRAG không chỉ bao quát được toàn bộ ngữ cảnh từ diện rộng đến chi tiết mà còn đạt tốc độ phản hồi cực kỳ nhanh với độ trễ tối ưu chỉ khoảng tám mươi mili-giây [8]. Một lợi thế cạnh tranh mang tính quyết định khác của hệ thống nằm ở khả năng cập nhật tri thức linh hoạt. Trong môi trường học thuật, nơi quy chế đào tạo thay đổi thường xuyên, LightRAG hỗ trợ cập nhật tăng dần một cách mượt mà bằng cách trích xuất tự động và nối thêm các thực thể vào mạng lưới đồ thị hiện hữu mà không làm phá vỡ tổng thể. Sự linh hoạt này đi kèm với khả năng tương thích cao trên nhiều nền tảng lưu trữ như Neo4j hay PostgreSQL.

Bên cạnh những điểm mạnh vượt trội, hệ thống hiện tại vẫn bộc lộ nhiều hạn chế cần được nghiên cứu khắc phục. Thứ nhất, quá trình trích

xuất thông tin để xây dựng đồ thị phụ thuộc hoàn toàn vào chất lượng của mô hình ngôn ngữ lớn làm nền tảng. Nếu mô hình không hiểu sâu sát văn phong học vụ, hệ thống dễ nhận diện sai hoặc bỏ sót mối quan hệ. Thứ hai, hệ thống xử lý tài liệu đa định dạng của LightRAG, đặc biệt là khi chuyển đổi sang định dạng Markdown, hiện chưa hỗ trợ tốt cho Tiếng Việt. Điều này bộc lộ rõ yếu điểm khi hệ thống phải trải qua bước trung gian là chuyển các tệp văn phòng mặc định như tài liệu Word hay PowerPoint sang định dạng PDF rồi mới thực hiện dịch và xử lý, gây thất thoát hoặc sai lệch định dạng văn bản gốc. =====

2.8 Hiện trạng của LightRAG (Phiên bản 1.4.11)

Nhằm hiểu rõ hơn về tiềm năng và khả năng ứng dụng thực tế của LightRAG trong ngữ cảnh hiện tại, việc đánh giá chi tiết cả những ưu điểm nổi trội lẫn các hạn chế còn tồn đọng là vô cùng cần thiết. Hiện tại, nền tảng đang được xem xét dựa trên phiên bản phát hành 1.4.11.

Xét về ưu điểm, khác với cấu trúc phân cụm phức tạp gây tốn kém tài nguyên của GraphRAG hay mô hình lược đồ cứng nhắc của OG-RAG, LightRAG sử dụng cấu trúc đồ thị thực thể và mối quan hệ dạng phẳng kết hợp với cơ chế truy xuất đa cấp độ. Hệ thống thực hiện song song việc khớp từ khóa, tìm kiếm vector và truy vấn đồ thị cục bộ lẫn toàn cục. Nhờ chiến lược truy xuất theo hai luồng riêng biệt này, LightRAG không chỉ bao quát được toàn bộ ngữ cảnh từ diện rộng đến chi tiết mà còn đạt tốc độ phản hồi cực kỳ nhanh với độ trễ tối ưu chỉ khoảng tám mươi mili-giây [8]. Một lợi thế cạnh tranh mang tính quyết định khác của hệ thống nằm ở khả năng cập nhật tri thức linh hoạt. Trong môi trường học thuật, nơi quy chế đào tạo thay đổi thường xuyên, LightRAG hỗ trợ cập nhật tăng dần một cách mượt mà bằng cách trích xuất tự động và nối thêm các thực thể vào mạng lưới đồ thị hiện hữu mà không làm phá vỡ tổng thể. Sự linh

hoạt này đi kèm với khả năng tương thích cao trên nhiều nền tảng lưu trữ như Neo4j hay PostgreSQL.

Bên cạnh những điểm mạnh vượt trội, hệ thống hiện tại (phiên bản 1.4.11) vẫn bộc lộ nhiều hạn chế cần được nghiên cứu khắc phục. Thứ nhất, quá trình trích xuất thông tin để xây dựng đồ thị phụ thuộc hoàn toàn vào chất lượng của mô hình ngôn ngữ lớn làm nền tảng. Nếu mô hình không hiểu sâu sát văn phong học vụ, hệ thống dễ nhận diện sai hoặc bỏ sót mối quan hệ. Thứ hai, cơ chế trích xuất nội dung văn bản mặc định của LightRAG chủ yếu dựa trên các thư viện đọc trực tiếp (như `pypdf`, `python-docx`, hoặc `docling`) để xử lý các tài liệu PDF, Word. Phương pháp này bộc lộ yếu điểm lớn khi áp dụng vào kho tài liệu thực tế của trường đại học, vốn chứa rất nhiều tài liệu dạng ảnh, các file PDF được scan từ văn bản giấy, hoặc các slide bài giảng có cấu trúc phức tạp. Do thiếu cơ chế Nhận dạng Ký tự Quang học (OCR) chuyên sâu được tối ưu hóa cho tài liệu Tiếng Việt dạng này, hệ thống không thể đọc được hoặc đọc sai lệch nội dung từ các hình ảnh, dẫn đến việc mất mát lượng lớn thông tin quan trọng trước khi đưa vào mô hình học. »»»> 1994009 (sửa lỗi viết phần 2 và cập nhập mineru chương 3 và chương 4)

Thứ ba, LightRAG hiện đang thiếu hụt một hệ thống thu thập dữ liệu tự động từ các nguồn trực tuyến. Việc không có sẵn các công cụ cào dữ liệu từ các trang web của trường hay khoa khiến quá trình cập nhật thông tin trở nên thủ công và mất thời gian. Thứ tư, các câu lệnh định hướng (prompt) mặc định của hệ thống được phân bổ quá chung chung. Chúng hoàn toàn không phù hợp để xử lý các tài liệu đặc thù của trường đại học, vốn chứa dày đặc các từ viết tắt, thuật ngữ chuyên ngành và các quy tắc học vụ riêng biệt. Cuối cùng, cấu trúc Đồ thị tri thức hiện tại của hệ thống vẫn còn thiếu các cơ chế ràng buộc hoặc kiểm chứng tính đúng đắn. Việc không có các quy tắc xác thực chặt chẽ khiến đồ thị dễ bị nhiễu bởi các thực thể sai lệch sinh ra từ quá trình trích xuất tự động, từ đó làm giảm đi độ tin cậy của toàn bộ hệ thống tư vấn.

2.9 Hệ thống Đa tác nhân (Multi-Agent Systems) và Luồng suy luận (Agentic Workflow)

Mặc dù sự kết hợp giữa LLM và LightRAG đã giải quyết được bài toán truy xuất thông tin học vụ chính xác, bản thân mô hình ngôn ngữ vẫn chỉ đóng vai trò thụ động (nhận câu hỏi và trả lời). Để hệ thống có thể tự động lập kế hoạch, tự kiểm tra lỗi và sử dụng các công cụ một cách linh hoạt, việc áp dụng kiến trúc AI Agent (Tác nhân AI) là thiết yếu.

2.9.1 Khái niệm AI Agent

«««< HEAD AI Agent (Tác nhân AI) là một hệ thống sử dụng Mô hình ngôn ngữ lớn (LLM) làm trung tâm để điều khiển quá trình suy luận và đưa ra quyết định [39]. Khác với LLM truyền thống, một Agent hoàn chỉnh bao gồm 4 thành phần cốt lõi:

- Hồ sơ (Persona/Profile): Định hình vai trò của Agent (ví dụ: Cố vấn học tập, Trợ giảng).
- Bộ nhớ (Memory): Khả năng lưu trữ ngữ cảnh ngắn hạn (lịch sử chat) và dài hạn (thông tin, sở thích, điểm số của sinh viên).
- Công cụ (Tools): Khả năng tương tác với môi trường bên ngoài (ví dụ: gọi API để tìm kiếm LightRAG, gọi công cụ search tool, bộ nhớ lưu trữ lịch sử trò chuyện).
- Lập kế hoạch: Khả năng chia nhỏ một truy vấn phức tạp của người dùng thành nhiều bước nhỏ để giải quyết từng phần.

===== AI Agent là một hệ thống sử dụng Mô hình ngôn ngữ lớn làm trung tâm để điều khiển quá trình suy luận và đưa ra quyết định. Một Agent hoàn chỉnh được cấu thành từ bốn thành phần cốt lõi bao gồm hồ sơ

để định hình vai trò, bộ nhớ để lưu trữ ngữ cảnh ngắn hạn và dài hạn, các công cụ để tương tác với môi trường bên ngoài, và khả năng lập kế hoạch để chia nhỏ các truy vấn phức tạp thành các bước giải quyết cụ thể. »»»> 1994009 (sửa lỗi viết phần 2 và cập nhập mineru chương 3 và chương 4)

2.9.2 Tích hợp LightRAG và Multi-Agent

Sự kết hợp giữa Multi-Agent và LightRAG tạo ra một hệ thống hỗ trợ học tập toàn diện. Trong đó, LightRAG đóng vai trò là “Bộ nhớ kiến thức dài hạn” (Long-term Knowledge Memory) với độ chính xác cao và chi phí tối ưu, còn Multi-Agent đóng vai trò là “Trung tâm xử lý logic” (Reasoning Engine). Tác nhân AI sẽ tự động quyết định khi nào cần sử dụng truy xuất cấp độ thấp (chỉ tìm thông tin 1 môn học) và khi nào cần truy xuất cấp độ cao (tìm lộ trình tổng thể) trên nền tảng LightRAG, từ đó mang lại trải nghiệm tư vấn cá nhân hóa và sát với thực tế nhất cho sinh viên Khoa Công nghệ Thông tin.

2.10 Đánh giá hệ thống RAG và Multi-Agent

Sau khi thiết kế và triển khai hệ thống thông minh dựa trên kiến trúc LightRAG và Multi-Agent, việc thiết lập một quy trình đánh giá chuẩn xác là vô cùng cần thiết. Đặc thù của dữ liệu giáo dục đại học yêu cầu hệ thống không chỉ trả lời tự nhiên mà phải đảm bảo tính chính xác tuyệt đối, không gây ảnh hưởng sai lệch đến quyết định đăng ký môn học của sinh viên.

2.10.1 Các chỉ số đánh giá RAG (RAGAS)

«««< HEAD Để đánh giá chất lượng của kiến trúc RAG một cách tự động và định lượng, RAGAS (Retrieval Augmented Generation Assessment) [7] hiện là framework tiêu chuẩn được cộng đồng nghiên cứu sử dụng

rộng rãi. RAGAS đánh giá hệ thống thông qua hai khía cạnh chính: khả năng truy xuất (Retrieval) và khả năng sinh văn bản (Generation), với 4 chỉ số cốt lõi:

- **Độ trung thực (Faithfulness):** Đo lường mức độ câu trả lời của AI bám sát vào ngữ cảnh (context) đã được truy xuất. Nếu hệ thống trả lời về một quy định chuẩn đầu ra Tiếng Anh không hề có trong tài liệu của trường, chỉ số này sẽ bị đánh giá thấp, chỉ ra hiện tượng “ảo giác” (hallucination).
- **Độ liên quan của câu trả lời (Answer Relevance):** Đánh giá xem câu trả lời có đi thẳng vào vấn đề mà sinh viên hỏi hay không, hay đang trả lời vòng vo, thừa thông tin.
- **Độ chính xác của ngữ cảnh (Context Precision):** Đo lường xem các tài liệu (hoặc các node trên đồ thị tri thức) quan trọng nhất có được bộ truy xuất xếp hạng ở những vị trí đầu tiên hay không.
- **Độ bao phủ của ngữ cảnh (Context Recall):** Đánh giá xem bộ truy xuất có thu thập “đủ” các thông tin cần thiết để giải quyết trọn vẹn câu hỏi hay không. Ví dụ: Nếu câu hỏi cần truy xuất cả đề cương môn học lẫn quy chế học vụ, hệ thống có lấy sót tài liệu nào không.

2.10.2 Đánh giá hệ thống đa agent (Multi-Agent Evaluation)

Việc đánh giá một hệ thống Đa tác nhân (Multi-Agent) phức tạp hơn RAG thông thường, bởi nó không chỉ là đầu vào - đầu ra (input/output) mà là một quy trình suy luận gồm nhiều bước (Agentic workflow). Các tiêu chí đánh giá tập trung vào:

- **Tỷ lệ hoàn thành nhiệm vụ (Task Completion Rate):** Đo lường tần suất các Agent phối hợp thành công để đưa ra một kế hoạch học tập hoàn chỉnh.

- Hiệu suất và Chi phí (Efficiency & Cost): Cần đánh giá độ trễ trung bình của hệ thống (Latency - thời gian từ lúc sinh viên hỏi đến khi có câu trả lời) và chi phí tài nguyên (số lượng API Token tiêu thụ cho mỗi luồng hội thoại). Việc sử dụng LightRAG được kỳ vọng sẽ tối ưu đáng kể chỉ số này so với các mô hình GraphRAG truyền thống.

2.10.3 Các phương pháp benchmark hiện nay

Để tiến hành đo lường các chỉ số trên, nhóm dự kiến áp dụng các phương pháp benchmark (chấm điểm chuẩn) sau:

- Xây dựng tập dữ liệu chuẩn (Golden Dataset / Ground Truth): Nhóm sẽ tự xây dựng một bộ khoảng 100-200 cặp câu hỏi - câu trả lời chuẩn xác. Các câu hỏi này bao quát nhiều kịch bản thực tế (đăng ký môn tiên quyết, tư vấn hướng chuyên ngành, quy định xét tốt nghiệp) và các câu trả lời sẽ được trích xuất thủ công, chính xác từ tài liệu của Khoa Công nghệ Thông tin. Tập dữ liệu này làm mốc để đối chiếu tự động với đầu ra của hệ thống.
- LLM-as-a-Judge (Sử dụng LLM mạnh hơn để chấm điểm): Thay vì chấm điểm thủ công hàng ngàn câu trả lời, nhóm sẽ sử dụng các mô hình tiên tiến nhất (như GPT-4 hoặc Claude 3.5) đóng vai trò là “Giám khảo” [41]. Giám khảo LLM này sẽ đọc câu hỏi, câu trả lời chuẩn (Ground Truth) và câu trả lời của hệ thống, từ đó chấm điểm các tiêu chí RAGAS một cách tự động và khách quan.
- Đánh giá từ người dùng thực tế (Human-in-the-loop Evaluation): Đây là bước kiểm chứng cuối cùng và quan trọng nhất. Hệ thống sẽ được triển khai thử nghiệm nghiệm (Beta testing) cho một nhóm sinh viên và giảng viên cố vấn học tập của Khoa. Phản hồi sẽ được thu thập thông qua khảo sát trải nghiệm người dùng (UX) và mức độ hữu ích của thông tin hỗ trợ, làm cơ sở để tinh chỉnh các Prompt và luồng hoạt động của Agent.

===== Để đánh giá chất lượng của kiến trúc RAG một cách tự động và định lượng, framework RAGAS thường được sử dụng thông qua bốn chỉ số cốt lõi. Độ trung thực giúp đo lường mức độ bám sát của câu trả lời vào ngữ cảnh, trong khi độ liên quan đánh giá tính trực tiếp của thông tin đối với câu hỏi. Song song đó, độ chính xác của ngữ cảnh và độ bao phủ của ngữ cảnh đảm bảo rằng các tài liệu quan trọng nhất được trích xuất đầy đủ và chính xác để giải quyết trọn vẹn thắc mắc của người dùng.

2.10.4 Đánh giá hệ thống đa agent (Multi-Agent Evaluation)

Việc đánh giá hệ thống Đa tác nhân tập trung vào các tiêu chí như tỷ lệ hoàn thành nhiệm vụ để đo lường khả năng phối hợp thành công, cùng với hiệu suất và chi phí nhằm tối ưu hóa độ trễ và tài nguyên tiêu thụ. Sự kết hợp này hướng tới việc cung cấp trải nghiệm tốt nhất cho người dùng trong khi vẫn đảm bảo tính bền vững về mặt kỹ thuật.

2.10.5 Các phương pháp benchmark hiện nay

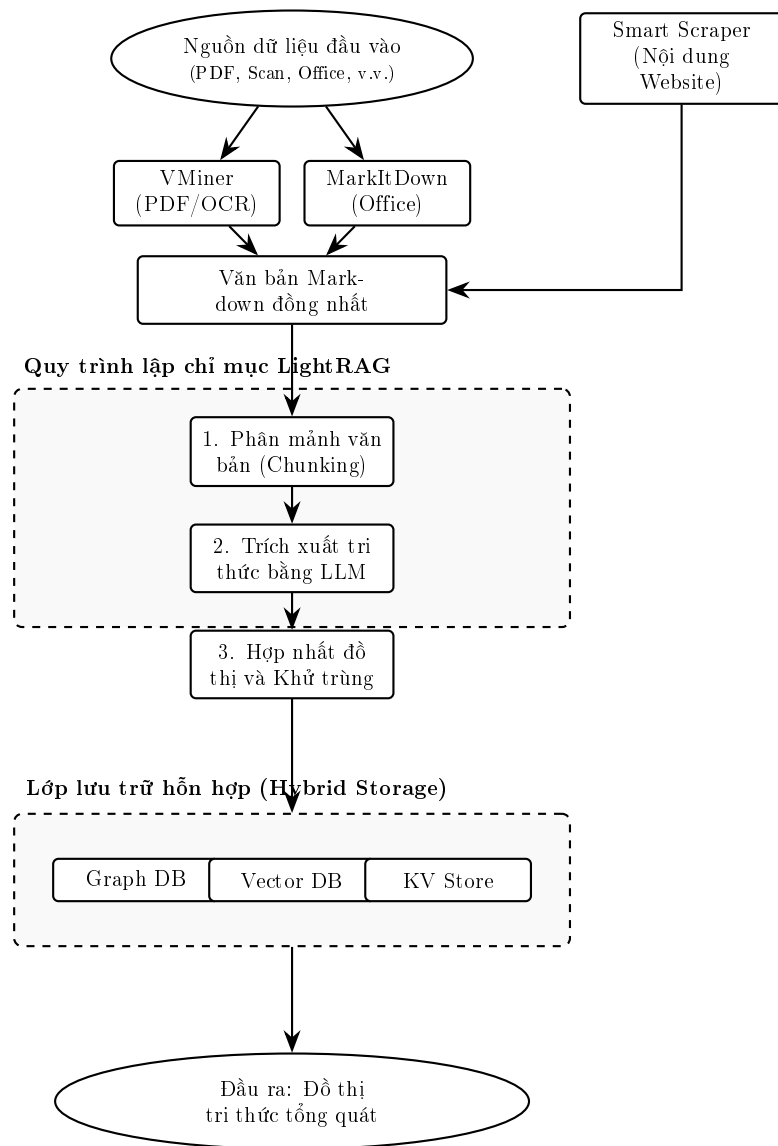
Để tiến hành đo lường, nhóm dự kiến áp dụng ba phương pháp benchmark chính. Đầu tiên là xây dựng tập dữ liệu chuẩn bao quát nhiều kịch bản thực tế để làm mốc đối chiếu. Tiếp theo, kỹ thuật LLM-as-a-Judge sẽ được sử dụng để tự động chấm điểm dựa trên các mô hình ngôn ngữ mạnh mẽ. Cuối cùng, việc đánh giá từ người dùng thực tế thông qua các đợt thử nghiệm Beta sẽ cung cấp những phản hồi quý giá để tinh chỉnh hệ thống sát với nhu cầu sử dụng. »»»> 1994009 (sửa lỗi viết phần 2 và cập nhập mineru chương 3 và chương 4)

Chương 3

Phương pháp đề xuất

Trên cơ sở phân tích lý thuyết và yêu cầu thực tiễn, đề xuất kiến trúc hệ thống hỗ trợ học tập thông minh dựa trên sự LightRAG [10] một mã nguồn mở được phát triển từ GraphRAG [28] và LangGraph.

3.1 Quy trình xử lý dữ liệu



Hình 3.1: Quy trình tổng quát từ văn bản thô đến đồ thị tri thức

Đây là quy trình xử lý thông tin, đóng vai trò "nguyên liệu đầu vào" cho toàn bộ hệ thống. Hệ thống không chỉ lưu trữ dữ liệu thô mà còn thực hiện các phép biến đổi vector hóa phức tạp. Quản lý nguồn dữ liệu đa dạng bao gồm hồ sơ người học và các học liệu phi cấu trúc như bài giảng, tài liệu tham khảo.

Đồ thị tri thức (Knowledge Graph) [11]: đây là thành phần quan trọng

nhất, lưu trữ tri thức dưới dạng cấu trúc mạng lưới. Các nút đại diện cho các khái niệm và các cạnh biểu diễn mối quan hệ sự phạm. Điều này giúp hệ thống hiểu được "vị trí" và "tầm quan trọng" của một khái niệm trong tổng thể chương trình học.

3.1.1 Tiền xử lý dữ liệu tri thức

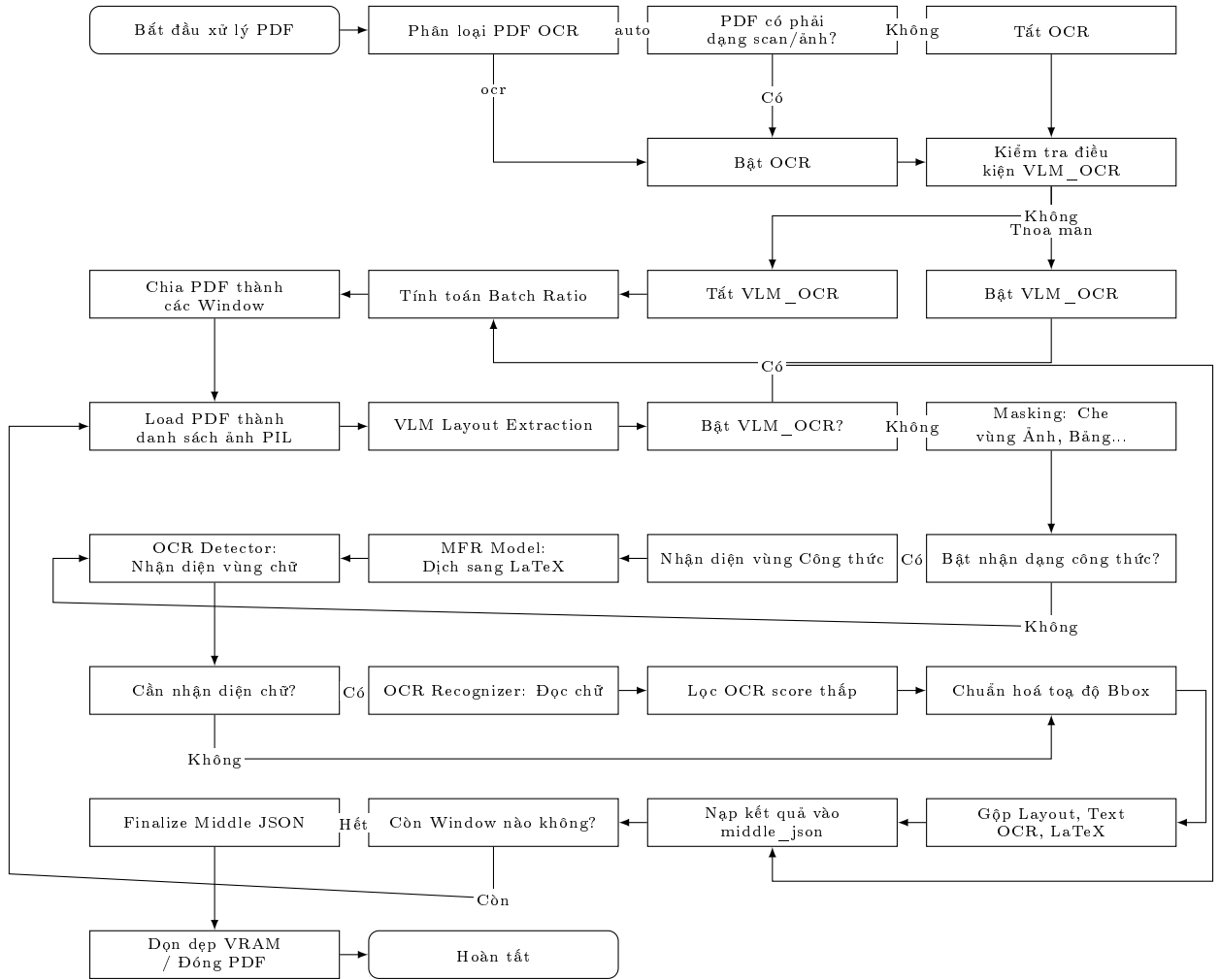
3.1.1.1 Cơ sở lựa chọn công cụ và vai trò trong hệ thống

Trong môi trường học thuật, nguồn dữ liệu đầu vào thường rất đa dạng và phức tạp, bao gồm các tệp PDF được scan từ văn bản giấy, tài liệu chứa nhiều biểu đồ, hình ảnh, và đặc biệt là các công thức toán học. Các công cụ trích xuất văn bản truyền thống thường xuyên gặp thất bại trong việc xử lý các tài liệu này, dẫn đến việc mất thông tin quan trọng hoặc làm xáo trộn cấu trúc bài giảng.

Để giải quyết triệt để thách thức này, hệ thống áp dụng công cụ MinerU **mineru_2024** (từ phiên bản 3.2.3 trở lên) làm giải pháp lõi cho quy trình tiền xử lý dữ liệu. Đây là một công cụ trích xuất dữ liệu đa phương thức chất lượng cao, tích hợp các mô hình tiên tiến như Vision-Language Model (VLM) cho việc phân tích bố cục (Layout Analysis), mô hình OCR để nhận diện văn bản trên ảnh, và mô hình Mathematical Formula Recognition (MFR) để chuyển đổi công thức toán học sang chuẩn LaTeX. Sự kết hợp này giúp MinerU không chỉ trích xuất chính xác văn bản mà còn bảo toàn hoàn hảo cấu trúc logic của tài liệu gốc, đặc biệt phù hợp với các giáo trình, bài báo khoa học và tài liệu lưu trữ của trường đại học.

3.1.1.2 Luồng xử lý tự động (Hybrid-Auto-Engine)

Chế độ **-b hybrid-auto-engine** trong MinerU thực thi một quy trình xử lý khép kín, kết hợp giữa mô hình đa phương thức (VLM) để bóc tách cấu trúc và các mô hình truyền thống (OCR, MFR) để nhận dạng văn bản. Dưới đây là sơ đồ chi tiết về luồng hoạt động này:



Hình 3.2: Quy trình xử lý Hybrid-Auto-Engine trong MinerU mineru_2024

Quy trình xử lý được thực hiện tuần tự qua các giai đoạn. Đầu tiên, hệ thống tiến hành phân loại và phân tích định dạng của tệp đầu vào để xác định là văn bản gốc hay bản quét. Dựa trên kết quả này, cơ chế nhận dạng ký tự quang học sẽ tự động được kích hoạt nếu cần thiết. Tiếp theo, hệ thống đánh giá tài nguyên phần cứng hiện có, cụ thể là dung lượng bộ nhớ đồ họa VRAM, để tính toán tỷ lệ phân lô phù hợp, đảm bảo tối ưu hóa hiệu suất mà không gây tràn bộ nhớ.

Sau quá trình khởi tạo, tài liệu được phân chia thành các đoạn nhỏ để xử lý dần. Tại mỗi phân đoạn, hệ thống sử dụng một mô hình đa phương

thức VLM mạnh mẽ để bóc tách cấu trúc tổng thể, nhận diện các khối văn bản, bảng biểu, hình ảnh và công thức. Đối với các vùng chứa hình ảnh và bảng biểu, hệ thống sẽ tiến hành làm mờ nhằm không gây nhiễu cho các mô hình phía sau. Đồng thời, các công thức toán học nằm trong dòng văn bản sẽ được mô hình MFR chuyên biệt nhận diện và dịch đổi sang định dạng mã chuẩn LaTeX. Các khối chữ chưa xử lý được chuyển qua mô hình nhận diện ký tự quang học OCR. Nhằm đảm bảo độ chính xác, các kết quả có độ tin cậy thấp sẽ bị hệ thống tự động lọc bỏ.

Cuối cùng, ở giai đoạn tổng hợp, tọa độ của các khối thông tin được chuẩn hóa về một hệ quy chiếu chung. Tất cả các dữ liệu trích xuất từ mô hình đa phương thức, mô hình nhận diện chữ và mô hình công thức được gộp chung vào một cấu trúc dữ liệu trung gian. Quy trình này lặp lại cho đến khi xử lý hết các phân đoạn của tài liệu, sau đó hệ thống sẽ giải phóng tài nguyên bộ nhớ và hoàn tất việc tạo ra một văn bản đầu ra đồng nhất và giữ nguyên cấu trúc logic gốc.

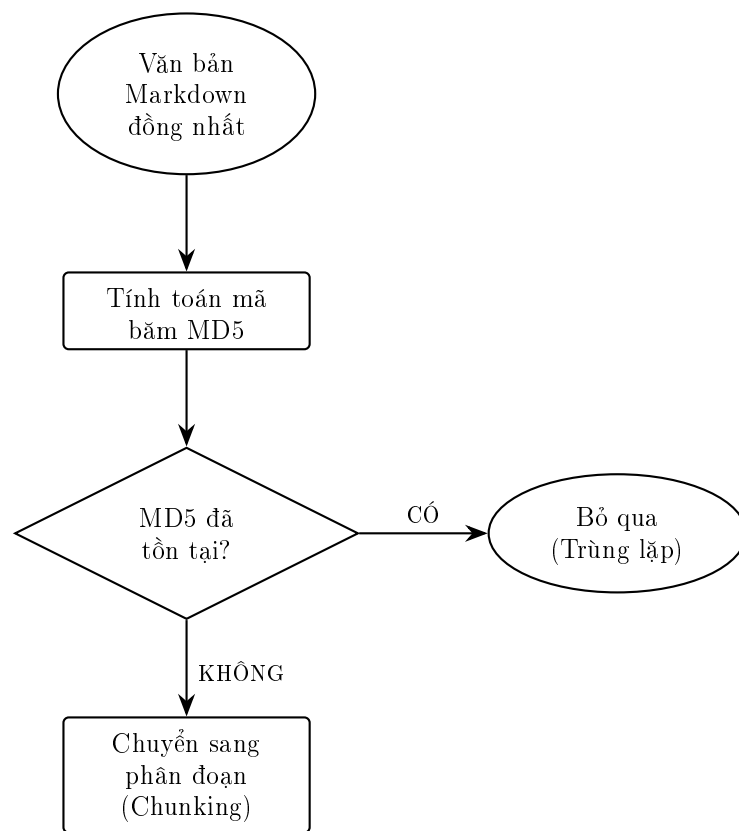
Nhờ đường ống Hybrid này, MinerU khắc phục hoàn toàn những nhược điểm của việc đọc tệp bằng `textract` hay `pypdf` thông thường, cung cấp dòng dữ liệu đầu vào sắc nét và có độ tin cậy tuyệt đối cho mô hình ngôn ngữ phía sau.

3.1.2 Luồng logic xử lý và cơ chế kiểm tra trùng lặp

Một trong những thành phần then chốt giúp hệ thống vận hành hiệu quả và tiết kiệm chi phí là cơ chế kiểm soát dữ liệu tại cổng vào. Sau khi văn bản được trích xuất từ các nguồn đa dạng (PDF, Office, Web) và đưa về định dạng Markdown đồng nhất, hệ thống thực hiện quy trình kiểm tra trùng lặp (Deduplication) dựa trên mã băm nội dung.

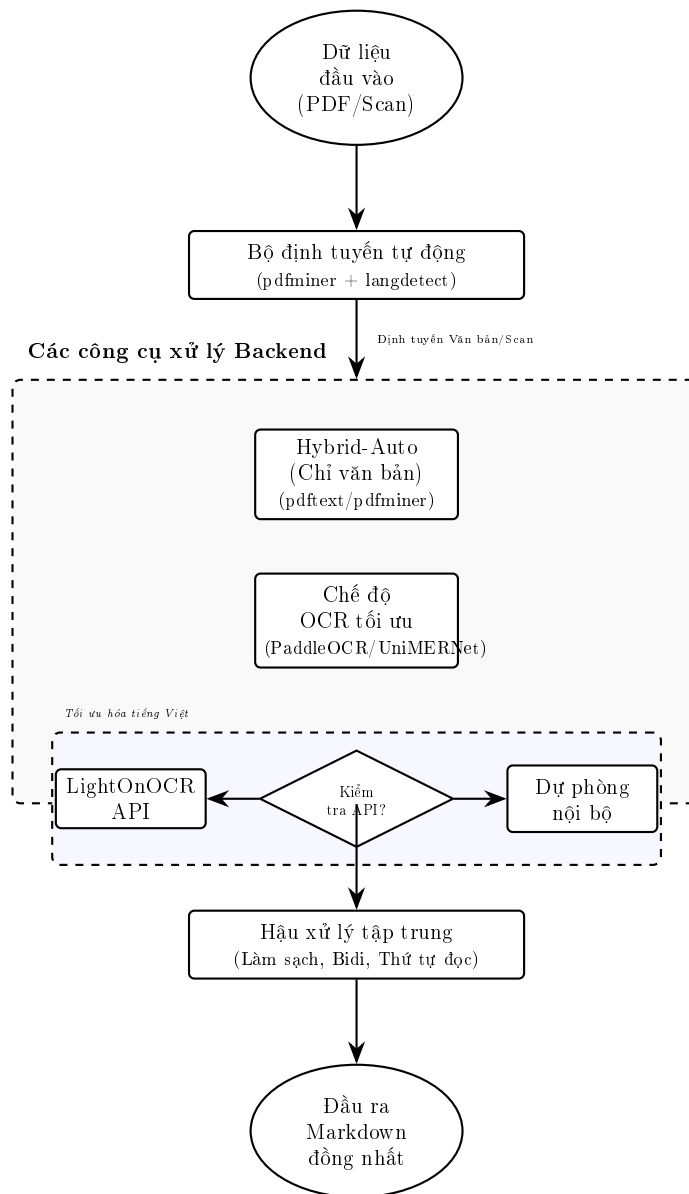
Cụ thể, hệ thống sử dụng thuật toán băm MD5 để tạo ra một "dấu vân tay" kỹ thuật số duy nhất cho toàn bộ nội dung văn bản. Mã băm này sau đó được đối chiếu với cơ sở dữ liệu tri thức hiện tại, dẫn đến hai kịch bản xử lý chính. Trong trường hợp trùng khớp hoàn toàn, nếu mã MD5

của tài liệu mới đã tồn tại, hệ thống sẽ ngay lập tức dừng quy trình xử lý nhằm ngăn chặn việc nạp lại dữ liệu dư thừa, bảo vệ tính toàn vẹn của đồ thị tri thức và tiết kiệm tối đa tài nguyên tính toán. Ngược lại, đối với trường hợp nội dung mới hoặc có sự thay đổi, tài liệu được coi là nguồn tri thức mới và được chuyển tiếp sang giai đoạn phân đoạn. Ngay cả khi tài liệu chỉ có một thay đổi nhỏ, hệ thống vẫn sẽ thực hiện lại quy trình để đảm bảo tính cập nhật, trong khi sự trùng lặp về mặt thực thể sẽ được tự động hợp nhất tại bước xây dựng đồ thị sau này.



Hình 3.3: Luồng logic kiểm tra trùng lặp bằng cơ chế băm nội dung

3.1.2.1 Hệ thống xử lý tài liệu đa định dạng

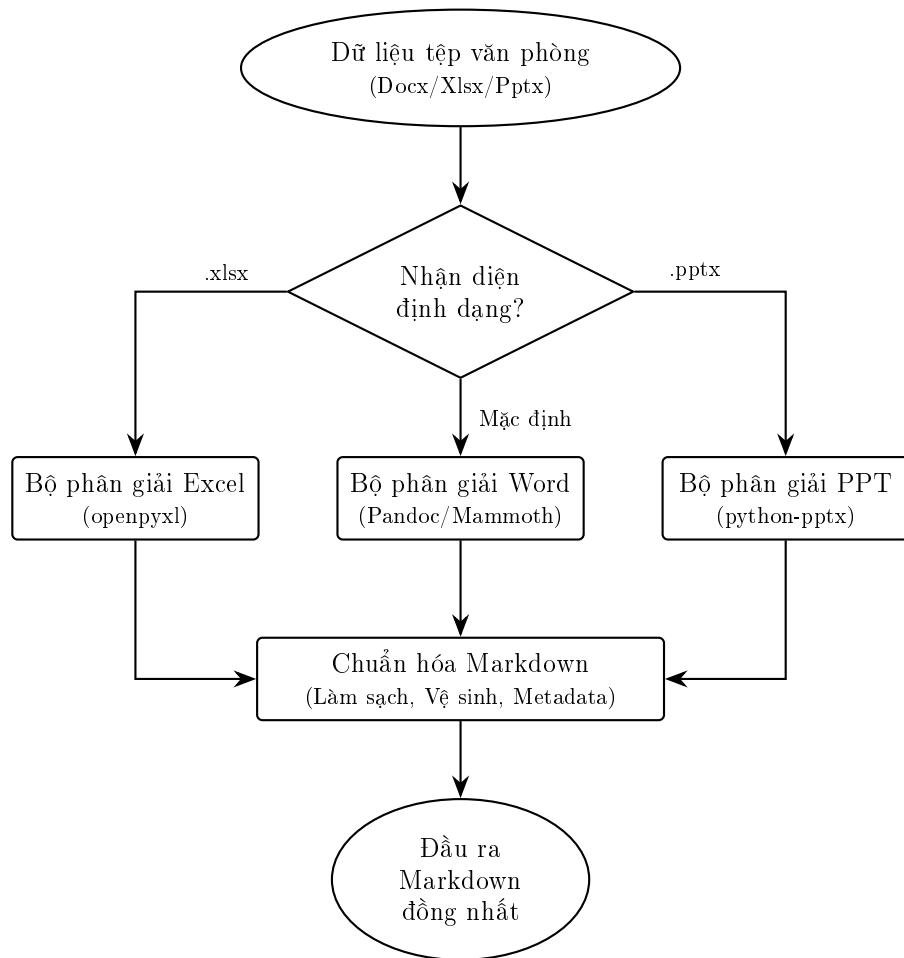


Hình 3.4: Chi tiết kiến trúc hệ thống VMiner hỗ trợ đa Backend

Dự án được phát triển dựa trên việc tùy biến hệ thống mã nguồn mở MinerU [30], tập trung vào tối ưu hóa công nghệ nhận dạng ký tự quang học chuyên biệt cho tiếng Việt. Mục tiêu cốt lõi là khắc phục triệt để những hạn chế trong việc nhận diện dấu tiếng Việt và xử lý các cấu trúc bảng biểu phức tạp. Hệ thống tự động hóa hoàn toàn quy trình chuyển đổi tài liệu từ PDF và hình ảnh sang chuẩn Markdown. Bên cạnh đó, dự

án tích hợp công cụ MarkItDown [29] để xử lý trực tiếp các định dạng văn phòng phổ biến như Word, Excel, PowerPoint một cách đồng bộ.

Quá trình chuyển đổi được thực hiện qua các giai đoạn chặt chẽ. Đầu tiên, hệ thống sử dụng các mô hình học sâu để phân tích cấu trúc hình học của trang tài liệu, tự động phân loại các vùng như đoạn văn bản, bảng biểu, hình ảnh và công thức toán học. Sau đó, từng vùng nội dung được điều hướng tới các bộ xử lý chuyên biệt. Các vùng văn bản được gửi đến các thư viện nhận diện ký tự để trích xuất chữ viết chính xác, trong khi các vùng bảng biểu được phân tích bởi các mô hình ngôn ngữ thị giác để tái tạo lại cấu trúc dữ liệu chuẩn xác.



Hình 3.5: Sơ đồ hoạt động của hệ thống MarkItDown

3.1.3 Chiến lược phân đoạn và chuẩn hóa văn bản

Quá trình xử lý khởi đầu bằng việc làm sạch dữ liệu đầu vào, loại bỏ các ký tự lỗi và đảm bảo văn bản tuân thủ chuẩn mã hóa quốc tế Unicode (UTF-8). Bước tiền xử lý này không chỉ chuẩn hóa định dạng mà còn loại bỏ các thẻ HTML dư thừa, khoảng trắng không cần thiết và các ký tự điều khiển có thể gây nhiễu cho mô hình ngôn ngữ ở các bước xử lý tiếp theo.

Về mặt kỹ thuật, hệ thống áp dụng song song hai chiến lược phân đoạn chính nhằm phục vụ hai mục tiêu khác biệt trong kiến trúc tổng thể: chiến lược phân đoạn theo kích thước cố định phục vụ trích xuất đồ thị tri thức và chiến lược phân đoạn phân cấp đa phương thức phục vụ truy xuất ngữ cảnh chính xác.

Chiến lược thứ nhất, phân đoạn theo kích thước cố định (fixed-size chunking), được thiết kế chuyên biệt để phục vụ quy trình trích xuất thực thể và quan hệ nhằm xây dựng đồ thị tri thức trong LightRAG [10]. Phương pháp này chia toàn bộ văn bản đã chuẩn hóa thành các đoạn có độ dài xác định trước, thường trong khoảng từ 512 đến 1200 token, kèm theo một vùng chồng lấp (overlap window) chiếm khoảng 10–20% kích thước đoạn giữa hai đoạn liên tiếp.

Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh” (context bridge): khi mô hình ngôn ngữ xử lý đoạn thứ i , nó đã có sẵn phần đuôi của đoạn thứ $i - 1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng. Cụ thể, với kích thước đoạn mặc định là 1200 token và vùng chồng lấp 100 token, hệ thống đảm bảo rằng các thực thể và mối quan hệ nằm gần ranh giới giữa hai đoạn liên tiếp vẫn được mô hình ngôn ngữ lớn nhận diện đầy đủ trong ít nhất một đoạn.

Chiến lược phân đoạn kích thước cố định được lựa chọn cho tác vụ trích xuất đồ thị do quy trình trích xuất thực thể và quan hệ trong LightRAG yêu cầu mỗi đoạn văn phải đủ dài để chứa ngữ cảnh phong phú, giúp mô hình ngôn ngữ lớn nhận diện chính xác các thực thể và suy luận được mối

liên hệ giữa chúng. Các đoạn quá ngắn sẽ thiếu ngữ cảnh, dẫn đến việc bỏ sót các quan hệ ngầm định giữa các câu không liền kề.

Chiến lược thứ hai, phân đoạn phân cấp (hierarchical chunking), được thiết kế nhằm bảo toàn tối đa cấu trúc và ngữ nghĩa của tài liệu gốc, phục vụ cho việc truy xuất ngữ cảnh chính xác khi hệ thống nhận truy vấn từ người dùng. Chiến lược này tham khảo từ MultiDocFusion [32], một hệ thống phân đoạn phân cấp được đề xuất cho các tài liệu dài và phức tạp, đã chứng minh hiệu quả vượt trội trong việc cải thiện độ chính xác truy xuất từ 8–15% so với các phương pháp phân đoạn truyền thống.

Khác với phân đoạn kích thước cố định vốn xử lý văn bản như một chuỗi ký tự thuần túy, chiến lược phân cấp đa phương thức nhận diện rằng tài liệu học thuật không chỉ chứa văn bản tuyến tính mà còn bao gồm nhiều thành phần có cấu trúc và ngữ nghĩa khác nhau. Do đó, quy trình phân đoạn không thể chỉ dựa trên số lượng token mà cần xem xét ba loại cấu trúc: (i) cấu trúc phân cấp (structural), tức mối quan hệ cha-con giữa các tiêu đề, mục, đoạn văn; (ii) cấu trúc bố cục (layout), tức vị trí vật lý, cột, vùng bao đóng của các thành phần trên trang; và (iii) cấu trúc ngữ nghĩa (semantic), tức mối quan hệ về nghĩa giữa các đoạn nội dung dù không kề nhau về vị trí vật lý.

Quy trình phân đoạn phân cấp đa phương thức trong hệ thống được triển khai qua bốn giai đoạn tuần tự, tương ứng với bốn bước chính trong kiến trúc MultiDocFusion [32]:

Giai đoạn 1: Phân tích cấu trúc tài liệu bằng thị giác máy tính (Vision-based Document Parsing). Hệ thống VMiner (đã trình bày ở mục trước) sử dụng các mô hình học sâu chuyên biệt để phân tích cấu trúc hình học của từng trang tài liệu. Quá trình này tự động phát hiện và phân loại các vùng nội dung thành các loại thành phần riêng biệt: vùng tiêu đề (heading), khối văn bản (text block), bảng biểu (table), hình ảnh và biểu đồ (figure/chart), danh sách (list), chú thích (footnote) và công thức toán học. Mỗi vùng được xác định bởi một hộp bao đóng (bounding box) chứa tọa độ chính xác trên trang, làm cơ sở cho việc sắp xếp thứ tự đọc hợp lý

và phân loại nội dung ở các bước tiếp theo.

Giai đoạn 2: Trích xuất nội dung và phân loại thành phần đa phương thức. Sau khi nhận diện các vùng nội dung, hệ thống tiến hành trích xuất nội dung tương ứng từ từng vùng. Đối với các vùng văn bản thuần túy, nội dung được trích xuất bằng các công cụ nhận dạng ký tự quang học (OCR) đã tối ưu hóa cho tiếng Việt. Đối với bảng biểu, hệ thống tái tạo cấu trúc dữ liệu dưới dạng Markdown hoặc HTML, bảo toàn nguyên vẹn thông tin hàng, cột và tiêu đề. Đối với hình ảnh và biểu đồ, hệ thống sinh mô tả văn bản tự động thông qua mô hình ngôn ngữ thị giác (Vision Language Model, VLM), kết hợp với chú thích (caption) gốc và văn bản ngữ cảnh xung quanh hình ảnh trong tài liệu. Bước phân loại này đảm bảo mỗi thành phần được xử lý bởi chiến lược phù hợp nhất với đặc trưng dữ liệu của nó.

Giai đoạn 3: Xây dựng cây cấu trúc phân cấp (Hierarchical Structure Parsing). Đây là giai đoạn cốt lõi phân biệt chiến lược phân cấp với các phương pháp phân đoạn truyền thống. Hệ thống tái cấu trúc toàn bộ tài liệu thành một cây phân cấp (hierarchical tree) phản ánh tổ chức logic của nội dung, trong đó gốc cây là tài liệu, các nút trung gian là các mục (section) và tiểu mục (sub-section), còn các nút lá là các đơn vị nội dung cụ thể như đoạn văn, bảng, hình ảnh. Cấu trúc phân cấp này được suy luận từ hệ thống tiêu đề (heading hierarchy) trong tài liệu, cụ thể các cấp tiêu đề H1, H2, H3 tương ứng với các mức độ lồng nhau khác nhau trong cây. Trong trường hợp tài liệu không có tiêu đề rõ ràng hoặc cấu trúc tiêu đề bị nhiễu, hệ thống bổ sung cơ chế phân tích dựa trên mô hình ngôn ngữ lớn (tương tự kỹ thuật DSHP-LLM trong MultiDocFusion [32]) để suy luận ranh giới ngữ nghĩa giữa các phần. Mỗi nút trong cây được gán một đường dẫn phân cấp (breadcrumb path) dạng: [Tài liệu] > [Chương X] > [Mục X.Y] > [Tiểu mục X.Y.Z], cung cấp ngữ cảnh vị trí chính xác cho từng đơn vị nội dung.

Giai đoạn 4: Gom nhóm và sinh phân đoạn theo chiến lược DFS (DFS-based Grouping). Giai đoạn cuối cùng thực hiện việc duyệt cây phân cấp

theo chiều sâu (Depth-First Search) để gom nhóm các nút lá thành các phân đoạn hoàn chỉnh, tuân thủ giới hạn kích thước token tối đa đã được cấu hình. Thuật toán duyệt DFS đảm bảo rằng các nút con thuộc cùng một nút cha sẽ được ưu tiên gom vào cùng một phân đoạn, từ đó bảo toàn tính liên kết ngữ nghĩa theo cấu trúc phân cấp gốc. Cụ thể, quy tắc gom nhóm được áp dụng như sau:

- Nếu thành phần là một đoạn văn ngắn (dưới 100 token): gộp với các đoạn văn tiếp theo thuộc cùng mục (section) cho đến khi đạt ngưỡng kích thước tối thiểu.
- Nếu thành phần là một đoạn văn dài (vượt quá giới hạn token): áp dụng kỹ thuật tách nhận biết ranh giới câu (sentence-aware splitting) để tránh cắt ngang ý tưởng.
- Nếu thành phần là bảng biểu: giữ nguyên toàn bộ bảng như một phân đoạn độc lập kèm theo chú thích bảng và ngữ cảnh mục chứa bảng. Đối với bảng quá lớn vượt giới hạn token, áp dụng chiến lược tách theo nhóm hàng kèm giữ nguyên hàng tiêu đề.
- Nếu thành phần là hình ảnh hoặc biểu đồ: tạo phân đoạn hình ảnh chứa mô tả sinh bởi VLM, chú thích (caption) gốc và đoạn văn bản ngữ cảnh trước/sau hình ảnh, thiết lập liên kết chéo giữa các thành phần này.
- Nếu thành phần là slide (đối với tài liệu PPTX): giữ nguyên toàn bộ nội dung slide gồm tiêu đề, khung văn bản, danh sách và ghi chú diễn giả làm một phân đoạn; tách khi nội dung vượt giới hạn.

Mỗi phân đoạn được tạo ra sẽ được nhúng (inject) thêm thông tin đường dẫn phân cấp (breadcrumb context) vào đầu nội dung, theo định dạng:

```
[Document: {tên_tài_liệu}] > [{đường_dẫn_mục}]
{nội_dung_gốc}
```

Kỹ thuật nhúng ngữ cảnh phân cấp này giúp mô hình ngôn ngữ lớn có đủ thông tin định vị ngay cả khi phân đoạn bị tách khỏi tài liệu gốc. Breadcrumb được nhúng vào đầu nội dung nhưng không được tính vào số lượng token của phần nội dung chính, nhằm không ảnh hưởng đến chất lượng biểu diễn vector nhúng (embedding) của nội dung thực.

Bảng 3.1 trình bày so sánh chi tiết giữa hai chiến lược phân đoạn được áp dụng trong hệ thống.

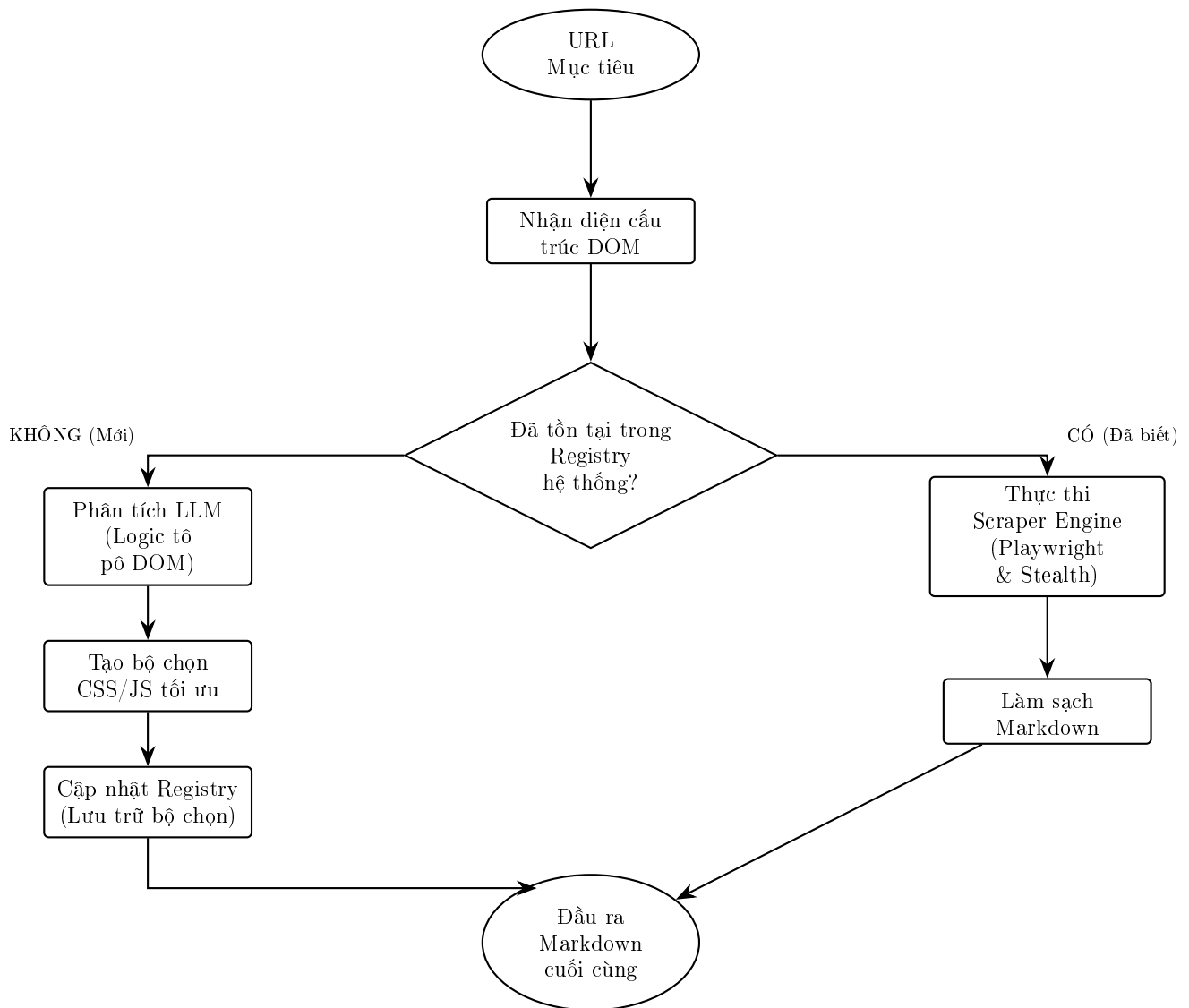
Bảng 3.1: So sánh hai chiến lược phân đoạn trong hệ thống

Tiêu chí	Chiến lược 1: Kích thước cố định	Chiến lược 2: Phân cấp đa phương thức
Mục đích chính	Trích xuất thực thể và quan hệ cho đồ thị tri thức	Truy xuất ngữ cảnh chính xác cho RAG
Đơn vị phân đoạn	Cửa sổ token cố định (1200 token)	Đơn vị ngữ nghĩa tự nhiên (đoạn văn, bảng, hình)
Bảo toàn cấu trúc	Không, xử lý văn bản thuần tuyến tính	Có, bảo toàn cấu trúc phân cấp gốc
Ngữ cảnh vị trí	Dựa vào vùng chồng lấp	Breadcrumb path nhúng vào mỗi chunk
Ưu điểm	Đơn giản, phân bổ tải đều, phù hợp cho trích xuất hàng loạt	Bảo toàn ngữ nghĩa, truy xuất chính xác, hỗ trợ đa phương thức
Hạn chế	Có thể cắt ngang ý tưởng	Phức tạp hơn, phụ thuộc vào chất lượng phân tích cấu trúc

Sự phối hợp giữa hai chiến lược phân đoạn tạo nên một kiến trúc lưu trữ kép (dual-indexing architecture) phục vụ cho hai luồng truy xuất khác nhau trong hệ thống. Chiến lược phân đoạn kích thước cố định cung cấp nguyên liệu đầu vào cho quy trình xây dựng đồ thị tri thức, nơi các thực thể và quan hệ được trích xuất và lưu trữ trong cơ sở dữ liệu đồ thị, phục vụ

cho các chế độ truy vấn cục bộ (local) và toàn cục (global) của LightRAG. Trong khi đó, chiến lược phân đoạn phân cấp đa phương thức cung cấp các phân đoạn có cấu trúc ngữ nghĩa chặt chẽ, được lưu trữ trong cơ sở dữ liệu vector kèm theo siêu dữ liệu phong phú (metadata), phục vụ cho việc truy xuất ngữ cảnh chính xác khi hệ thống cần bổ sung thông tin chi tiết từ tài liệu gốc. Kiến trúc kép này đảm bảo rằng hệ thống vừa có khả năng lập luận đa bước trên đồ thị tri thức, vừa có khả năng truy xuất ngữ cảnh tài liệu gốc một cách trung thực và đầy đủ.

3.1.4 Hệ thống thu thập tri thức từ trang web



Hình 3.6: Quy trình thu thập tri thức từ trang web với Smart Scraper

Hệ thống cào web thông minh hoạt động theo quy trình tự động hóa từ việc nhận diện cấu trúc trang đến trích xuất dữ liệu. Logic cốt lõi dựa trên công cụ Smart Scraper Engine với cơ chế vận hành qua bốn bước chính: lấy dấu vân tay, đối chiếu kho lưu trữ, phân tích cấu trúc bằng mô hình ngôn ngữ và thực thi bộ phân giải.

Quy trình bắt đầu bằng việc tạo ra một bản tóm tắt cấu trúc cây DOM

của trang web, tập trung vào các thuộc tính định danh như thẻ HTML, tên lớp và ID. Hệ thống trích xuất một bộ khung xương kỹ thuật của trang web để làm cơ sở cho việc nhận diện. Sau đó, bản tóm tắt này được đối chiếu với kho lưu trữ các bộ phân giải đã biết. Mỗi mục trong kho lưu trữ chứa các thông tin về dấu hiệu nhận biết, bộ chọn nội dung chính, các thành phần cần loại bỏ và chiến lược chờ đợi phù hợp cho từng loại công nghệ web như React, Vue, hay các hệ quản trị nội dung truyền thống.

Trong trường hợp gặp một trang web mới không có trong kho lưu trữ, hệ thống sẽ gửi một phần cấu trúc DOM tiêu biểu cho mô hình ngôn ngữ lớn. Mô hình này có nhiệm vụ phân tích logic phân bố nội dung và tự động sinh ra mã lệnh điều hướng. Mã lệnh này bao gồm các bộ chọn CSS tối ưu để trích xuất văn bản có ý nghĩa và các quy tắc loại bỏ những thành phần gây nhiễu như trình đơn điều hướng, biểu ngữ quảng cáo hoặc các đoạn mã kịch bản. Bộ phân giải mới sinh ra sẽ được kiểm tra và lưu lại vào kho lưu trữ để sử dụng cho các lần truy cập sau, giúp tối ưu hóa hiệu suất và giảm thiểu chi phí gọi API.

Đối với các tên miền mặc định như `fit.hcmus.edu.vn` và `hcmus.edu.vn`, hệ thống đã được cấu hình sẵn các bộ phân giải chuyên biệt cho nền tảng DotNetNuke. Do đặc thù của nền tảng này thường chứa nhiều nội dung lồng ghép trong các khung phức tạp, hệ thống sử dụng các bộ chọn mục tiêu như `.dnn_contentPane` và `#dnn_ContentPane` để định vị chính xác vùng chứa thông tin học thuật. Quy trình trích xuất văn bản tại đây không chỉ lấy chữ thuần túy mà còn thực hiện chuyển đổi các liên kết quan trọng sang định dạng Markdown để bảo toàn thông tin tham chiếu.

Đặc biệt, đối với các trang có tính tương tác cao như danh mục môn học hoặc đề cương chi tiết, hệ thống áp dụng chế độ tương tác vết càn. Chế độ này sử dụng thư viện Playwright để tự động thực hiện các hành động như nhấn vào các nút mở rộng, cuộn trang để tải thêm dữ liệu và chờ đợi các tiến trình AJAX hoàn tất. Điều này đảm bảo rằng toàn bộ tri thức về chương trình đào tạo, mô tả môn học và thông tin giảng viên được thu thập đầy đủ nhất, ngay cả khi chúng nằm sau các thành phần giao

điện động. Dữ liệu sau khi trích xuất sẽ được làm sạch và chuẩn hóa trước khi đưa vào các bước phân đoạn và xây dựng đồ thị tri thức tiếp theo.

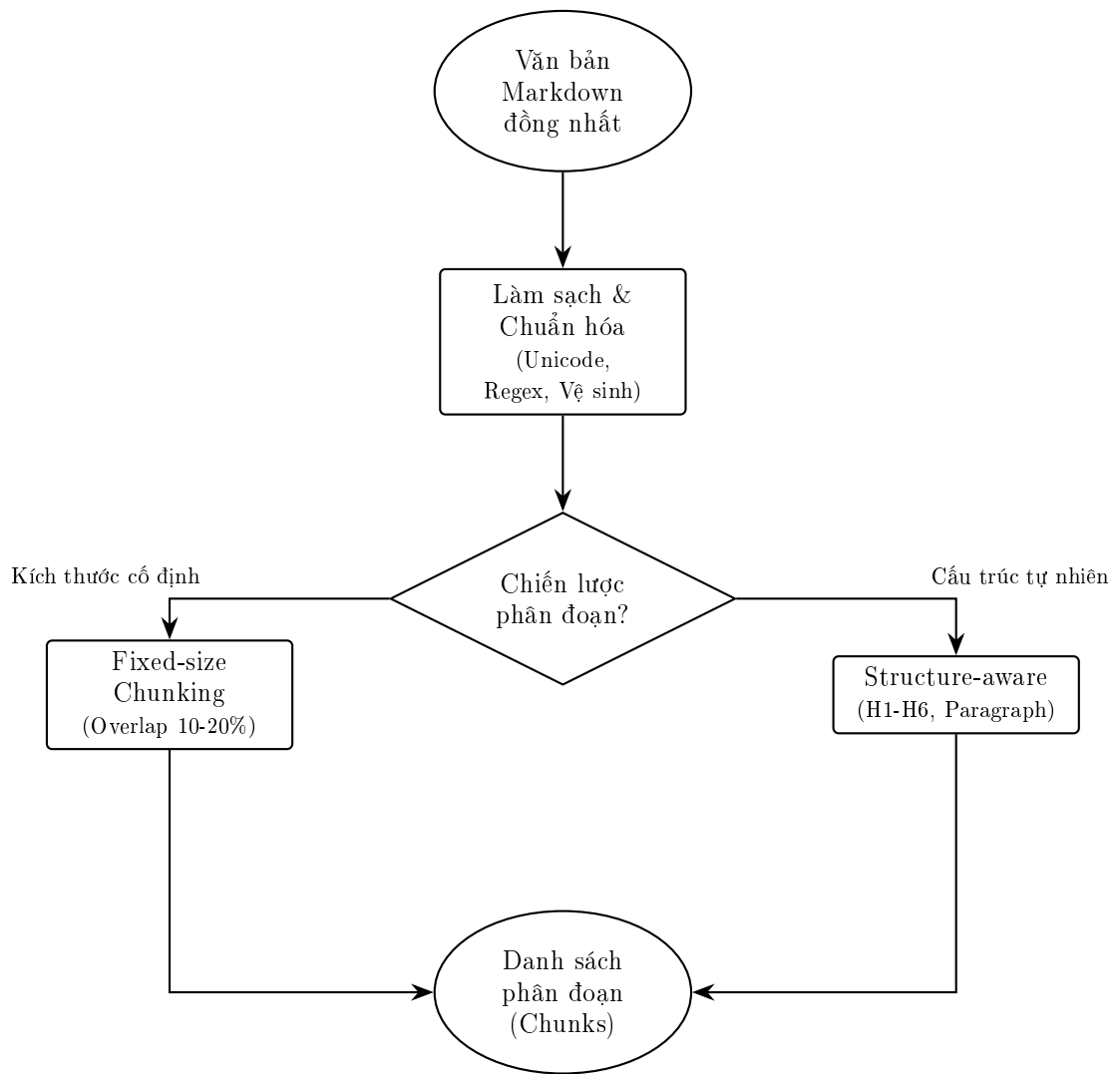
3.1.5 Quy trình điều phối và phân đoạn dữ liệu

Sau khi dữ liệu vượt qua bước kiểm tra trùng lặp, hệ thống bước vào giai đoạn điều phối và phân đoạn. Đây là quy trình then chốt nhằm chuyển đổi các văn bản Markdown dài thành những đơn vị tri thức có kích thước tối ưu, sẵn sàng cho việc trích xuất thực thể và quan hệ. Quy trình điều phối này không chỉ đơn thuần là cắt nhỏ văn bản mà còn đảm bảo tính logic và ngữ cảnh giữa các phần.

LightRAG [10] thực hiện quy trình này thông qua hai chiến lược phân đoạn chính để bảo toàn tính toàn vẹn tri thức:

Phương pháp thứ nhất, phân đoạn theo kích thước cố định (fixed-size chunking), chia văn bản thành các đoạn có độ dài xác định trước (thường từ 512 đến 1200 token), kèm theo một vùng chồng lấp (overlap window) chiếm khoảng 10–20% kích thước đoạn giữa hai đoạn liên tiếp. Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh”: khi mô hình xử lý đoạn thứ i , nó đã có sẵn phần đuôi của đoạn thứ $i-1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng.

Phương pháp thứ hai, phân đoạn theo cấu trúc tự nhiên (structure-aware chunking), khai thác các dấu hiệu cấu trúc sẵn có trong tài liệu như tiêu đề, tiêu đề phụ, danh sách có đánh số, hay các đoạn văn được phân tách bởi ký tự xuống dòng kép. Hệ thống sử dụng các biểu thức chính quy (regex) kết hợp với phân tích cú pháp nhẹ (lightweight parsing) để nhận diện ranh giới tự nhiên này. Ưu điểm của phương pháp này là mỗi đoạn văn giữ nguyên tính nguyên vẹn ngữ nghĩa — ví dụ, toàn bộ một mục trong tài liệu kỹ thuật sẽ không bị tách ra giữa chừng.



Hình 3.7: Luồng logic xử lý từ dữ liệu thô đến giai đoạn phân đoạn

3.1.6 Cơ chế trích xuất thực thể và xây dựng quan hệ

Sau khi văn bản được chia thành các phân đoạn (chunk), LightRAG thực hiện quy trình trích xuất tri thức để chuyển đổi dữ liệu phi cấu trúc thành các thành phần đồ thị tri thức tường minh. Khác với các hệ thống RAG truyền thống thường bỏ qua mối liên kết giữa các đoạn văn, cơ chế này tập trung vào việc nhận diện các thực thể (Entities) và thiết lập các quan hệ (Relationships) xuyên suốt tài liệu.

3.1.6.1 Chiến lược trích xuất đa nhiệm trong một lần gọi (One-step Extraction)

Để tối ưu hóa chi phí tính toán và duy trì tính nhất quán ngữ nghĩa, LightRAG sử dụng một lời nhắc có cấu trúc (structured prompt) yêu cầu mô hình ngôn ngữ lớn (LLM) thực hiện đồng thời hai nhiệm vụ trích xuất trong một lần gọi API duy nhất.

Nhiệm vụ 1: Nhận diện thực thể (Entity Extraction). Mô hình được yêu cầu xác định tất cả các thực thể quan trọng xuất hiện trong chunk. Đối với mỗi thực thể, LLM phải trích xuất các thuộc tính cốt lõi bao gồm tên định danh (Entity Name), loại thực thể (Entity Type) như PERSON, ORGANIZATION, CONCEPT, EVENT,... và một đoạn văn bản ngắn mô tả (Entity Description) giải thích vai trò, đặc điểm và ngữ cảnh của thực thể đó. Khác với NER truyền thống [23], phương pháp này khai thác tối đa khả năng hiểu ngữ nghĩa sâu của LLM để tạo ra các mô tả tri thức phong phú.

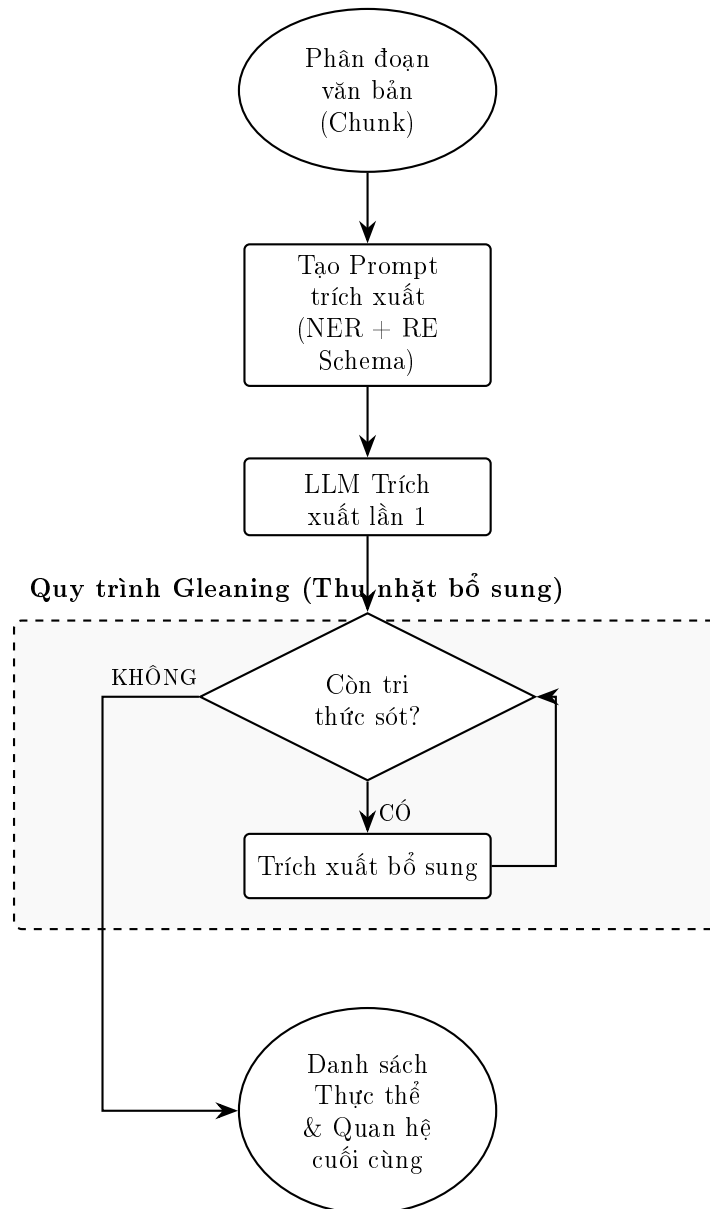
Nhiệm vụ 2: Trích xuất quan hệ (Relation Extraction). Sau khi xác định các thực thể, mô hình thực hiện quét các cặp thực thể để tìm kiếm các mối liên kết có ý nghĩa. Các thuộc tính trích xuất bao gồm hai thực thể tham gia (Source & Target), các từ khóa định nghĩa bản chất liên kết (Relationship Keywords) và mô tả chi tiết (Relationship Description) lý do tại sao hai thực thể này có liên quan đến nhau dựa trên bằng chứng

trong văn bản.

3.1.6.2 Cơ chế Gleaning: Tự chỉnh sửa và Thu nhặt bổ sung

Một trong những thách thức lớn nhất của việc trích xuất tri thức bằng LLM là hiện tượng bỏ sót thông tin do giới hạn về khả năng chú ý (attention) khi xử lý các đoạn văn phức tạp. LightRAG giải quyết vấn đề này thông qua kỹ thuật *Gleaning* (thu nhặt bổ sung).

Sau vòng trích xuất đầu tiên, hệ thống không kết thúc ngay mà thực hiện gửi lại danh sách tri thức vừa tìm được cho LLM kèm theo một yêu cầu phản biện: “Dựa trên đoạn văn này và danh sách thực thể/quan hệ đã có, liệu còn bất kỳ chi tiết nào bị bỏ sót không?”. Nếu mô hình phát hiện thêm tri thức mới, nó sẽ trả về kết quả bổ sung để hệ thống hợp nhất vào kết quả ban đầu. Quy trình này lặp lại cho đến khi mô hình xác nhận không còn thông tin sót hoặc đạt giới hạn số vòng lặp cấu hình.



Hình 3.8: Luồng logic quy trình trích xuất tri thức và cơ chế Gleaning

Kết quả cuối cùng là một tập hợp các thực thể và quan hệ có cấu trúc, sẵn sàng cho bước hợp nhất đồ thị (Graph Merging). Cơ chế này đảm bảo rằng mỗi đoạn văn bản đều được khai thác triệt để tri thức, tạo tiền đề cho việc xây dựng một đồ thị tri thức dày đặc và chính xác.

3.1.7 Luồng xử lý Prompt linh hoạt cho đa dạng nguồn dữ liệu

Một thách thức lớn trong việc xây dựng đồ thị tri thức cho môi trường học thuật là sự khác biệt sâu sắc về cấu trúc và ngôn từ giữa các nguồn tài liệu. Đề cương chi tiết môn học thường mang tính quy phạm, ngắn gọn và sử dụng dày đặc các mã ký hiệu viết tắt như G1, G2 (mục tiêu môn học), CLO (chuẩn đầu ra), hay các thuật ngữ quản lý đào tạo như tích lũy, tiền quyết. Trong khi đó, dữ liệu từ hệ thống website của khoa lại mang tính chất thông tin đại chúng, chứa nhiều tin tức, sự kiện và mô tả tổng quát. Để giải quyết vấn đề này, hệ thống đã triển khai luồng xử lý lời nhắc linh hoạt (Dynamic Prompt Flow) với cơ chế tự động điều phối dựa trên miền tri thức.

Quy trình bắt đầu bằng bước phân loại tài liệu ngay tại tầng trích xuất. Hệ thống thực hiện quét sơ bộ các dấu hiệu định dạng để xác định tài liệu thuộc nhóm "Chương trình đào tạo" hay "Thông tin Website". Sau khi nhận diện, bộ điều phối lời nhắc (Prompt Dispatcher) sẽ kích hoạt các bản mẫu (templates) chuyên biệt. Đối với nhóm đề cương, lời nhắc tập trung vào việc giải mã các mã ký hiệu: hệ thống hướng dẫn mô hình ngôn ngữ lớn thiết lập các mối liên kết ngữ nghĩa giữa mã hiệu (G1) và nội dung thực tế mà nó đại diện, đồng thời trích xuất các thực thể đặc thù như LEARNING OUTCOME hay ASSESSMENT METHOD. Đối với nhóm website, lời nhắc lại ưu tiên việc trích xuất các thực thể về sự kiện, mốc thời gian và các thực thể vai trò người (giảng viên, cán bộ).

Chiến lược xử lý viết tắt được tích hợp trực tiếp vào trong nội dung lời nhắc thông qua kỹ thuật Few-shot Learning. Hệ thống cung cấp các ví dụ mẫu về cách chuyển đổi từ các từ viết tắt phổ biến của khoa (như FIT, HCMUS, DHQG) sang thực thể đầy đủ, đồng thời ép mô hình phải duy trì tính nhất quán về định danh (Entity Canonicalization) ngay từ bước sơ khởi. Việc này giúp giảm thiểu đáng kể số lượng thực thể phân mảnh trong đồ thị và đảm bảo rằng các nút trích xuất từ các nguồn khác nhau

nhưng cùng chỉ về một đối tượng sẽ dễ dàng được hợp nhất ở các bước sau. Cơ chế này đóng vai trò nền tảng để biến những dữ liệu thô, rời rạc thành một mạng lưới tri thức có tính liên kết chặt chẽ và chiều sâu ngữ nghĩa.

3.1.8 Kiểm chứng tri thức bằng chuẩn SHACL và cơ chế Staging

Để đảm bảo tính toàn vẹn và chất lượng của dữ liệu tri thức trước khi đưa vào đồ thị chính thức, hệ thống tích hợp chuẩn SHACL (Shapes Constraint Language) [36]. SHACL cung cấp một ngôn ngữ mạnh mẽ để mô tả các ràng buộc về cấu trúc và ngữ nghĩa của dữ liệu đồ thị, giúp tự động hóa quy trình kiểm tra các lỗi phổ biến trong quá trình trích xuất bằng mô hình ngôn ngữ lớn [19].

Cơ chế kiểm chứng được tổ chức theo quy trình 3 lớp bảo vệ (3-Guard). Lớp thứ nhất thực hiện kiểm tra trùng lặp để tránh việc nạp lại các đoạn văn bản đã tồn tại trong hệ thống. Lớp thứ hai là trọng tâm của việc kiểm chứng SHACL, sử dụng các định nghĩa hình dạng (Shapes) để kiểm tra tính hợp lệ của từng thực thể và quan hệ. Các quy tắc bao gồm kiểm tra sự tồn tại của tên và kiểu thực thể, độ dài tối thiểu của mô tả, dải giá trị của trọng số và đặc biệt là kiểm tra các thực thể bị cô lập (không có quan hệ kết nối). Hệ thống phân loại mức độ vi phạm thành các cấp độ như lỗi (error), cảnh báo (warning) hoặc thông tin (info) để đưa ra các đề xuất xử lý phù hợp.

Kết quả kiểm chứng SHACL trực tiếp điều hướng dữ liệu vào vùng đệm Staging. Thay vì trực tiếp hợp nhất các thực thể và quan hệ vừa trích xuất, hệ thống lưu trữ chúng vào một cơ sở dữ liệu tạm thời kèm theo nhật ký kiểm chứng. Lớp bảo vệ thứ ba thực hiện tổng hợp báo cáo và xác định trạng thái sẵn sàng của dữ liệu. Các dữ liệu vượt qua tất cả các bài kiểm tra nghiêm ngặt sẽ được đề xuất trạng thái phê duyệt (Approved), trong khi các dữ liệu có vi phạm mức cảnh báo như thực thể cô lập sẽ được đưa vào danh sách cần xem xét (Pending Review). Các trường hợp vi phạm

ngghiêm trọng về cấu trúc sẽ bị đề xuất từ chối (Rejected).

Sự kết hợp giữa SHACL và cơ chế Staging tạo ra một quy trình kiểm soát chất lượng dữ liệu chặt chẽ. Nó không chỉ giúp tự động hóa việc phát hiện sai sót của mô hình ngôn ngữ mà còn cung cấp giao diện để người quản trị có thể can thiệp, chỉnh sửa và phê duyệt dữ liệu một cách minh bạch. Điều này đảm bảo rằng đồ thị tri thức cuối cùng luôn duy trì được tính nhất quán, đầy đủ và đáng tin cậy cao.

3.1.9 Hợp nhất đồ thị và chuẩn hóa kho tri thức

Sau khi toàn bộ các đoạn văn được xử lý, LightRAG tiến hành hợp nhất tất cả các thực thể và quan hệ được trích xuất vào một đồ thị tri thức toàn cục duy nhất (global knowledge graph). Quá trình này đặt ra thách thức về độ trùng lặp: cùng một thực thể có thể xuất hiện với tên gọi khác nhau (ví dụ “Mạng nơ-ron” và “Neural Network”), hoặc các quan hệ tương đương có thể được diễn đạt theo nhiều cách.

LightRAG giải quyết vấn đề này thông qua cơ chế hợp nhất thực thể (entity merging): các thực thể có tên trùng khớp (sau khi chuẩn hóa chữ hoa/thường và loại bỏ khoảng trắng thừa) được gộp thành một nút duy nhất trong đồ thị. Mô tả của các phiên bản khác nhau không bị loại bỏ mà được nối lại (concatenate) để tạo ra một mô tả tổng hợp toàn diện hơn, phản ánh tất cả ngữ cảnh mà thực thể đó xuất hiện trong tài liệu gốc. Đồng thời, siêu dữ liệu nguồn gốc (provenance metadata) — bao gồm ID đoạn văn, tên tài liệu, số trang — được lưu trữ kèm theo mỗi nút và cạnh, phục vụ cho việc truy vết khi cần kiểm chứng thông tin.

Một thành phần quan trọng trong quá trình này là cơ chế staging, đóng vai trò là vùng đệm trung gian trước khi dữ liệu được chính thức đưa vào đồ thị tri thức toàn cục. Thay vì trực tiếp hợp nhất các thực thể và quan hệ vừa trích xuất, hệ thống lưu trữ chúng vào một cơ sở dữ liệu tạm thời kèm theo nhật ký kiểm chứng. Điểm đặc biệt của vùng staging là khả năng thực hiện quy trình kiểm tra chéo (Reflection) thông qua một mô

hình ngôn ngữ lớn đóng vai trò là Critic LLM.

Khi dữ liệu được đưa vào staging, hệ thống sẽ truy xuất lại nội dung gốc của đoạn văn bản (thông qua `chunk_id`) để đối chiếu với danh sách các thực thể và quan hệ đã được trích xuất. Mô hình Critic LLM sẽ phân tích ngữ cảnh một cách sâu sắc hơn để phát hiện các thông tin bị bỏ sót hoặc các mối quan hệ ngầm định mà quy trình trích xuất ban đầu chưa nhận diện được. Kỹ thuật này được gọi là thu nhặt bổ sung (gleaning), cho phép hệ thống thực hiện nhiều vòng lặp duyệt lại dữ liệu. Trong mỗi vòng lặp, Critic LLM sẽ đưa ra các đề xuất về thực thể mới hoặc các liên kết bổ sung để tăng tính đầy đủ của đồ thị.

Ngoài việc đề xuất thông tin mới, vùng staging còn cung cấp một cơ chế kiểm soát chất lượng nghiêm ngặt. Mỗi thực thể và quan hệ đều được gán các trạng thái như đang chờ duyệt, đã chấp nhận, đã chỉnh sửa hoặc bị từ chối dựa trên điểm tin cậy và kết quả kiểm chứng SHACL. Nhật ký kiểm chứng (audit log) ghi lại chi tiết các phát hiện của Critic LLM, giúp người quản trị có cái nhìn toàn diện về tính chính xác của từng thành phần tri thức. Quy trình này đảm bảo rằng chỉ những dữ liệu đã được kiểm chứng và bổ sung đầy đủ mới được phép tham gia vào quá trình merge chính thức, từ đó tối ưu hóa tính toàn vẹn và giá trị sử dụng của kho tri thức toàn cục.

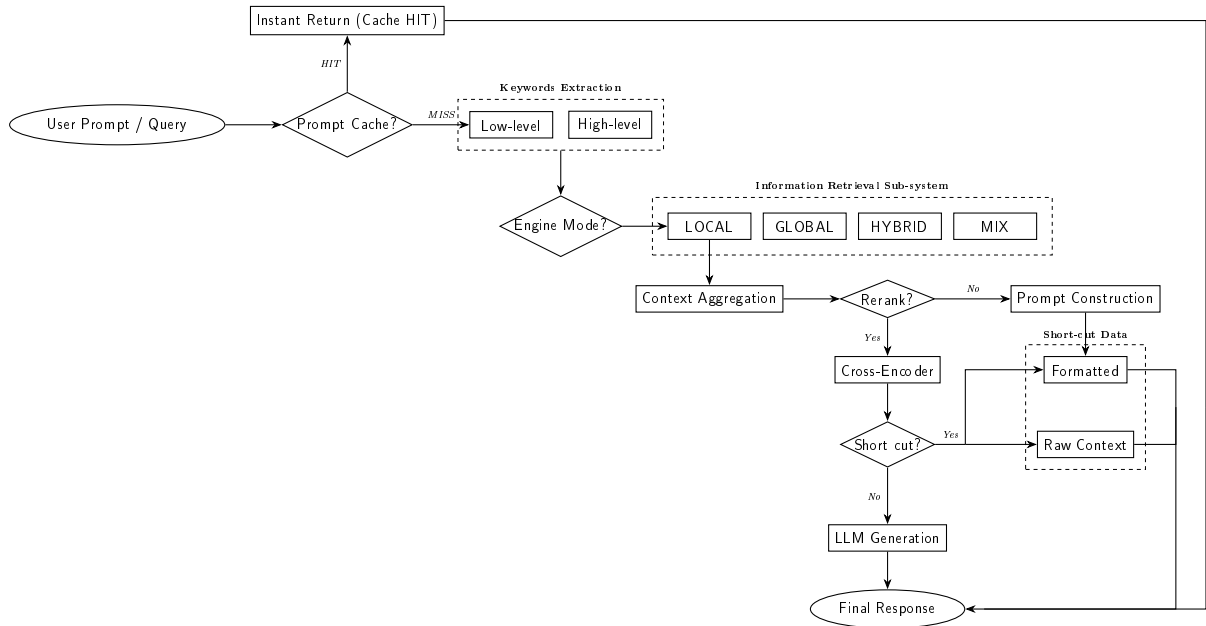
Song song với việc xây dựng đồ thị, hệ thống thực hiện đồng bộ hóa với cơ sở dữ liệu vector (vector store synchronization). Mỗi thực thể và quan hệ sau khi chuẩn hóa được chuyển đổi thành một véc-tơ nhúng (embedding vector) thông qua một mô hình nhúng chuyên dụng. Điều này tạo ra một kiến trúc lưu trữ kép (dual storage): đồ thị tri thức lưu giữ cấu trúc quan hệ tường minh (explicit relational structure), trong khi không gian vector cho phép tìm kiếm ngữ nghĩa mờ (fuzzy semantic search) dựa trên độ tương đồng cosine. Sự kết hợp này là nền tảng cho hai chế độ truy vấn đặc trưng của LightRAG.

3.1.10 Quản lý và duy trì bền vững hệ thống

LightRAG thiết kế hệ thống quản trị với triết lý nhất quán tăng dần (incremental consistency): thay vì xây dựng lại toàn bộ đồ thị mỗi khi có tài liệu mới, hệ thống chỉ xử lý các đoạn văn mới hoặc đã thay đổi, sau đó hợp nhất kết quả vào đồ thị hiện có. Cơ chế làm sạch tự động (auto-cleaning) thường xuyên kiểm tra tính nhất quán giữa đồ thị tri thức và nội dung tài liệu gốc: nếu một tài liệu bị xóa khỏi kho dữ liệu, tất cả các nút và cạnh có nguồn gốc duy nhất từ tài liệu đó sẽ tự động bị loại bỏ, trong khi các nút được hỗ trợ bởi nhiều nguồn sẽ chỉ được cập nhật lại mô tả.

Người quản trị có thể can thiệp thủ công thông qua các API quản trị: hiệu chỉnh nhãn thực thể (entity label editing), cập nhật mô tả hoặc hợp nhất thủ công hai thực thể bị nhận diện nhầm là khác nhau. Sau bất kỳ thay đổi nào, hệ thống kích hoạt quy trình đồng bộ hóa tự động: véc-tơ nhúng của các thực thể bị ảnh hưởng được tính toán lại và cập nhật trong cơ sở dữ liệu vector, đảm bảo sự nhất quán tuyệt đối giữa hai lớp lưu trữ.

Cuối cùng, LightRAG hỗ trợ tùy chỉnh lời nhắc trích xuất theo lĩnh vực (domain-specific prompt tuning): người dùng có thể cung cấp danh sách các kiểu thực thể ưu tiên và các mẫu quan hệ đặc thù, từ đó định hướng mô hình tập trung vào tri thức có giá trị nhất trong từng miền chuyên môn cụ thể.



Hình 3.9: Quy trình truy vấn đa chế độ và cơ chế tối ưu phản hồi trong LightRAG

Một trong những ưu điểm nổi bật nhất của LightRAG so với các hệ thống RAG truyền thống là khả năng thực hiện các truy vấn đa chế độ, cho phép hệ thống đáp ứng linh hoạt các loại câu hỏi khác nhau từ người học. Hệ thống hỗ trợ bốn chế độ truy vấn chính: Naive, Local, Global và Mix.

Chế độ truy vấn cơ bản (Naive Query): Đây là phương pháp tìm kiếm truyền thống dựa trên độ tương đồng vector. Hệ thống truy xuất các đoạn văn bản (chunks) có bản nhúng vector gần nhất với câu hỏi mà không tận dụng cấu trúc đồ thị. Chế độ này phù hợp cho các câu hỏi đơn giản, yêu cầu trích xuất thông tin trực tiếp từ một đoạn văn cụ thể.

Chế độ truy vấn cục bộ (Local Query): Chế độ này được tối ưu hóa cho các câu hỏi chi tiết về một khái niệm cụ thể (Low-level Query). Hệ thống sử dụng cơ chế tìm kiếm vector để xác định các thực thể liên quan nhất, sau đó mở rộng truy xuất các nút lân cận và các mối quan hệ trực tiếp trong đồ thị tri thức. Quy trình này cung cấp một ngữ cảnh tri thức tập trung, giúp mô hình ngôn ngữ trả lời chính xác các thuộc tính và mối

quan hệ vi mô của thực thể.

Chế độ truy vấn toàn cục (Global Query): Đối với các câu hỏi mang tính tổng quát cao (High-level Query), đòi hỏi sự hiểu biết về toàn bộ kho dữ liệu, chế độ toàn cục phát huy vai trò chủ đạo. Thay vì tìm kiếm đoạn văn rời rạc, hệ thống tận dụng cấu trúc đồ thị ở mức độ vĩ mô bằng cách tổng hợp các mô tả tóm tắt của thực thể và quan hệ quan trọng. Cách tiếp cận này giúp mô hình ngôn ngữ bao quát được các chủ đề lớn và đưa ra những nhận định mang tính hệ thống vượt qua giới hạn của sở ngữ cảnh thông thường.

Chế độ truy vấn hỗn hợp (Mix Query): Đây là sự kết hợp giữa tìm kiếm cục bộ và toàn cục. Chế độ Mix đồng thời truy xuất các chi tiết cụ thể từ đồ thị lân cận và các tóm tắt tri thức vĩ mô từ cấp độ toàn cục. Quy trình xử lý này đảm bảo câu trả lời vừa có độ chính xác của dữ liệu chi tiết, vừa có chiều sâu của lập luận hệ thống, đáp ứng tốt nhất cho các nhiệm vụ phân tích tri thức đa tầng.

3.2 Mô hình hóa trạng thái hệ thống (Agent State)

Hệ thống được thiết kế dưới dạng một đồ thị trạng thái, trong đó mỗi nút đại diện cho một tác vụ tính toán và các cạnh đại diện cho luồng điều khiển. Trạng thái của hệ thống được định nghĩa bởi tập hợp $S = \{Q, U, H, A, M, D, R_{data}, R_{draft}, E, F\}$. Trong đó, Q (Query) là câu hỏi đầu vào của người dùng, U (User ID) là mã định danh người dùng để cá nhân hóa dữ liệu, và H (Chat History) là lịch sử hội thoại ngắn hạn đã được tóm tắt. Các thành phần khác bao gồm A (Analysis) là kết quả phân tích ý định, M (Long-term Memory) là hồ sơ người dùng dài hạn, và D (Decision) là quyết định điều phối. Cuối cùng, R_{data} (Retrieved Data) là dữ liệu tri thức thu được, R_{draft} (Draft Response) là bản thảo câu trả lời, E (Evaluation) là kết quả đánh giá chất lượng và F (Final Response) là

câu trả lời cuối cùng trả về cho người dùng.

3.2.1 Kiến trúc các tác vụ (Agents)

Hệ thống bao gồm 6 thực thể Agent chuyên biệt. Đầu tiên, *Analyze Agent* sử dụng mô hình ngôn ngữ lớn để phân loại câu hỏi và xác định từ khóa, bản chất học thuật cùng ý định của người dùng. Tiếp theo, *Long-term Memory Agent* thực hiện quản lý hồ sơ người dùng bền vững trên MongoDB bằng cách trích xuất các sự kiện và chuyên ngành để cập nhật vào hồ sơ tóm tắt. *Orchestration Agent* đóng vai trò trung tâm điều khiển, sử dụng cơ chế phản hồi hai bước để quyết định chế độ truy xuất hoặc trả lời trực tiếp. *Generator Agent* chịu trách nhiệm tổng hợp thông tin từ dữ liệu truy xuất và ngữ cảnh lịch sử để tạo câu trả lời theo cấu trúc Markdown. Cuối cùng, *Evaluator Agent* kiểm soát chất lượng bằng cách đánh giá bản thảo dựa trên tính xác thực, sự đầy đủ và sự liên quan, từ đó kích hoạt vòng lặp tái truy xuất nếu cần thiết.

3.2.2 Quy trình điều phối và Luồng dữ liệu

Quy trình thực thi của hệ thống được mô tả qua các bước logic sau:

Bước 1: Phân tích và Tải ngữ cảnh. Khi nhận câu hỏi Q , hệ thống đồng thời thực hiện *Analyze Agent* để hiểu ý định và truy vấn MongoDB để lấy bộ nhớ dài hạn M .

Bước 2: Ra quyết định điều phối (Routing). Dựa trên kết quả phân tích, bộ điều phối sẽ thực hiện điều hướng theo hai kịch bản chính: nếu câu hỏi không mang tính học thuật hoặc thông tin đã có sẵn trong hồ sơ người dùng, yêu cầu sẽ được chuyển thẳng đến *Generator Agent* để phản hồi; ngược lại, nếu cần kiến thức chuyên sâu từ kho tri thức, hệ thống sẽ chuyển đến *Retriever* thông qua *LightRAG* để trích xuất ngữ cảnh phù hợp.

Bước 3: Vòng lặp phản biện (Refinement Loop). Đây là điểm mới trong kiến trúc đề xuất. Sau khi *Generator* tạo bản thảo, *Evaluator* sẽ

kiểm tra. Nếu phát hiện thiếu hụt thông tin (Knowledge Miss), hệ thống sẽ bổ sung từ khóa vào truy vấn và thực hiện truy xuất lại. Số lần lặp tối đa được giới hạn ($n = 1$) để đảm bảo độ trễ của hệ thống.

Bước 4: Cập nhật bộ nhớ dài hạn. Hệ thống áp dụng chiến lược cập nhật định kỳ. Cứ sau mỗi 6 lượt tương tác ($history_len \pmod 6 == 0$), *Long-term Memory Agent* sẽ được kích hoạt để nén và lưu trữ thông tin mới vào cơ sở dữ liệu, đảm bảo trợ lý ngày càng hiểu rõ lộ trình học tập của sinh viên.

3.2.3 Cơ chế truy xuất tri thức (LightRAG Integration)

Khác với RAG truyền thống dựa trên vector similarity, chúng tôi sử dụng LightRAG để truy xuất trên cấu trúc đồ thị tri thức (Knowledge Graph). Truy vấn gửi đến LightRAG được làm giàu bằng cách kết hợp:

$$Q_{RAG} = Q + \text{Keywords} + \text{Refinement Critique}$$

Điều này giúp hệ thống khắc phục hiện tượng mất ngữ cảnh khi câu hỏi của sinh viên quá ngắn hoặc mơ hồ.

3.2.4 Thuật toán điều phối (Pseudocode)

Algorithm 1 Quy trình điều phối Agent thông minh

Require: $query, user_id$ $\triangleright$ Đầu vào từ sinh viên HCMUS

Ensure: $FinalResponse$ $\triangleright$ Câu trả lời đã qua kiểm định

```
1:  $state.analysis \leftarrow \text{AnalyzeAgent}(query)$ 
2:  $state.memory \leftarrow \text{LoadFromMongo}(user\_id)$ 
3:  $decision \leftarrow \text{OrchestrationAgent}(state.analysis, state.memory)$ 
4:  $refinement\_count \leftarrow 0$ 
5: repeat
6:   if  $decision.type == \text{"retrieve"}$  then
7:      $query_{rag} \leftarrow query + refinement\_critique$ 
8:      $state.data \leftarrow \text{LightRAG}(query_{rag}, decision.mode)$ 
9:   else
10:     $state.data \leftarrow \text{InternalKnowledge}$ 
11:     $state.draft \leftarrow \text{Generator}(query, state.data)$ 
12:     $eval \leftarrow \text{Evaluator}(query, state.draft, state.data)$ 
13:    if  $eval.score < 8$  then
14:       $refinement\_critique \leftarrow eval.critique$ 
15:       $refinement\_count \leftarrow refinement\_count + 1$ 
16: until  $eval.score \geq 8$  or  $refinement\_count \geq 1$ 
17: return  $state.draft$ 
```

Chương 4

Thiết kế và xây dựng ứng dụng

4.1 Ý tưởng ứng dụng

Sinh viên Khoa Công nghệ Thông tin tại Trường Đại học Khoa học Tự nhiên (HCMUS) đang phải đối mặt với một khối lượng thông tin học vụ lớn và phân tán: từ chương trình đào tạo, đề cương chi tiết từng môn học, quy chế học vụ đến mạng lưới môn tiên quyết và song hành. Để tìm được câu trả lời cho một câu hỏi đơn giản như “Mình cần học gì trước khi học Trí tuệ Nhân tạo?”, sinh viên thường phải lần lượt tra cứu qua nhiều tệp PDF rời rạc hoặc chờ đợi giờ tư vấn của giảng viên.

Ý tưởng trung tâm của hệ thống là xây dựng một **trợ lý học tập thông minh** có khả năng hiểu sâu về toàn bộ kho tri thức học vụ của Khoa. Hệ thống không chỉ trả lời các câu hỏi đơn lẻ mà còn hiểu được mối liên hệ logic giữa các môn học, chuyên ngành và yêu cầu tốt nghiệp, từ đó đưa ra lời tư vấn cá nhân hóa và có căn cứ rõ ràng.

4.1.1 Các tình huống sử dụng tiêu biểu

Tình huống 1: Tra cứu lộ trình môn học. Sinh viên năm hai đang lập kế hoạch học tập đặt câu hỏi: “*Mình muốn học môn Trí tuệ Nhân tạo thì cần học những môn gì trước?*”. Hệ thống AI Agent phân tích câu hỏi, truy xuất đồ thị tri thức để xác định chuỗi môn tiên quyết, sau đó trả lời chi

tiết kèm mã môn học và lộ trình học kỳ gợi ý.

Tình huống 2: Kiểm tra điều kiện tốt nghiệp. Sinh viên năm bốn hỏi: *“Mình còn thiếu những gì để đủ điều kiện xét tốt nghiệp?”*. Hệ thống đối chiếu hồ sơ học tập cá nhân được lưu trong bộ nhớ dài hạn với quy định chuẩn đầu ra của Khoa, liệt kê các môn còn thiếu và đưa ra gợi ý đăng ký cho học kỳ tiếp theo.

Tình huống 3: Hỏi đáp về quy chế học vụ. Sinh viên thắc mắc: *“Nếu mình trượt môn tiên quyết thì kỳ sau có được đăng ký môn tiếp theo không?”*. Hệ thống tra cứu quy chế đào tạo tín chỉ của trường, trả lời chính xác với đoạn trích dẫn từ văn bản quy chế tương ứng.

4.1.2 Đối tượng người dùng mục tiêu

Hệ thống hướng tới hai nhóm người dùng chính: **Sinh viên** Khoa Công nghệ Thông tin tại Trường Đại Học Khoa Học Tự Nhiên, những người cần tra cứu thông tin học vụ và lập kế hoạch học tập.

4.2 Phân tích và đặc tả yêu cầu

4.2.1 Yêu cầu chức năng

Hệ thống được xác định cần đáp ứng các nhóm chức năng chính được liệt kê trong Bảng 4.1.

Bảng 4.1: Danh sách yêu cầu chức năng

STT	Nhóm chức năng	Mô tả
1	Xác thực người dùng	Đăng ký tài khoản, đăng nhập bằng email/mật khẩu, khôi phục mật khẩu qua email.
2	Chat với AI Agent	Đặt câu hỏi học vụ bằng ngôn ngữ tự nhiên, nhận câu trả lời có nguồn gốc từ đồ thị tri thức, xem lại lịch sử hội thoại theo phiên.

4.2.2 Yêu cầu phi chức năng

Bảng 4.2: Yêu cầu phi chức năng

STT	Yêu cầu	Ý nghĩa / Mô tả
1	Hiệu suất	Giao diện AI Chat phải đảm bảo tải trong vòng ≤ 2 giây để đảm bảo trải nghiệm người dùng.
2	Khả năng mở rộng	Hệ thống phải mở rộng được để xử lý lượng lớn người dùng và phiên hội thoại đồng thời.
3	Khả dụng	Hệ thống phải hoạt động liên tục với up-time $\geq 99.9\%$ /năm.
4	Đáp ứng	Các thao tác cơ bản như đăng nhập, AI Chat phản hồi nhanh (dưới 1 giây).
5	Khả năng sử dụng	UI/UX đơn giản, dễ hiểu, dễ đặt câu hỏi AI.

4.2.3 Danh sách Actor

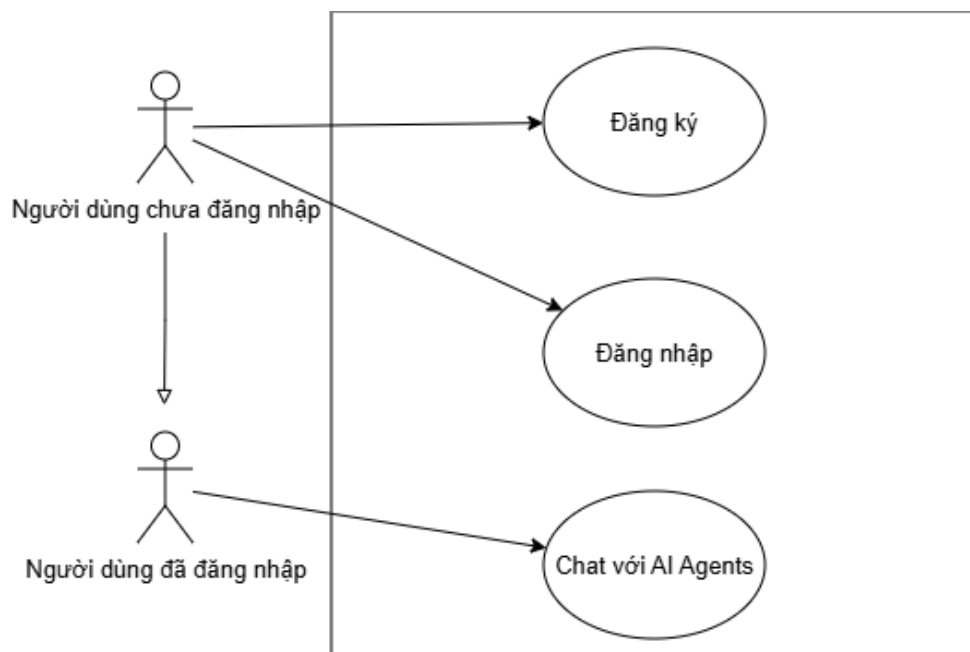
Hệ thống xác định một Actor chính tương tác trực tiếp với toàn bộ các chức năng:

Bảng 4.3: Danh sách Actor

STT	Actor	Ý nghĩa / Ghi chú
1	Người dùng	Sinh viên trực tiếp sử dụng hệ thống, thao tác các chức năng và gửi phản hồi.

4.3 Mô hình Use Case

Hệ thống có một phân hệ chính là **Sinh viên**, tương tác với hệ thống qua ba chức năng: đăng ký tài khoản, đăng nhập và chat với AI Agent. Sơ đồ Use Case tổng quan được thể hiện tại Hình 4.1.



Hình 4.1: Sơ đồ Use Case hệ thống

4.3.1 Danh sách Use Case

Bảng 4.4: Danh sách Use Case

STT	Tên Use Case	Mô tả	Actor
1	Đăng ký tài khoản	Sinh viên tạo tài khoản mới trên hệ thống.	Sinh viên
2	Đăng nhập	Sinh viên đăng nhập bằng email và mật khẩu.	
3	Chat với AI Agent	Sinh viên đặt câu hỏi học vụ và nhận tư vấn thông minh từ AI.	

4.3.2 Đặc tả Use Case

4.3.2.1 Use Case: Đăng ký tài khoản

Tên use-case	Đăng ký tài khoản
Tóm tắt	Cho phép sinh viên tạo tài khoản mới để sử dụng hệ thống.
Tác nhân	Sinh viên
Use-case liên quan	Đăng nhập
Điều kiện tiên quyết	Email chưa được đăng ký trong hệ thống.

Dòng sự kiện chính	1. Sinh viên truy cập trang đăng ký. 2. Sinh viên nhập họ tên, email và mật khẩu. 3. Sinh viên xác nhận mật khẩu và đồng ý điều khoản. 4. Sinh viên nhấn nút <i>Đăng ký</i>. 5. Hệ thống kiểm tra hợp lệ và tạo tài khoản. 6. Hệ thống chuyển hướng sang trang đăng nhập.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 2, email đã tồn tại: Hệ thống hiển thị thông báo lỗi và yêu cầu sinh viên sử dụng email khác.
Hậu điều kiện	Tài khoản được tạo thành công; sinh viên có thể đăng nhập vào hệ thống.

4.3.2.2 Use Case: Đăng nhập

Tên use-case	Đăng nhập hệ thống
Tóm tắt	Cho phép sinh viên xác thực danh tính và truy cập vào hệ thống.
Tác nhân	Sinh viên

Use-case liên quan	Đăng ký tài khoản, Chat với AI Agent
Điều kiện tiên quyết	Sinh viên đã có tài khoản được đăng ký trong hệ thống.
Dòng sự kiện chính	1. Sinh viên truy cập trang đăng nhập. 2. Sinh viên nhập email và mật khẩu. 3. Sinh viên nhấn nút <i>Đăng nhập</i>. 4. Hệ thống xác thực thông tin đăng nhập. 5. Hệ thống cấp phát JWT token và chuyển hướng đến trang chính.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 4, thông tin không hợp lệ: Hệ thống hiển thị thông báo lỗi và yêu cầu sinh viên nhập lại. A2. Quên mật khẩu: Sinh viên chọn <i>Quên mật khẩu</i>, hệ thống gửi email chứa liên kết khôi phục.
Hậu điều kiện	Sinh viên đăng nhập thành công và có thể sử dụng đầy đủ chức năng hệ thống.

4.3.2.3 Use Case: Chat với AI Agent

Tên use-case	Chat với AI Agent
---------------------	-------------------

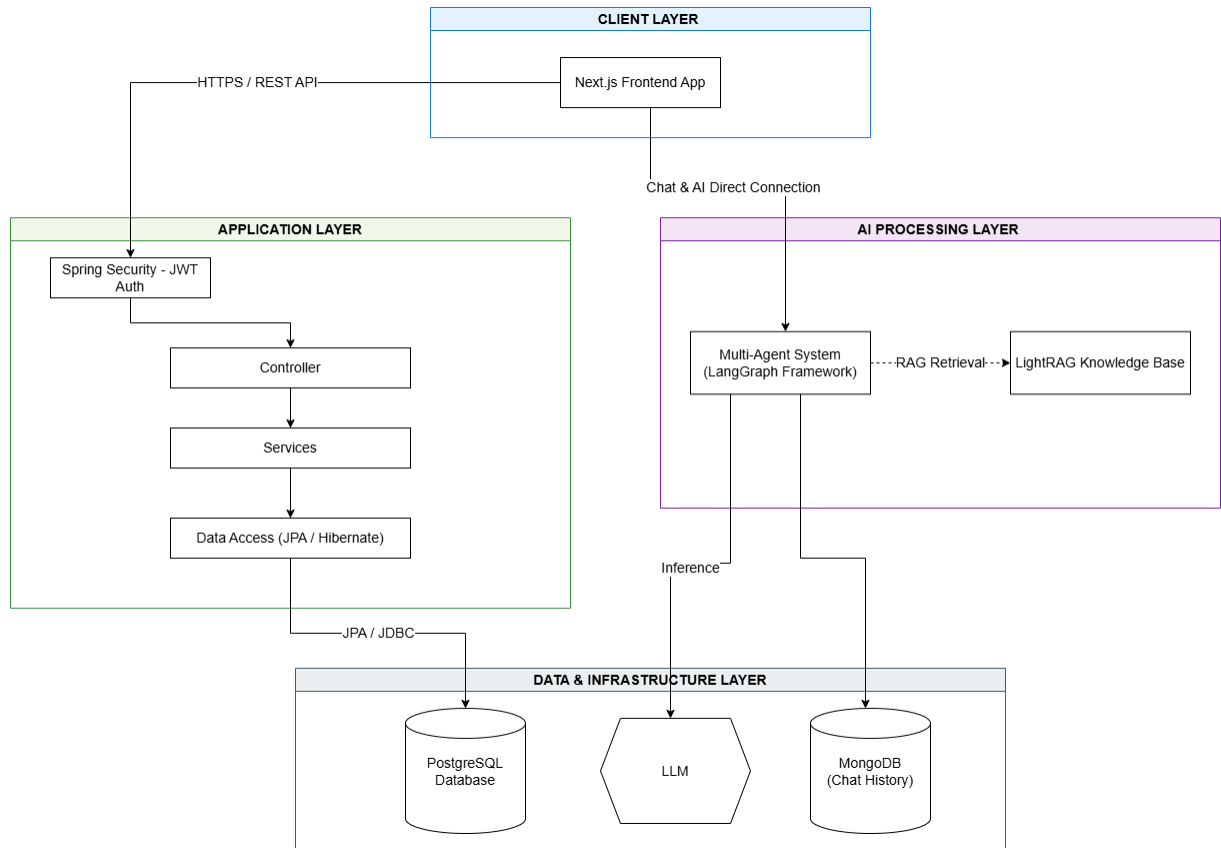
Tóm tắt	Sinh viên đặt câu hỏi về học vụ và nhận câu trả lời thông minh từ AI Agent.
Tác nhân	Sinh viên
Use-case liên quan	Đăng nhập
Điều kiện tiên quyết	Sinh viên đã đăng nhập, dịch vụ AI Agent đang hoạt động.
Dòng sự kiện chính	1. Sinh viên điều hướng đến phần AI Chat. 2. Sinh viên nhập câu hỏi liên quan đến học vụ. 3. Hệ thống chuyển tiếp câu hỏi đến hệ thống AI Agents. 4. AI Agent phân tích, truy xuất đồ thị tri thức và tạo phản hồi. 5. Hệ thống hiển thị câu trả lời trong giao diện chat. 6. Sinh viên có thể đặt câu hỏi tiếp theo, lặp lại bước 2–5.

Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 5, dịch vụ AI không khả dụng: Hệ thống hiển thị thông báo lỗi và gợi ý sinh viên thử lại sau. A2. Câu hỏi ngoài phạm vi: AI phản hồi rằng câu hỏi nằm ngoài phạm vi hỗ trợ của hệ thống.
Hậu điều kiện	Sinh viên nhận được câu trả lời, cuộc hội thoại được lưu vào lịch sử phiên.

4.4 Kiến trúc hệ thống

Mô hình kiến trúc tổng thể của hệ thống được đề xuất dựa trên kiến trúc đa tầng (Multi-tier Architecture), nhằm đảm bảo tính tách biệt giữa giao diện người dùng, logic nghiệp vụ và các phân hệ xử lý trí tuệ nhân tạo. Sơ đồ chi tiết được thể hiện tại Hình 4.2.

Sơ lược về kiến trúc được nhóm sử dụng như sau: hệ thống được tổ chức theo mô hình đa tầng gồm bốn tầng chính, trong đó Tầng Giao diện tiếp nhận yêu cầu từ sinh viên và phân phối sang hai luồng xử lý độc lập, Tầng Nghiệp vụ xử lý logic xác thực và nghiệp vụ, Tầng Điều phối AI đảm nhận toàn bộ xử lý trí tuệ nhân tạo, và Tầng Dữ liệu cung cấp hạ tầng lưu trữ chuyên biệt cho từng loại dữ liệu.



Hình 4.2: Mô hình kiến trúc hệ thống tổng thể

4.4.1 Tầng Giao diện (Client Layer)

Tầng Giao diện được xây dựng trên Next.js, là điểm tiếp xúc duy nhất giữa sinh viên và toàn bộ hệ thống. Khi sinh viên sử dụng ứng dụng, tầng này vận hành hai luồng giao tiếp song song tùy theo tính năng được yêu cầu.

Đối với các tác vụ liên quan đến tài khoản và thông tin cá nhân, ứng dụng giao tiếp với Tầng Nghiệp vụ thông qua giao thức HTTPS và REST API. Đây là luồng truyền thống, đảm bảo tính nhất quán và bảo mật cho các thao tác đọc ghi dữ liệu có cấu trúc.

Đối với tính năng Chat tư vấn học vụ, ứng dụng thiết lập kết nối trực tiếp đến Tầng Điều phối AI (Chat & AI Direct Connection) mà không đi qua Tầng Nghiệp vụ trung gian. Thiết kế giúp giảm thiểu độ trễ và cho

phép hiển thị câu trả lời theo từng phần, giúp mang lại trải nghiệm hội thoại tự nhiên và mượt mà hơn cho người dùng.

4.4.2 Tầng Nghiệp vụ (Application Layer)

Tầng Nghiệp vụ được xây dựng trên Spring Boot, đảm nhận nhiệm vụ xử lý toàn bộ logic nghiệp vụ và kiểm soát luồng dữ liệu giữa các tầng trong hệ thống.

Mọi yêu cầu từ phía Client trước tiên đều phải đi qua lớp bảo mật Spring Security kết hợp với cơ chế xác thực JWT (JSON Web Token). Giúp kiểm tra tính hợp lệ của token và phân quyền truy cập trước khi cho phép yêu cầu đi tiếp vào bên trong hệ thống.

Sau khi xác thực thành công, yêu cầu được định tuyến đến Controller rồi chuyển xuống lớp Services để thực thi logic nghiệp vụ. Lớp Data Access sử dụng Spring Data JPA và Hibernate để ánh xạ đối tượng và giao tiếp với cơ sở dữ liệu PostgreSQL thông qua giao thức JPA/JDBC.

4.4.3 Tầng Điều phối AI (AI Processing Layer)

Tầng Điều phối AI là trung tâm xử lý trí tuệ nhân tạo của hệ thống, tiếp nhận câu hỏi từ sinh viên qua luồng trực tiếp từ Client thông qua Chat & AI Direct Connection.

Khi nhận được câu hỏi, Multi-Agent System được xây dựng trên Lang-Graph Framework sẽ điều phối các tác nhân AI chuyên biệt để phân tích ý định câu hỏi, lập kế hoạch trả lời và tổng hợp kết quả. Trước khi sinh ra câu trả lời, hệ thống kích hoạt cơ chế RAG Retrieval để truy xuất các đoạn thông tin liên quan từ LightRAG Knowledge Base, kho tri thức học vụ đã được xây dựng và lập chỉ mục từ tài liệu của Khoa. Việc bổ sung ngữ cảnh từ tri thức thực tế giúp hạn chế tối đa hiện tượng “ảo giác” (hallucination) của mô hình ngôn ngữ, đảm bảo câu trả lời có căn cứ rõ ràng.

Sau khi có đầy đủ ngữ cảnh, hệ thống dùng LLM để tạo ra câu trả lời cuối cùng và trả về cho sinh viên.

4.4.4 Tầng Dữ liệu và Hạ tầng (Data & Infrastructure Layer)

Tầng này gồm ba thành phần chính, mỗi thành phần phục vụ một mục đích riêng biệt.

PostgreSQL được dùng để lưu thông tin tài khoản của sinh viên, bao gồm thông tin đăng nhập, phân quyền và hồ sơ cá nhân. Nhóm chọn PostgreSQL vì hỗ trợ tốt các ràng buộc toàn vẹn dữ liệu, phù hợp với yêu cầu bảo mật cao cho dữ liệu người dùng. Tầng Nghiệp vụ kết nối với PostgreSQL thông qua giao thức JPA/JDBC.

MongoDB lưu lịch sử hội thoại giữa sinh viên và AI Agent theo từng phiên. Do dữ liệu hội thoại không có cấu trúc cố định và cần truy xuất nhanh, MongoDB với cấu trúc tài liệu linh hoạt là lựa chọn phù hợp hơn so với cơ sở dữ liệu quan hệ.

LLM (Large Language Model) là thành phần thực hiện suy luận và sinh ra câu trả lời cuối cùng. Khi Tầng Điều phối AI đã thu thập đủ ngữ cảnh từ Knowledge Graph, yêu cầu Inference được gửi đến LLM để tổng hợp và trả lời cho sinh viên.

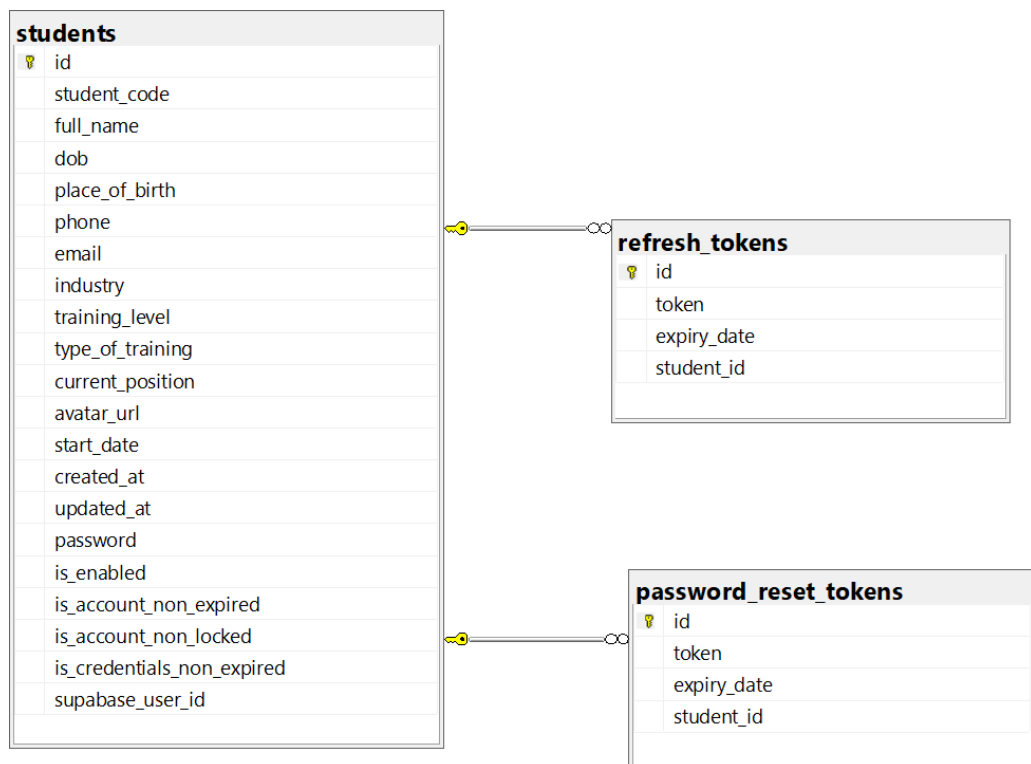
4.5 Thiết kế chi tiết các thành phần

Hệ thống sử dụng ba loại cơ sở dữ liệu với vai trò khác nhau: PostgreSQL lưu thông tin tài khoản người dùng, MongoDB lưu lịch sử hội thoại với AI Agent, và Knowledge Graph (LightRAG) lưu toàn bộ tri thức học vụ phục vụ cho tính năng tư vấn.

4.5.1 Thiết kế cơ sở dữ liệu

4.5.1.1 Cơ sở dữ liệu Quan hệ (PostgreSQL)

PostgreSQL được dùng để lưu thông tin tài khoản và hồ sơ người dùng trong hệ thống. Cấu trúc quan hệ giúp đảm bảo tính toàn vẹn dữ liệu và hỗ trợ phân quyền chặt chẽ. Sơ đồ tổng quan của cơ sở dữ liệu được thể hiện tại Hình 4.3.



Hình 4.3: Mô hình sơ đồ cơ sở dữ liệu quan hệ người dùng

Bảng 4.8: Danh sách các bảng trong cơ sở dữ liệu quan hệ

STT	Tên bảng	Ý nghĩa
1	students	Lưu toàn bộ thông tin hồ sơ người dùng: mã sinh viên, họ tên, ngày sinh, email, mật khẩu, ngành học, trình độ đào tạo và các cờ trạng thái tài khoản (is_enabled, is_account_non_locked,...).
2	refresh_tokens	Lưu token phiên làm việc (token, expiry_date) để duy trì trạng thái đăng nhập mà không cần xác thực lại.
3	password_reset_tokens	Lưu token bảo mật (token, expiry_date) dùng trong quy trình đặt lại mật khẩu.

Ràng buộc Khóa ngoại

Bảng 4.9: Mô tả khóa ngoại của các bảng trong CSDL quan hệ

STT	Tên Bảng	Tên cột	Bảng tham chiếu	Cột tham chiếu
1	refresh_tokens	student_id	students	id
2	password_reset_tokens	student_id	students	id

4.5.1.2 Cơ sở dữ liệu Phi cấu trúc (MongoDB)

Để phục vụ cho tác nhân duy trì bộ nhớ dài hạn (Long-term Memory Agent) và quản lý trạng thái luồng suy luận trong kiến trúc LangGraph, hệ thống sử dụng MongoDB. Dữ liệu được tổ chức linh hoạt để lưu trữ lịch sử hội thoại và các điểm kiểm soát (checkpoints) của AI Agent.

Bảng 4.10: Danh sách các Collection trong cơ sở dữ liệu hcmus_langgraph

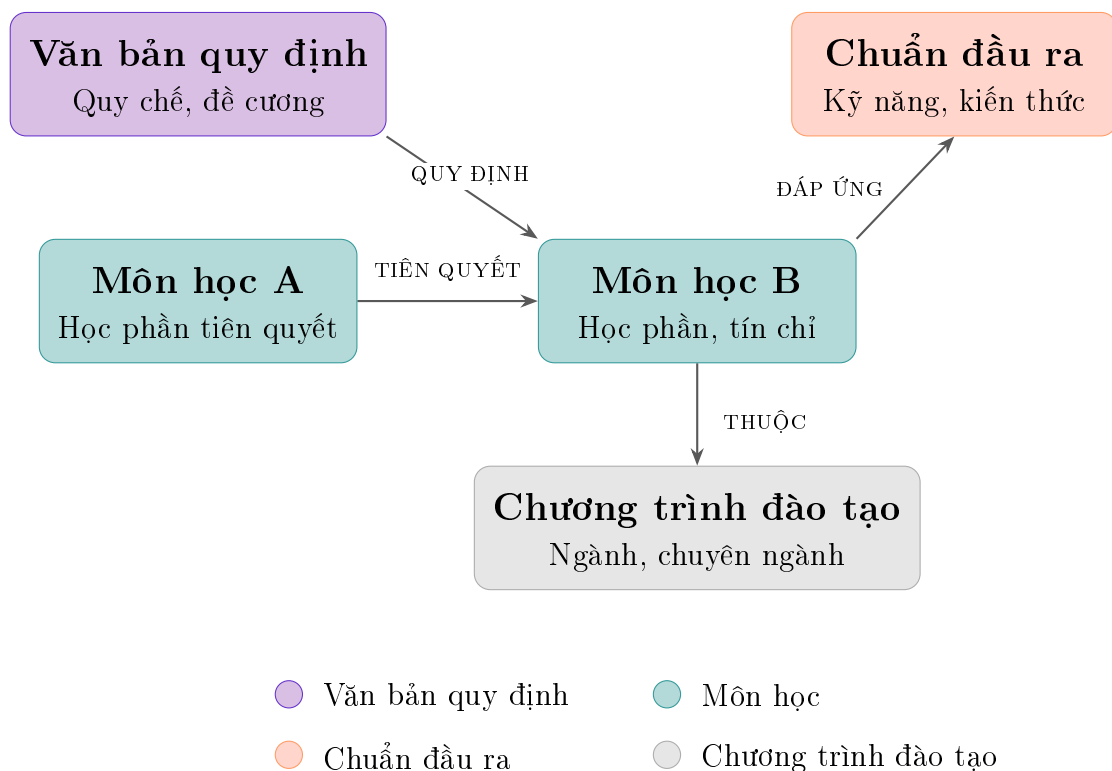
STT	Tên Collection	Ý nghĩa dữ liệu thực tế
1	session_info	Lưu trữ thông tin định danh của phiên chat bao gồm: session_id, user_id, tiêu đề cuộc hội thoại và thời gian cập nhật gần nhất.
2	long_term_memory	Lưu trữ các tri thức hoặc thông tin quan trọng được đúc kết từ các cuộc hội thoại cũ để AI Agent sử dụng cho các lần tương tác sau.
3	checkpoints	Lưu trữ trạng thái hiện tại (State) của luồng LangGraph, cho phép hệ thống phục hồi lại đúng bước đang xử lý khi có lỗi hoặc ngắt quãng.
4	checkpoint_writes	Ghi lại nhật ký các thay đổi của trạng thái Agent qua từng bước trong luồng điều phối.

4.5.1.3 Cơ sở dữ liệu Đồ thị (Knowledge Graph)

Hệ thống sử dụng LightRAG để xây dựng đồ thị tri thức học vụ, phục vụ lưu trữ thông tin phức tạp với nhiều mối quan hệ đan xen. Thay vì cấu trúc bảng (Table), dữ liệu được biểu diễn dưới dạng Thực thể (Nodes) và Mối quan hệ (Edges/Relationships), lưu trữ trong Neo4j.

Cấu trúc đồ thị tri thức được phân tích theo hai lớp: lớp khái niệm (logical view) thể hiện ngữ nghĩa các loại thực thể và quan hệ, và lớp lưu trữ vật lý (physical view) phản ánh cách triển khai thực tế trong Neo4j.

4.5.1.3.1 Lớp khái niệm (logical view) Hình 4.4 thể hiện mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ.



Hình 4.4: Mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ

Về mặt khái niệm, đồ thị gồm bốn nhóm thực thể chính. Môn học là thực thể trung tâm, lưu các học phần cùng thông tin tín chỉ, các môn

có thể liên kết tiên quyết với nhau qua quan hệ TIÊN QUYẾT. Chương trình đào tạo nhóm các môn học theo ngành và chuyên ngành qua quan hệ THUỘC. Chuẩn đầu ra mô tả các kỹ năng, kiến thức cần đạt qua quan hệ ĐÁP ỨNG. Văn bản quy định gồm quy chế và đề cương là nguồn gốc tri thức của hệ thống, liên kết qua quan hệ QUY ĐỊNH.

4.5.1.3.2 Lớp lưu trữ vật lý (physical view) Về mặt triển khai trong Neo4j, toàn bộ các quan hệ đều được lưu dưới một relationship type duy nhất là DIRECTED, ngữ nghĩa cụ thể được mã hóa trong thuộc tính keywords do mô hình ngôn ngữ lớn sinh ra trong quá trình trích xuất.

Mỗi thực thể được lưu dưới dạng một node với hai nhãn: nhãn *workspace* (phân vùng dữ liệu, ví dụ base) và nhãn *entity_type* (phân loại ngữ nghĩa, ví dụ Course, Concept). Bảng 4.11 liệt kê các thuộc tính của node.

Bảng 4.11: Thuộc tính của node thực thể trong Knowledge Graph (Neo4j)

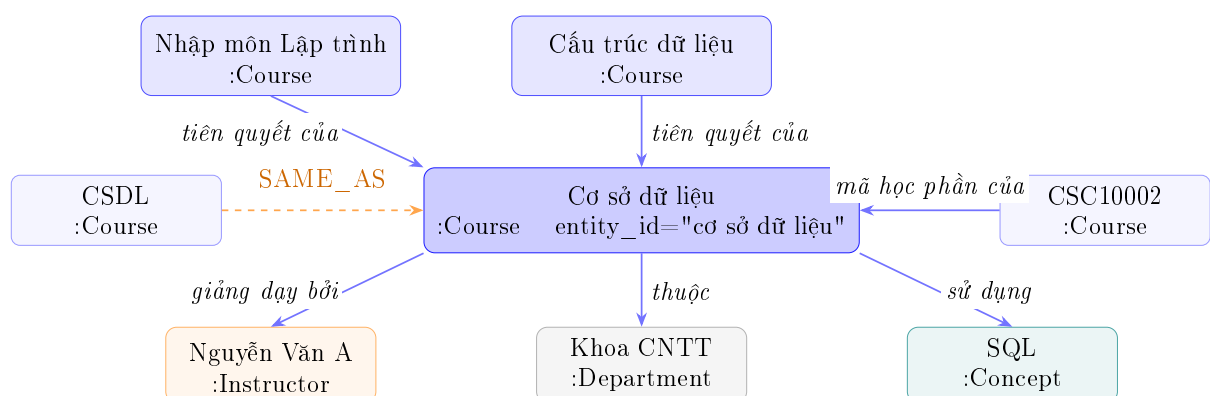
Thuộc tính	Kiểu	Mô tả & Ví dụ
entity_id	string	Tên thực thể (chuẩn hóa chữ thường). VD: "cơ sở dữ liệu"
entity_type	string	Loại thực thể do LLM phân loại. VD: "Course"
description	string	Mô tả tổng hợp từ nhiều tài liệu, các đoạn nối bằng <SEP>. VD: "Môn học về quản lý dữ liệu<SEP>Nghiên cứu SQL"
source_id	string	Danh sách ID chunk nguồn, nối bằng <SEP>. VD: "chunk-abc123<SEP>chunk-def456"
file_path	string	Tài liệu gốc chứa thực thể, nhiều giá trị nối bằng <SEP>. VD: "chuong_trinh.pdf<SEP>decuong.pdf"
created_at	integer	Unix timestamp thời điểm tạo. VD: 1719571200
truncate	string	Trạng thái giới hạn source_id: "" hoặc "KEEP Old"

Trường hợp đặc biệt là quan hệ đồng nhất thực thể, được biểu diễn bằng cùng type DIRECTED nhưng với keywords = "SAME_AS". Bảng 4.12 mô tả các thuộc tính của relationship.

Bảng 4.12: Thuộc tính của relationship trong Knowledge Graph (Neo4j)

Thuộc tính	Kiểu	Mô tả & Ví dụ
keywords	string	Từ khóa mô tả bản chất quan hệ, do LLM sinh ra. VD: "tiên quyết, học trước" hoặc "SAME_AS"
description	string	Diễn giải quan hệ theo ngữ cảnh tài liệu, nhiều mô tả nối bằng <SEP>. VD: "Nhập môn Lập trình là môn tiên quyết của Cơ sở dữ liệu"
weight	float	Trọng số quan hệ, tỉ lệ với số lần quan hệ xuất hiện. VD: 3.0
source_id	string	ID các chunk nguồn hỗ trợ quan hệ này. VD: "chunk-abc123"
file_path	string	Tài liệu gốc của quan hệ. VD: "chuong_trinh.pdf"
created_at	integer	Unix timestamp thời điểm tạo. VD: 1719571200

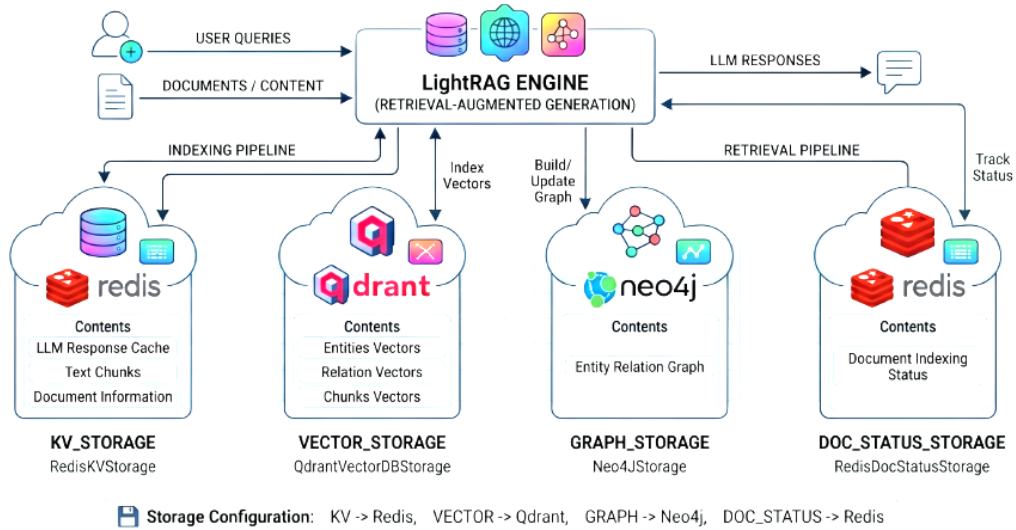
Hình 4.5 minh họa một đoạn đồ thị tri thức được trích xuất từ đề cương môn học, trong đó Cơ sở dữ liệu là thực thể canonical, CSDL và CSC10002 là alias liên kết qua quan hệ SAME_AS, các mũi tên thể hiện quan hệ ngữ nghĩa với nhãn keywords tương ứng.



Hình 4.5: Ví dụ subgraph trích xuất từ đề cương môn học trong Knowledge Graph

4.5.2 Kiến trúc triển khai lưu trữ cho LightRAG

Để vận hành hiệu quả các quy trình lập chỉ mục và truy xuất tri thức, hệ thống LightRAG được triển khai với kiến trúc lưu trữ hỗn hợp gồm bốn thành phần chuyên biệt. Sơ đồ tổng quan được thể hiện tại Hình 4.6.



Hình 4.6: Tổng quan kiến trúc lưu trữ và triển khai LightRAG

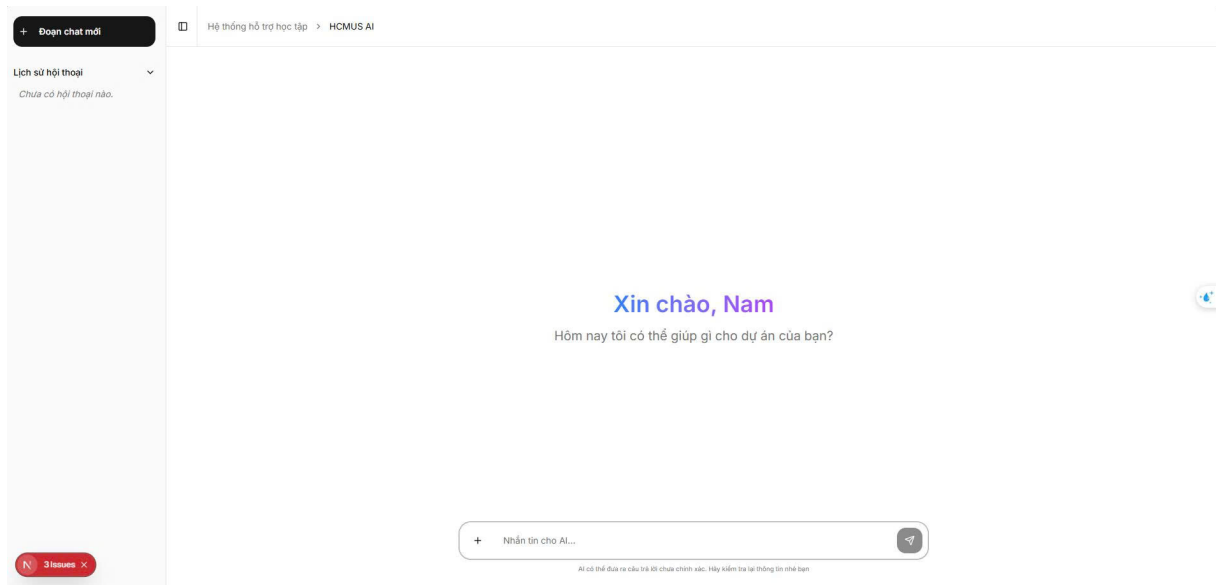
Thành phần thứ nhất là **KV_STORAGE**, triển khai trên Redis, chịu trách nhiệm lưu trữ bộ nhớ đệm phản hồi LLM, các đoạn văn bản đã phân mảnh và siêu dữ liệu tài liệu gốc. Thành phần thứ hai là **VECTOR_STORAGE**, triển khai trên Qdrant, lưu trữ vector nhúng của thực thể, quan hệ và đoạn văn bản để phục vụ tìm kiếm ngữ nghĩa. Thành phần thứ ba là **GRAPH_STORAGE**, triển khai trên Neo4j, lưu toàn bộ cấu trúc đồ thị tri thức với ngôn ngữ truy vấn Cypher. Thành phần thứ tư là **DOC_STATUS_STORAGE**, triển khai trên Redis, theo dõi trạng thái lập chỉ mục từng tài liệu để tránh xử lý trùng lặp.

Bảng 4.13: Các thành phần lưu trữ trong kiến trúc LightRAG

Thành phần	Công nghệ	Lớp triển khai	Dữ liệu lưu trữ
KV_STORAGE	Redis	RedisKVStorage	LLM Response Cache, Text Chunks, Document Information
VECTOR_STORAGE	Qdrant	QdrantVectorDBStorage	Entities Vectors, Relation Vectors, Chunks Vectors
GRAPH_STORAGE	Neo4j	Neo4JStorage	Entity Relation Graph
DOC_STATUS_STORAGE	Redis	RedisDocStatusStorage	Document Indexing Status

4.6 Thiết kế giao diện

4.6.1 Màn hình AI Chat



Hình 4.7: Giao diện Màn hình AI Chat

Mục đích: Cung cấp không gian trò chuyện trực tiếp giữa sinh viên và AI Agent để được tư vấn thông tin học vụ, lộ trình môn học và quy chế đào tạo.

Bao gồm:

1. Thanh lịch sử hội thoại (bên trái)
2. Khu vực hiển thị hội thoại (trung tâm)
3. Ô nhập câu hỏi và nút gửi (phía dưới)

Cách sử dụng:

- **Để đặt câu hỏi cho AI Agent:**

1. Nhập câu hỏi vào ô nhập liệu ở phía dưới màn hình.

2. Nhấn nút gửi.
3. Hệ thống phân tích câu hỏi, truy xuất đồ thị tri thức và trả về câu trả lời chi tiết trong khu vực hội thoại.

- **Để xem lại lịch sử hội thoại:**

1. Chọn phiên hội thoại tại thanh danh sách bên trái.
2. Hệ thống hiển thị toàn bộ nội dung của phiên đó.

Chương 5

Kết quả thí nghiệm

5.1 Đánh giá chất lượng MinerU trên tập tài liệu đầu vào

Trong kiến trúc hệ thống đề xuất, chất lượng của pha tiền xử lý tài liệu phi cấu trúc có ảnh hưởng trực tiếp đến độ chính xác và toàn vẹn của đồ thị tri thức. Nhằm đánh giá mức độ tin cậy của bộ giải mã MinerU, nghiên cứu đã tiến hành thực nghiệm trích xuất trên tập dữ liệu Chương trình đào tạo chuẩn của sáu ngành và khối ngành thuộc Khoa, bao gồm: Cử nhân tài năng, Công nghệ thông tin, Hệ thống thông tin, Khoa học máy tính, Kỹ thuật phần mềm, và Trí tuệ nhân tạo. Đặc biệt, để đảm bảo tính đại diện của tập mẫu, tài liệu được thu thập và phân rã theo từng khóa tuyển sinh từ năm 2022 đến năm 2025.

Phương pháp đánh giá được thực hiện thông qua việc đối sánh kết quả trích xuất tự động với nhãn chuẩn (Ground Truth) dựa trên các độ đo học thuật:

- CER (Character Error Rate) và WER (Word Error Rate): Tỷ lệ sai sót ở cấp độ ký tự và cấp độ từ. Giá trị càng thấp biểu thị khả năng nhận dạng quang học càng chính xác (ví dụ: Khoa học máy tính khóa 2022 đạt kết quả xuất sắc với CER chỉ 0.0019 và WER 0.0102).

- Edit Distance Sim (Độ tương đồng khoảng cách sửa đổi): Đo lường sự sai khác chuỗi tổng thể dựa trên số thao tác tối thiểu cần thiết để chuyển đổi kết quả sinh ra về nhãn chuẩn (ví dụ: hầu hết các tài liệu thuộc Hệ thống thông tin đều đạt trên 0.99, cho thấy độ chính xác gần như tuyệt đối).
- BLEU và METEOR: Các độ đo tiêu chuẩn trong dịch máy và sinh văn bản nhằm định lượng mức độ trùng khớp n-gram giữa văn bản trích xuất và văn bản gốc (ví dụ: CNTT khóa 2022 đạt BLEU lên tới 0.9928, bảo toàn hoàn hảo các thuật ngữ gốc).
- TEDS-Struct và TEDS-Content (Tree-Edit-Distance-based Similarity): Độ đo chuyên biệt dùng cho việc phân tích cấu trúc bảng biểu. TEDS-Struct đánh giá mức độ bảo toàn khung hàng/cột của bảng, trong khi TEDS-Content bao hàm cả tính chính xác của nội dung từng ô (ví dụ: ngoại trừ KTPM khóa 2024 sụt giảm còn 0.3342, phần lớn các khóa khác đều đạt TEDS-Struct 1.0000, giữ nguyên vẹn cấu trúc bảng).

Tập tài liệu	CER ↓	WER ↓	Edit Dist Sim ↑	BLEU ↑	METEOR ↑	TEDS-Struct ↑	TEDS-Content ↑
<i>Công nghệ thông tin (CNTT)</i>							
Khóa 2022	0.0008	0.0045	0.9992	0.9928	0.9963	1.0000	0.9999
Khóa 2023	0.1255	0.1571	0.8745	0.7743	0.8149	0.6055	0.5552
Khóa 2024	0.2166	0.1808	0.8151	0.8008	0.8475	0.7416	0.7051
Khóa 2025	0.0006	0.0035	0.9994	0.9926	0.9974	1.0000	0.9999
<i>Cử nhân tài năng (CNTN)</i>							
Khóa 2022	0.0023	0.0107	0.9977	0.9804	0.9912	0.9925	0.9924
Khóa 2023	0.1462	0.1247	0.8720	0.8714	0.8508	0.7298	0.6946
Khóa 2024	0.1643	0.1249	0.8539	0.8538	0.8696	0.9994	0.9632
Khóa 2025	0.0020	0.0100	0.9980	0.9805	0.9914	1.0000	0.9998
<i>Hệ thống thông tin (HTTT)</i>							
Khóa 2022	0.0009	0.0051	0.9991	0.9929	0.9958	1.0000	0.9999
Khóa 2023	0.0011	0.0056	0.9989	0.9892	0.9954	1.0000	0.9997
Khóa 2024	0.0070	0.0284	0.9930	0.9469	0.9736	1.0000	0.9995
Khóa 2025	0.0038	0.0202	0.9962	0.9617	0.9807	1.0000	0.9999
<i>Khoa học máy tính (KHMT)</i>							
Khóa 2022	0.0019	0.0102	0.9981	0.9845	0.9914	1.0000	0.9999
Khóa 2023	0.0018	0.0096	0.9982	0.9832	0.9920	1.0000	0.9997
Khóa 2024	0.0061	0.0188	0.9939	0.9676	0.9853	1.0000	0.9994
Khóa 2025	0.0014	0.0075	0.9986	0.9843	0.9935	0.9976	0.9936
<i>Kỹ thuật phần mềm (KTPM)</i>							
Khóa 2022	0.0012	0.0069	0.9988	0.9891	0.9943	1.0000	1.0000
Khóa 2023	0.0016	0.0084	0.9984	0.9843	0.9930	1.0000	0.9998
Khóa 2024	0.2633	0.2137	0.7760	0.8018	0.8069	0.3342	0.2291
Khóa 2025	0.0031	0.0167	0.9969	0.9678	0.9862	1.0000	0.9997
<i>Trí tuệ nhân tạo (TNTT)</i>							
Khóa 2022	0.0013	0.0074	0.9987	0.9876	0.9939	1.0000	1.0000
Khóa 2023	0.0014	0.0074	0.9986	0.9854	0.9939	1.0000	0.9998
Khóa 2024	0.2266	0.1660	0.8075	0.8117	0.8432	0.4277	0.3081
Khóa 2025	0.0109	0.0201	0.9891	0.9644	0.9849	0.4247	0.3373

Bảng 5.1: Kết quả chi tiết đánh giá MinerU trên các tập tài liệu Chương trình đào tạo (2022-2025)

Biện luận về độ quan trọng của các chỉ số: Đối với kiến trúc truy xuất tăng cường đồ thị (LightRAG) được sử dụng trong hệ thống, các chỉ số BLEU, TEDS-Struct, TEDS-Content và Edit Distance Sim đóng vai trò then chốt nhất. Lý do xuất phát từ đặc thù của tài liệu Chương trình đào

tạo, nơi thông tin trọng yếu về điều kiện tiên quyết, phân bổ tín chỉ và ma trận kỹ năng luôn được tổ chức dưới dạng bảng biểu phức tạp. Nếu mô hình bóc tách thất bại trong việc tái tạo cấu trúc bảng (thể hiện qua TEDS-Struct thấp), lượng tri thức này sẽ bị suy biến thành văn bản tuyến tính phi cấu trúc. Chỉ số BLEU cao minh chứng rằng văn bản được bóc tách bảo toàn chính xác các cụm từ (n-gram) từ tài liệu gốc, giúp mô hình ngôn ngữ lớn (LLM) không bị sai lệch ngữ cảnh. Hậu quả của việc điểm số suy giảm là LLM phía sau sẽ trích xuất sai các quan hệ (Relations) và thực thể (Entities), dẫn đến việc sinh ra các cạnh rác trong đồ thị tri thức. Song song đó, chỉ số Edit Distance Sim đảm bảo tính nguyên vẹn của các mã học phần và thuật ngữ kỹ thuật, tạo tiền đề bắt buộc để thuật toán đối soát và hợp nhất thực thể (Entity Deduplication) hoạt động ổn định và chính xác.

Tóm lại, dựa trên các thông số thu nghiệm được ở Bảng 5.1, bộ giải mã MinerU cho thấy khả năng duy trì cực kỳ tốt cấu trúc tài liệu phức tạp cũng như bảo toàn nội dung văn bản gốc ở đa số các ngành. Cụ thể, các chỉ số cốt lõi như TEDS-Struct và TEDS-Content đều duy trì ở mức hoàn hảo (1.0000) đối với nhóm ngành Hệ thống thông tin (HTTT) và Khoa học máy tính (KHMT), đi kèm với chỉ số Edit Distance Sim tiệm cận mức 1. Mặc dù ở một vài tài liệu cá biệt (như Kỹ thuật phần mềm khóa 2024 hay Trí tuệ nhân tạo khóa 2024) các chỉ số bóc tách bảng có dấu hiệu sụt giảm (TEDS-Struct dưới 0.45), nhưng tổng thể mức độ ổn định của toàn bộ tập dữ liệu vẫn ở mức rất cao. Có thể kết luận rằng, hệ thống bóc tách MinerU hoàn toàn đáp ứng được các yêu cầu khắt khe về tiền xử lý dữ liệu và là một công cụ xuất sắc, đủ tiêu chuẩn để ứng dụng làm đầu vào chất lượng cao cho luồng xử lý của hệ thống LightRAG.

5.2 Đánh giá chất lượng đồ thị tri thức

Hệ thống được triển khai thực nghiệm trên hạ tầng máy chủ đám mây nhằm đảm bảo khả năng xử lý các tác vụ phức tạp của Mô hình ngôn ngữ

lớn (LLM) [39] và cơ sở dữ liệu đồ thị [12]. Bộ dữ liệu kiểm thử (Benchmark Dataset) bao gồm 200 câu hỏi chuẩn được trích xuất từ các tình huống thực tế của sinh viên Khoa Công nghệ Thông tin liên quan đến quy chế đào tạo, sổ tay học vụ và lộ trình môn học.

5.2.1 Tổng quan và Lý do lựa chọn bộ tiêu chí đánh giá

Khi triển khai kiến trúc LightRAG [10] vào bài toán dữ liệu có tính hệ thống cao như Chương trình đào tạo, rủi ro lớn nhất là việc mô hình ngôn ngữ sẽ gặp "ảo giác" (Hallucination) hoặc thiết lập các chuỗi quan hệ rác, mất tính liên kết. Thay vì phụ thuộc 100% vào hệ thống học máy, việc nâng cấp sử dụng phương pháp phân tích tĩnh và động (Hybrid Assessment) cho việc đánh giá đồ thị mạng lưới [11] giúp mang lại khả năng đo đạc minh bạch.

Đồ thị tri thức sinh tự động từ LLM trên miền dữ liệu đặc thù tiếng Việt đặt ra ba thách thức kỹ thuật cụ thể: (i) phát hiện vi phạm ràng buộc cấu trúc mà không cần ontology cố định trước, (ii) đánh giá độ trung thực ngữ nghĩa của từng triple trên văn bản nguồn tiếng Việt chuyên ngành, và (iii) đo lường hiệu quả hợp nhất thực thể (entity deduplication) trong điều kiện nhãn ground-truth bị giới hạn. Ba thách thức này trực tiếp quy định bộ tiêu chí đánh giá được trình bày dưới đây [14].

Nhóm 1 — Đánh giá cấu trúc đồ thị (SHACL): Chuẩn SHACL của W3C [36] được lựa chọn vì là tiêu chuẩn duy nhất có thể kiểm tra ràng buộc cấu trúc đồ thị mà không yêu cầu tập dữ liệu nhãn thủ công. Hai phương án thay thế được xem xét là *OWL Reasoner* và *SPARQL-based validation*. OWL Reasoner có ưu điểm là hỗ trợ suy diễn logic phức tạp (ví dụ kế thừa lớp, đóng mở thế giới), nhưng yêu cầu bản thể học (ontology) phải được định nghĩa đầy đủ tường minh trước — điều không phù hợp với đồ thị tri thức được sinh tự động từ LLM với cấu trúc linh hoạt và chưa hoàn chỉnh. SPARQL-based validation có ưu điểm là rất linh hoạt

trong việc viết truy vấn kiểm tra, nhưng đòi hỏi một endpoint SPARQL riêng biệt và không có cơ chế báo cáo vi phạm chuẩn hóa, làm tăng chi phí vận hành hạ tầng. SHACL đã được kiểm chứng thực tiễn quy mô lớn trên Wikidata [19], một trong những đồ thị tri thức mở lớn nhất thế giới, chứng minh khả năng mở rộng và độ tin cậy ở quy mô thực tế.

Nhóm 2 — Đánh giá độ hữu cơ của quan hệ (LLM-as-a-Judge): Phương pháp sử dụng mô hình ngôn ngữ lớn làm trọng tài [41] được ưu tiên so với bốn phương án thay thế: (1) *Pattern matching* có ưu điểm là nhanh và rẻ, nhưng không hiệu quả với tiếng Việt chuyên ngành vì thiếu tập quy tắc được chuẩn hóa; (2) *Triple validation* qua cơ sở tri thức bên ngoài (Wikidata, ConceptNet) có ưu điểm là khách quan nhưng gần như bất khả thi với tri thức đặc thù của một khoa đại học trong nước — không có cơ sở dữ liệu nào chứa thông tin về chuẩn đầu ra CLO hay mã học phần nội bộ; (3) *KGEval* [25] — framework đánh giá KG thông qua các bài toán link prediction trên Knowledge Graph Embedding — cho kết quả tốt trên đồ thị chuẩn hóa như Freebase và DBpedia nhưng yêu cầu tập thực thể đầy đủ và nhãn quan hệ cố định trước, không khả thi với đồ thị sinh tự động có nhãn mở; (4) *GENIE* [17] — benchmark đánh giá chất lượng văn bản sinh kết hợp đánh giá tự động và con người — được thiết kế cho các tác vụ cấp câu như summarization và dialogue, không cung cấp cơ chế kiểm tra groundedness ở cấp độ từng triple riêng lẻ trong đồ thị tri thức.

Nhóm 3 — Đánh giá hợp nhất thực thể (V-Measure): Nhóm này được chú trọng đặc biệt vì bài toán deduplication trên đồ thị sinh tự động rất dễ gặp lỗi gộp nhầm các thực thể có tên tương đồng nhưng bản chất khác nhau. Sáu chỉ số được xem xét bao gồm các phương pháp truyền thống và các kỹ thuật hiện đại nhất (2023–2024):

- **MUC** (1995) [35]: Ưu điểm là đơn giản, nhược điểm là thiên vị cụm lớn (large-cluster bias). Không phù hợp vì gây đánh giá sai lệch trong deduplication.
- **B-Cubed** (1998) [4]: Giảm thiên vị cluster-size nhưng không phân

tách được lỗi gộp nhầm (over-merge) và bỏ sót gộp (under-merge).

- **CEAF** (2005) [24] và **ARI** (1985) [13]: Có độ chính xác cao nhưng yêu cầu ground-truth clustering đầy đủ — một rào cản quá lớn với dữ liệu sinh tự động quy mô nghìn thực thể.
- **Variation of Information (VI)** [26]: Một chỉ số dựa trên lý thuyết thông tin (entropy) hiện đại và mạnh mẽ, đo lường lượng thông tin bị mất hoặc tăng thêm giữa hai phân cụm. VI có ưu điểm là thỏa mãn các tính chất toán học của một "metric" (khoảng cách), nhưng nhược điểm là khó diễn giải giá trị tuyệt đối (không nằm trong khoảng $[0, 1]$ cố định) nên khó so sánh trực quan giữa các phiên bản đồ thị khác nhau.
- **Generalized Merge Distance (GMD)** [27]: Phương pháp hiện đại cho phép tùy chỉnh "chi phí" (cost) cho các lỗi split (chia nhỏ cụm) và merge (gộp nhầm). Đây là công cụ chẩn đoán cực kỳ mạnh mẽ, nhưng đòi hỏi phải thiết lập ma trận chi phí chủ quan, có thể gây thiếu khách quan trong báo cáo khoa học nếu không có cơ sở trọng số rõ ràng.

V-Measure [31] vượt qua các hạn chế trên nhờ cách tiếp cận dựa trên lý thuyết thông tin: đo đồng thời *homogeneity* (mỗi cụm chỉ chứa một loại thực thể) và *completeness* (mọi thực thể cùng loại vào cùng một cụm), tổng hợp thành điểm F tổng quát. Thiết kế này cho phép phân tích tách biệt lỗi over-merge và under-merge mà không cần ground-truth gán nhãn đầy đủ — chỉ cần tập mẫu kiểm tra ngẫu nhiên đủ lớn theo lý thuyết thống kê [18].

Tính bền vững và khả năng mở rộng (2024+): Mặc dù V-Measure ra đời năm 2007, nó vẫn là nền tảng cho các framework đánh giá tiên tiến nhất hiện nay. Khảo sát của Ji et al. [14] và các nghiên cứu về *Pointwise Clustering Metrics* từ Google (2024) [34] đều nhấn mạnh việc phân tách rõ rệt giữa tính đồng nhất (Homogeneity) và tính đầy đủ (Completeness)

là hướng đi đúng đắn để tối ưu hệ thống AI. V-Measure được chọn vì nó chuẩn hóa (normalize) các khái niệm entropy từ VI vào thang điểm $[0, 1]$ dễ hiểu, đồng thời không yêu cầu ground-truth toàn phần như ARI hay CEAF. Việc sử dụng V-Measure đảm bảo báo cáo thực nghiệm vừa mang tính học thuật hiện đại, vừa có khả năng chẩn đoán lỗi trích xuất thực thể từ LLM một cách minh bạch.

5.2.2 Kết quả đánh giá thực nghiệm

Tính chính xác và toàn vẹn của đồ thị tri thức quyết định trực tiếp khả năng suy luận của hệ thống. Nhóm thực hiện quy trình thẩm định sâu về ngữ nghĩa và cấu trúc thông qua ba trụ cột kỹ thuật chính kế thừa từ các nghiên cứu tiên tiến.

Phân hệ thứ nhất tập trung vào tình trạng cấu trúc đồ thị, hoàn toàn không sử dụng mô hình ngôn ngữ lớn mà ứng dụng bộ quy tắc kiểm định ràng buộc tĩnh kế thừa từ chuẩn ngôn ngữ SHACL của W3C [36]. Phương pháp này đã được ứng dụng rộng rãi trong việc làm sạch cấu trúc đồ thị Wikidata [19]. Các hạng mục rà soát bao gồm việc tìm kiếm các nút mồ côi thông qua duyệt đồ thị có hướng để phát hiện các đỉnh bậc không, đồng thời sử dụng phương pháp nhúng vector để phát hiện các thực thể trùng lặp dựa trên giới hạn độ tương đồng cosine [11], [23].

Phân hệ thứ hai tập trung vào việc đánh giá độ lắng đọng trích xuất bằng kỹ thuật LLM-as-a-Judge, lấy nền tảng từ quy trình kiểm duyệt chất lượng của kiến trúc GraphRAG [28]. Tiêu chí cốt lõi là độ có cơ sở (Groundedness), bắt buộc mỗi quan hệ phải tồn tại chứng cứ tường minh từ các đoạn văn bản gốc. Nhóm đã cải tiến cửa sổ ngữ cảnh lên tối đa 128,000 tokens để ngăn chặn hiện tượng âm tính giả.

Phân hệ thứ ba rà soát lỗi quên gộp thực thể dựa trên khung chuẩn ghép cặp thực thể trong cơ sở dữ liệu và liên kết bản ghi [18]. Hệ thống đo lường hiệu quả thông qua chỉ số V-Measure, vốn là chuẩn đánh giá cao cấp của Google Cloud Enterprise Knowledge Graph [11].

Để đo lường khách quan hiệu quả của các quy trình tối ưu hóa, nhóm sử dụng sáu chỉ số đánh giá cốt lõi:

- **Nút mồ côi (Dangling Nodes):** Là các đỉnh trong đồ thị không có bất kỳ mối quan hệ kết nối nào. Tỷ lệ này cao cho thấy tri thức bị phân mảnh, rời rạc.
- **Độ phủ văn bản (Chunk Coverage):** Đo lường khả năng trích xuất toàn vẹn tri thức từ dữ liệu thô. Chỉ số này phản ánh việc hệ thống có bỏ sót thông tin quan trọng trong tài liệu gốc hay không.
- **Độ có cơ sở (Groundedness):** Đảm bảo mọi quan hệ trích xuất đều có bằng chứng xác thực từ văn bản nguồn, ngăn chặn lỗi ảo giác thông tin của LLM.
- **Bỏ sót gộp (C1 Rate - Under-merge):** Tỷ lệ các thực thể cùng thực thể nhưng bị trích xuất thành nhiều nút khác nhau (ví dụ: "Khoa CNTT" và "Khoa Công nghệ Thông tin").
- **Độ thuần khiết (C2 Rate - Purity):** Đảm bảo các thực thể thực sự khác nhau không bị gộp chung vào một nút (tránh hiện tượng đồng nhất hóa sai lầm).
- **Overall V-Measure:** Chỉ số tổng quát đánh giá sự cân bằng giữa tính đầy đủ và tính chính xác của quy trình deduplication thực thể.

Nhóm đã lựa chọn mô hình **Qwen/Qwen2.5-72B-Instruct** làm trọng tài (Judge) cho quá trình đánh giá. Đây là dòng mô hình mã nguồn mở hàng đầu hiện nay, có khả năng tư duy logic (reasoning) và am hiểu sâu sắc ngữ pháp tiếng Việt, giúp phân tách được các khái niệm chuyên môn phức tạp mà các mô hình nhỏ thường nhầm lẫn.

Trong quá trình thực nghiệm trên bộ dữ liệu Chương trình đào tạo và Website thông tin của khoa, hệ thống đã xây dựng hai phiên bản đồ thị để đối soát chất lượng: Phiên bản gốc (Origin) và phiên bản tích hợp SHACL

(Tối ưu). Để đạt độ bảo vệ dữ liệu cấp doanh nghiệp, nhóm đã tích hợp các định lý từ Knowledge Vault về việc kết hợp phân tích đa nguồn [11]. Việc lựa chọn quy mô mẫu thử $N=500$ dựa trên lý thuyết hội tụ tri thức, đảm bảo bao phủ được các quan hệ hiếm gặp và đạt độ tin cậy thống kê 95% với sai số biên nhỏ.

Dưới đây là kết quả thực nghiệm đối soát toàn diện giữa kiến trúc Ligh-tRAG nguyên bản (Origin) và kiến trúc tối ưu (Tối ưu) tích hợp SHACL:

Chỉ số đánh giá	Cấp độ $N=10$		Cấp độ $N=200$		Cấp độ $N=500$	
	Origin	Tối ưu	Origin	Tối ưu	Origin	Tối ưu
Nút mồ côi (Dangling)	19.4%	5.2%	19.4%	5.2%	19.4%	5.2%
Độ phủ văn bản (Chunk)	36.1%	96.9%	36.1%	96.9%	36.1%	96.9%
Độ có cơ sở (Grounded)	0.910	0.914	0.990	1.000	0.986	0.998
Bỏ sót gộp (C1 Rate)	44.0%	6.0%	45.0%	7.0%	36.8%	8.0%
Độ thuần khiết (C2 Rate)	60.0%	70.0%	71.5%	78.0%	74.4%	78.7%
Overall V-Measure	57.9%	80.2%	62.2%	84.8%	68.3%	84.8%

Bảng 5.2: Tổng hợp kết quả đánh giá đồ thị tri thức trên các cấp độ lấy mẫu khác nhau

Phân tích kết quả thực nghiệm cho thấy sự cải thiện vượt bậc sau khi áp dụng quy trình tối ưu:

- **Giảm tỷ lệ nút mồ côi:** Tỷ lệ giảm mạnh từ **19.4% xuống còn 5.2%**. Kết quả này chứng minh rằng việc áp dụng SHACL và thuật toán duyệt đồ thị đã buộc các thực thể phải được kết nối vào ngữ cảnh chung, loại bỏ các "ốc đảo tri thức" cô lập, từ đó giúp câu trả lời của AI luôn có sự liên kết logic.
- **Đột phá về độ phủ văn bản:** Tăng từ **36.1% lên mức ấn tượng 96.9%**. Điều này có nghĩa là gần như toàn bộ thông tin quan trọng trong Chương trình đào tạo và Website của Khoa đã được hệ thống

"hấp thụ" vào đồ thị tri thức, không còn hiện tượng bỏ sót thông tin như phiên bản cũ.

- **Cải thiện chất lượng hợp nhất thực thể:** Chỉ số V-Measure tăng từ 60% lên 85%, cùng với việc tỷ lệ bỏ sót gộp (C1) giảm từ 44% xuống còn 8%. Điều này giải quyết bài toán nhức nhối khi một giảng viên hoặc một môn học bị trích xuất thành nhiều thực thể khác nhau, giúp đồ thị trở nên tinh gọn và chính xác hơn. Đặc biệt, các trường hợp chưa được hệ thống tự động nhận diện vẫn có thể được hợp nhất thủ công thông qua giao diện quản lý đồ thị, đảm bảo tính chuẩn xác tuyệt đối của tri thức.
- **Độ có cơ sở (Groundedness):** Luôn duy trì ở mức tiệm cận tuyệt đối (0.998 - 1.000), khẳng định sự kết hợp giữa ràng buộc cấu trúc SHACL và mô hình ngôn ngữ cấp chuyên gia đã loại bỏ hoàn toàn các quan hệ sai lệch, đảm bảo tri thức đạt chuẩn tin cậy học thuật [28].

5.3 Đánh giá chất lượng truy xuất và sinh văn bản (RAG Metrics)

Chất lượng sinh văn bản của hệ thống được đo lường khắt khe dựa trên khung đánh giá RAGAS (Retrieval-Augmented Generation Assessment) [7]. Các chỉ số đạt được trên bộ 200 câu hỏi chuẩn như sau:

- **Độ trung thực (Faithfulness):** Đạt mức 76.07%. Chỉ số này chứng minh hệ thống có khả năng bám sát ngữ cảnh từ Đồ thị tri thức, hạn chế tối đa việc AI bị ảo giác (Hallucination) và tự bịa đặt thông tin [15].
- **Độ liên quan của câu trả lời (Answer Relevance):** Đạt mức 88.76%. Câu trả lời sinh ra đáp ứng đúng trọng tâm truy vấn của sinh viên, vượt trội so với các phương pháp RAG truyền thống [22].

- **Độ gọi bối cảnh (Context Recall):** Đạt mức 76.01%. Nhờ cơ chế Graph-aware Retrieval lấy cảm hứng từ GraphRAG [6], hệ thống truy xuất thành công các khái niệm phân tán mà phương pháp Vector Search truyền thống thường bỏ sót.

5.4 Đánh giá hiệu năng điều phối Đa tác nhân (Agentic Metrics)

Việc tích hợp kiến trúc LangGraph mang lại sự linh hoạt nhưng cũng đặt ra bài toán về hiệu năng tính toán. Quá trình kiểm định ghi nhận các thông số sau:

- **Tỷ lệ điều phối đúng công cụ (Routing Accuracy):** Tác nhân Orchestrator đạt độ chính xác cao trong việc phân loại ý định người dùng và điều hướng đến đúng công cụ truy xuất (Local, Global hoặc Hybrid).
- **Tốc độ phản hồi (Average Latency):** Thời gian trung bình từ khi nhận câu hỏi đến khi xuất ra câu trả lời hoàn chỉnh là 7.32 giây.

5.5 Đánh giá hệ thống từ phía người dùng cuối

Quá trình thử nghiệm thực tế (Beta testing) được tiến hành với sự tham gia của các sinh viên Khoa Công nghệ Thông tin. Hệ thống đã giải quyết trơn tru 40 câu hỏi chuẩn, cung cấp lộ trình học tập logic, mở ra cơ hội ứng dụng mạnh mẽ các mô hình ngôn ngữ lớn vào giáo dục [16].

Đặc biệt, hệ thống xử lý hiệu quả các truy vấn phức tạp trên diễn đàn (Forum) chứa từ khóa lóng và văn phong tự do. Các câu hỏi vi phạm chính sách hoặc mang tính chất spam đều bị Tác nhân phân tích nhận diện và từ chối phản hồi ngay lập tức, đảm bảo môi trường học thuật lành mạnh.

Chương 6

Kết luận

6.1 Kết quả đạt được

Đề tài đã nghiên cứu và xây dựng thành công "Hệ thống hỗ trợ thông tin học tập cho sinh viên sử dụng AI Agents". Bằng việc kết hợp sức mạnh của Đồ thị tri thức [12] và kiến trúc điều phối Đa tác nhân [20], hệ thống đã góp phần giải quyết bài toán phân mảnh thông tin trong giáo dục đại học [40].

Những đóng góp chính của đề tài bao gồm:

- Số hóa và cấu trúc hóa thành công kho dữ liệu học vụ phức tạp thành Đồ thị tri thức (LightRAG) có khả năng suy luận nhanh chóng [8].
- Xây dựng luồng xử lý Đa tác nhân thông minh, cho phép AI tự động chọn lọc công cụ tìm kiếm phù hợp với từng dạng câu hỏi của sinh viên.
- Triển khai hệ thống đánh giá tự động thông minh (LLM-as-a-Judge) [41] kết hợp RAGAS [7] giúp đo lường và tinh chỉnh chất lượng câu trả lời một cách khách quan.

Nhìn chung, hệ thống đã giúp sinh viên dễ dàng tra cứu, thấu hiểu mối liên hệ giữa các môn học tiên quyết và lập kế hoạch học tập hiệu quả.

6.2 Hạn chế của hệ thống

Dù đạt được những kết quả khả quan, hệ thống vẫn còn tồn tại một số giới hạn về mặt kỹ thuật:

- **Độ trễ xử lý (Latency):** Do bản chất của việc duy trì hội thoại đa tác nhân [38] và suy luận phức tạp của mô hình LLM, tốc độ phản hồi của hệ thống vẫn còn tương đối chậm, đặc biệt khi có nhiều người dùng truy vấn đồng thời.
- **Hạn chế trong số hóa tài liệu tiếng Việt:** Việc bóc tách cấu trúc bảng biểu phức tạp từ các file PDF quy chế đào tạo đôi khi vẫn gặp lỗi nhận diện (OCR), dẫn đến việc Đồ thị tri thức bị khuyết dữ liệu ở một vài tiểu mục nhỏ.

6.3 Hướng phát triển trong tương lai

Qua quá trình nghiên cứu và thực hiện đề tài, nhóm đã đạt được các mục tiêu trọng tâm về lý thuyết, giải pháp và thực nghiệm. Về lý thuyết, nhóm xây dựng thành công quy trình đánh giá đồ thị tri thức đa tầng dựa trên các nghiên cứu về hệ thống RAG hiện đại [5], [10], [28]. Về giải pháp, việc đề xuất kiến trúc vùng đệm lưu trữ kết hợp với quy tắc SHACL đã tối ưu hóa cấu trúc đồ thị và đảm bảo tính liên kết chặt chẽ [36]. Thực nghiệm trên toàn bộ kho học liệu của khoa Công nghệ Thông tin đã minh chứng tính ổn định của hệ thống với kết quả vượt trội so với các kiến trúc truyền thống.

Trong thời gian tới, nhóm dự kiến hoàn thiện giao diện cho phép giảng viên trực tiếp tham gia phê duyệt thực thể, hướng tới mục tiêu đạt được độ chính xác tuyệt đối. Đồng thời, việc tự động hóa đồng bộ thuật ngữ Anh-Việt trong đề cương chi tiết sẽ được chú ý phát triển. Cuối cùng, một module gợi ý lộ trình học tập cá nhân hóa dựa trên đồ thị tri thức sư phạm sẽ được tích hợp để hỗ trợ sinh viên trong việc định hướng nghề nghiệp

và xây dựng kế hoạch học tập tối ưu. Để khắc phục các hạn chế hiện tại và đưa hệ thống vào ứng dụng thực tiễn ở quy mô toàn trường, các hướng nghiên cứu tiếp theo bao gồm:

- **Nâng cấp công cụ xử lý tài liệu:** Nghiên cứu tích hợp các mô hình chuyên biệt để bóc tách chính xác 100% các bảng biểu môn học từ file PDF.
- **Tối ưu hóa hiệu năng:** Triển khai kỹ thuật Semantic Caching nhằm lưu trữ lại các câu trả lời cho những câu hỏi thường gặp, giúp giảm thời gian phản hồi.
- **Mở rộng hệ sinh thái sinh viên:** Tích hợp trực tiếp hệ thống AI Agent vào cổng thông tin đào tạo của trường, cho phép AI tự động cập nhật lịch học, điểm số và gợi ý đăng ký môn học theo thời gian thực để hỗ trợ giáo dục thông minh toàn diện [16].

Danh mục công trình của tác giả

1. Tạp chí ABC
2. Tạp chí XYZ

Tài liệu tham khảo

Tiếng Anh

- [4] Bagga, A. and Baldwin, B., “Algorithms for scoring coreference chains,” in *Proceedings of the First International Conference on Language Resources and Evaluation Workshop on Linguistic Coreference*, 1998, pp. 563–566.
- [5] Dong, X. L. et al., “Knowledge-based trust: Estimating the trustworthiness of web sources,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1638–1649, 2015.
- [6] Edge, D. et al., “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [7] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S., “Ragas: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [8] Guo, H. et al., “Lightrag: Simple and fast retrieval-augmented generation,” *arXiv preprint arXiv:2410.05779*, 2024.
- [9] Gutiérrez, B. et al., “Hipporag2: Enhancing retrieval-augmented generation with composite knowledge graphs,” in *Advances in Neural Information Processing Systems*, 2025.
- [10] HKUDS. “Lightrag: Institutional memory and personal knowledge management with light-weight retrieval-augmented generation,” Ac-

cessed: Apr. 18, 2026. [Online]. Available: <https://github.com/HKUDS/LightRAG>

- [11] Hogan, A. et al., “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3447772>
- [12] Hogan, A. et al., “Knowledge graphs,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37, 2021.
- [13] Hubert, L. and Arabie, P., “Comparing partitions,” *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.
- [14] Ji, S., Pan, S., Cambria, E., Marttinen, P., and Yu, P. S., “A comprehensive study of knowledge graph construction and evaluation methods,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2022.
- [15] Ji, Z. et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [16] Kasneci, E. et al., “Chatgpt for good? on opportunities and challenges of large language models in education,” *Learning and individual differences*, vol. 103, p. 102 274, 2023.
- [17] Khashabi, D. et al., “Genie: Toward reproducible and standardized human evaluation for text generation,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2022.
- [18] Köpcke, H. and Rahm, E., “Frameworks for entity matching: A comparison,” *Data & Knowledge Engineering*, vol. 69, no. 2, pp. 197–210, 2010.
- [19] Labra Gayo, J. E. et al., “Formalizing and validating wikidata’s property constraints using shacl and sparql,” *ResearchGate*, 2024. [Online]. Available: <https://www.researchgate.net/publication/>

383382328_Formalizing_and_validating_Wikidata's_property_constraints_using_SHACL_and_SPARQL

- [20] LangChain, *Langgraph: Multi-actor workflows with llms*, <https://langchain-ai.github.io/langgraph/>, 2024. Accessed: May 1, 2024.
- [21] Lee, M. et al., “Hypergraphrag: N-ary relation retrieval for complex data,” *arXiv preprint*, 2025.
- [22] Lewis, P. et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [23] Li, J., Sun, A., Han, J., and Li, C., “A survey on deep learning for named entity recognition,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9039685>
- [24] Luo, X., “On coreference resolution performance metrics,” in *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing*, 2005, pp. 25–32.
- [25] Luo, Z. et al., “Kgeval: Evaluating knowledge graphs via knowledge graph embedding,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023.
- [26] Meilă, M., “Comparing clusterings by the variation of information,” in *Learning Theory and Kernel Machines*, Springer, 2003, pp. 173–187.
- [27] Menestrina, D., Whang, S. E., and Garcia-Molina, H., “Evaluating entity resolution results,” *Proceedings of the VLDB Endowment*, vol. 3, no. 1, pp. 208–219, 2010.

- [28] Microsoft. “Graphrag: A modular graph-based retrieval-augmented generation system,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/microsoft/graphrag>
- [29] Microsoft. “Markitdown: A tool for converting various document formats to markdown,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/microsoft/markitdown>
- [30] OpenDataLab. “Mineru: A high-quality tool for parsing mixed image and text documents,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/opendatalab/MinerU>
- [31] Rosenberg, A. and Hirschberg, J., “V-measure: A conditional entropy-based external cluster evaluation measure,” in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, Association for Computational Linguistics, 2007, pp. 410–420.
- [32] Shin, J., Park, C., Park, J., Seo, J., and Lim, H., “MultiDocFusion : Hierarchical and multimodal chunking pipeline for enhanced RAG on long industrial documents,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V., Eds., Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 20 985–21 004, ISBN: 979-8-89176-332-6. DOI: 10.18653/v1/2025.emnlp-main.1062 [Online]. Available: <https://aclanthology.org/2025.emnlp-main.1062/>
- [33] Smith, J. et al., “Og-rag: Ontology-grounded retrieval-augmented generation,” *arXiv preprint*, 2024.
- [34] Staden, S. van, *Pointwise metrics for clustering evaluation*, 2024. arXiv: 2405.10421 [cs.IR]. [Online]. Available: <https://arxiv.org/abs/2405.10421>

- [35] Vilain, M., Burger, J., Aberdeen, J., Connolly, D., and Hirschman, L., “A model-theoretic coreference scoring scheme,” in *Proceedings of the 6th Conference on Message Understanding*, 1995, pp. 45–52.
- [36] W3C. “Shapes constraint language (shacl),” Accessed: Apr. 18, 2026. [Online]. Available: <https://www.w3.org/TR/shacl/>
- [37] Wang, X. et al., “Pathrag: Pruning relational paths for efficient graph-based retrieval,” *arXiv preprint*, 2025.
- [38] Wu, Q. et al., “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [39] Xi, Z. et al., “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [40] Zawacki-Richter, O., Marín, V. I., Bond, M., and Gouverneur, F., “Systematic review of research on artificial intelligence applications in higher education—where are the educators?” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, pp. 1–27, 2019.
- [41] Zheng, L. et al., “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.

Phụ lục A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Phụ lục B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.